Generalizable Neural Solvers for Vehicle Routing Problems

Zhou Jianan
College of Computing and Data Science

A thesis submitted to the Nanyang Technological University
in partial fulfillment of the requirements for the degree of
Doctor of Philosophy

2025

Statement of Originality

I hereby certify that the work embodied in this thesis is the result of original research, is free of plagiarised materials, and has not been submitted for a higher degree to any other University or Institution.

07/03/2025

Date

Zhou Jianan

Supervisor Declaration Statement

I have reviewed the content and presentation style of this thesis and declare it is free of plagiarism and of sufficient grammatical clarity to be examined. To the best of my knowledge, the research and writing are those of the candidate except as acknowledged in the Author Attribution Statement. I confirm that the investigations were conducted in accord with the ethics policies and integrity standards of Nanyang Technological University and that the research data are presented honestly and without prejudice.

07/03/2025

Date

Prof. Zhang Jie

Authorship Attribution Statement

This thesis contains material from 3 papers published in the following peer-reviewed journals / from papers accepted at conferences in which I am listed as the first author.

Chapter 3 is published as **Jianan Zhou**, Yaoxin Wu, Zhiguang Cao, Wen Song, Jie Zhang, Zhiqi Shen. Collaboration! Towards Robust Neural Methods for Routing Problems. 38th Conference on Neural Information Processing Systems, 2024. The contributions of the co-authors are as follows:

- I proposed the method, conducted experiments and wrote the manuscript draft. Yaoxin Wu provided valuable discussions and guided the research direction. Zhiguang Cao, Wen Song, Jie Zhang, and Zhiqi Shen contributed to refining the manuscript.

Chapter 4 is published as **Jianan Zhou**, Yaoxin Wu, Wen Song, Zhiguang Cao, Jie Zhang. Towards Omni-generalizable Neural Methods for Vehicle Routing Problems. 40th International Conference on Machine Learning, 2023. The contributions of the co-authors are as follows:

- I proposed the method, conducted experiments and wrote the manuscript draft. Yaoxin Wu provided valuable discussions and guided the research direction. Zhiguang Cao, Wen Song, and Jie Zhang helped polish the manuscript.

Chapter 5 is published as **Jianan Zhou**, Zhiguang Cao, Yaoxin Wu, Wen Song, Yining Ma, Jie Zhang, Chi Xu. MVMoE: Multi-Task Vehicle Routing Solver with Mixture-of-Experts. 41st International Conference on Machine Learning, 2024. The contributions of the co-authors are as follows:

- I proposed the method, conducted experiments and wrote the manuscript draft. Yaoxin Wu provided valuable discussions and guided the research direction. Zhiguang Cao, Wen Song, Yining Ma, Jie Zhang, and Chi Xu contributed to refining the manuscript.

07/03/2025

Date

Zhou Jianan

Acknowledgements

I would like to express my deepest gratitude to all those who supported me throughout this PhD journey. First and foremost, I am profoundly grateful to my advisor, Prof. Zhang Jie, whose unwavering guidance, insightful feedback, and constant encouragement have been invaluable in shaping both this thesis and my academic career. My sincere appreciation also goes to my co-supervisor, Dr. Xu Chi, and my mentors, Prof. Wu Yaoxin, Prof. Cao Zhiguang, and Prof. Song Wen, for their invaluable discussions and generous assistance throughout my research. Additionally, I wish to thank the members of my dissertation committee for their constructive comments, which has significantly enriched this work.

I am deeply grateful to my friends who have accompanied me along the way over the past four years – Dr. Ma Yining, Dr. Li Shuo, Dr. Tian Haoxiang, Ms. Qu Ying, Dr. Sun Youchen, Ms. Bi Jieyi, Prof. Sun Jing, Dr. Fan Mingfeng, Ms. Chi Huijue, Dr. Zhang Cong, Ms. Xia Yunwen, et al. – for their companionship. I extend my sincere appreciation to my co-authors from the AI4CO community and institutions such as MIT, NUS, KAIST, TUE, and SMU. Their collaborative and insightful contributions have made this challenging journey both rewarding and enjoyable. Additionally, I wish to express my gratitude to my favourite singer, Mr. Zhou Shen. His angelic voice has been a constant source of strength and comfort throughout this journey.

Finally, I would like to express my heartfelt gratitude for the unwavering support of my family, especially my grandparents, Ms. Kang Zhiling and Mr. Hu Jinlong, and my parents, Ms. Hu Xiaobo and Mr. Zhou Zhongbo. Their endless encouragement, nurturing spirit, and steadfast belief in me have been the driving forces behind my journey. This achievement belongs to them as much as it does to me, and I am forever grateful for their unwavering faith in me.

Abstract

Vehicle routing problems (VRPs) are a fundamental class of combinatorial optimization problems (COPs) in computer science and operations research, with diverse real-world applications in logistics, transportation, and manufacturing. The intrinsic NP-hard nature makes VRPs exponentially expensive to be solved by exact solvers. As an alternative, heuristic solvers deliver suboptimal solutions within reasonable time but require extensive hand-crafted rules and domain expertise tailored to each specific problem. Recently, neural combinatorial optimization (NCO), which leverages machine learning to learn heuristics in a data-driven manner, has gained significant attention. These neural solvers demonstrate strong performance while reducing computational overhead and reliance on domain expertise compared to traditional solvers. However, they face significant generalization challenges. For example, their performance may degrade substantially when applied to instances with different data distributions, problem scales, or constraints than those encountered during training. This generalization issue severely limits their practical applicability. This thesis aims to systematically address these challenges and advance the development of generalizable neural solvers for VRPs.

The thesis begins by enhancing the cross-distribution generalization of neural VRP solvers through an adversarial training approach. We introduce an ensemble-based Collaborative Neural Framework (CNF) that adversarially trains multiple models in a collaborative manner, promoting robustness against adversarial attacks while also boosting performance on clean instances. In this framework, an attacker generates hard instances by perturbing the data distribution, challenging the models to adapt and thereby strengthening cross-distribution generalization. Additionally, a neural router is designed to efficiently distribute training instances among the models, thereby improving load balancing and collaborative efficacy. Extensive experiments on two classic routing problems – the travelling salesman problem (TSP) and the capacitated vehicle routing problem (CVRP) – demonstrate the effectiveness and versatility of CNF in enhancing the cross-distribution generalization.

Next, the thesis further expands neural solvers to simultaneously address both cross-size and cross-distribution generalization challenges. We propose Omni-VRP, a generic meta-learning framework that enables effective training of an initial model with the capacity for rapid adaptation to new tasks during inference, where each task corresponds to a set of instances with the same problem scale and data distribution. Additionally, we introduce a simple yet efficient approximation method to reduce the training overhead associated with second-order derivative calculations. Extensive experiments on both synthetic and benchmark instances of TSP and CVRP validate the effectiveness of our approach.

Finally, the thesis pioneers the study of cross-problem generalization, pushing the boundaries of neural VRP solvers in learning generalizable representations across diverse constraints. We propose MVMoE, a unified neural solver capable of solving 16 VRP variants simultaneously in a zero-shot manner through attribute composition. To efficiently enhance model capacity, we employ a mixture-of-experts architecture with a hierarchical gating mechanism, striking a balance between empirical performance and computational complexity. Experimentally, our method significantly promotes zero-shot generalization on 10 unseen VRP variants, and showcases decent results on the few-shot setting and real-world benchmark instances.

Overall, this thesis represents a pioneering exploration and substantial advancement in developing generalizable neural solvers for VRPs. It investigates innovative designs across various components, including model architectures, training algorithms, and problem settings, to enhance the generalization of neural solvers across diverse contexts such as varying data distributions, problem scales, and constraints. These contributions encourage neural solvers to learn more robust and generalizable representations, paving the way for the first generation of foundation models capable of addressing a wide range of COPs. Ultimately, the insights gained here not only propel the field of NCO forward but also enrich the broader landscape of learning-based optimization methods.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background

Combinatorial optimization problems (COPs) are a fundamental area of mathematical optimization, drawing significant attention in computer science and operations research. Among them, vehicle routing problems (VRPs) represent a critical class with diverse applications in logistics, transportation, and manufacturing. Due to their NP-hard nature, these problems are notoriously difficult to solve exactly, as their discrete solution spaces preclude the direct use of gradient-based optimization techniques. As problem size increases, the challenge of identifying optimal or near-optimal solutions among an exponentially expanding set of possibilities becomes even more intractable. Traditional approaches to solving VRPs fall into two main categories: exact and inexact solvers. Exact solvers, such as CPLEX [1], Gurobi [2], and SCIP [3], rely on branch-and-bound and its variants to guarantee optimal solutions. However, their exponential time complexity makes them impractical for large-scale problems. In contrast, inexact solvers, such as the Lin-Kernighan-Helsgaun (LKH) [4, 5] and hybrid genetic search (HGS) [6], can efficiently generate high-quality solutions within reasonable timeframes, yet they often struggle with large or complex instances. Furthermore, both exact and inexact solvers require extensive domain expertise and meticulous tuning, increasing development complexity and potentially limiting their overall effectiveness.

To address these limitations, neural combinatorial optimization (NCO) [7] has emerged as a promising data-driven approach to learning heuristics. In general,

these neural VRP solvers leverage advanced deep learning models (e.g., pointer network [8], attention mechanism [9] and graph neural network [10]) to learn heuristics for route construction or improvement with supervised learning (SL) or reinforcement learning (RL). By capturing underlying representations across a set of training instances, these models are capable of autonomously discovering effective heuristics for solving VRPs without relying on extensive domain expertise. Specifically, there are three mainstream types of neural VRP solvers in literature: 1) *Autoregressive construction solvers* [11, 12] sequentially select feasible nodes, constructing the solution in an autoregressive manner. This approach efficiently generates feasible solutions and naturally handle complex constraints through feasibility masking. 2) *Non-autoregressive construction solvers* [13, 14] build a matrix, such as a heatmap representing the probability distribution of each edge in the optimal solution. Advanced search algorithms (e.g., MCTS) are then employed to extract high-quality solutions, making this approach particularly effective for solving large-scale problem instances. 3) *Improvement solvers* [15, 16] iteratively refine an incumbent solution by exploiting classical local search methods (e.g., k-opt). This approach has the potential to outperform traditional solvers given sufficient runtime.

Compared to traditional solvers, neural solvers offer several advantages, including high computational parallelism on GPUs, reduced reliance on domain-specific knowledge replaced by automated patterns and knowledge discovery from data, and adaptability to diverse problem settings. They have demonstrated superior performance, either achieving lower optimality gaps or faster solving times, establishing NCO as a promising paradigm for solving VRPs.

1.2 Motivations

Despite their advantages and rapid advancement, neural VRP solvers still face significant challenges, such as generalization and interpretability. This thesis focuses on the generalization challenge, where the performance of neural solvers deteriorates significantly when applied to out-of-distribution (OOD) problem instances. In specific, most neural solvers are trained on monotonic synthetic data. For example, popular attention-based models [11, 12] are only trained and tested on instances of a fixed size (e.g., 100 nodes) with node coordinates sampled from the

uniform distribution. Their performance drops drastically when applied to an unseen task during training. As noted by Joshi et al. [17], their performance can even fall behind that of a simple insertion heuristic as the problem size increases. This generalization issue raises significant concerns about the practical applicability of neural VRP solvers in real-world settings.

The root cause of this generalization challenge is the inability of neural solvers to learn generalizable representations across the problem space, leading to performance degradation on OOD cases. To address this issue, the central research question of this thesis is *how to enhance the generalization of neural VRP solvers, particularly from three perspectives: cross-size, cross-distribution, and cross-problem generalization*. Below, we discuss the motivation and practical significance of each.

Cross-size Generalization. The NP-hard nature of COPs implies an exponential worst-case time complexity as the problem scales up, a fundamental challenge for these problems. Over the past several decades, the operations research community has devoted considerable effort to designing effective yet efficient heuristics (e.g., with polynomial time complexity per iteration). Although neural solvers generally also operate with polynomial time complexity, they suffer from significant generalization challenges. Thus, it is crucial for the NCO community to address these issues. Moreover, training neural solvers directly on large-scale instances is challenging due to substantial GPU memory requirements. As a result, it is crucial for solvers trained on small-scale instances to maintain decent performance when applied to larger-scale problem instances. This capability is essential for bridging the gap between controlled experimental settings and real-world applications, where problem sizes can vary dramatically.

Cross-distribution Generalization. Deep learning-based neural solvers are generally trained under the principle of empirical risk minimization (ERM), which minimizes the average loss over a training dataset as an approximation of the true expected risk across the entire data distribution. Due to inaccessibility to the entire data distribution, the model parameters are optimized on a limited, known training distribution. In real-world VRP applications, such as in a logistics company, the historical data available forms the training distribution, but new customers and evolving market conditions introduce data that falls outside this distribution. Consequently, the neural solver may face degradation when encountering these

OOD cases. Addressing cross-distribution generalization is therefore essential for developing robust neural solvers that can adapt to dynamic, real-world scenarios.

Cross-problem Generalization. Cross-size and cross-distribution generalization focus on instance-level adaptation within a single problem, while cross-problem generalization operates at a higher level by exploring commonness across diverse optimization tasks. Most existing neural VRP solvers are only structured and trained independently on a problem variant with specific constraints, making them less generic and practical. In contrast, traditional operations research solvers (i.e., exact and inexact solvers) are capable of handling a wide range of problem variants, making them versatile tools for solving VRPs. Despite the recent surge in neural solvers, these methods still require task-specific customization, resulting in significant computational and developmental overhead. Moreover, from a machine learning perspective, breakthroughs in foundation models (FMs) have revolutionized representation learning across fields such as language, vision, graphs, and decision-making. This progress motivates the NCO community to pioneer multi-task neural VRP solvers that leverage shared knowledge acquired from diverse problem variants to tackle novel optimization tasks. Transitioning from single-task to multi-task neural solvers represents a significant advancement toward the capabilities of FMs, which may drive representation learning in NCO, potentially unlock the fundamental principles of learning-to-optimize and enable transformative solutions to complex and practical COPs beyond human capabilities.

In summary, cross-size, cross-distribution, and cross-problem generalization represent distinct levels of generalization that neural solvers should address to develop generalizable representations across diverse VRP contexts. Motivated by these challenges, this thesis centers on enhancing the generalization of neural VRP solvers by introducing novel training frameworks, innovative model architectures, and expanded problem settings, which will be discussed in the following subsection.

1.3 Contributions

Aligned with our motivations, we introduce three novel approaches to enhance cross-size, cross-distribution, and cross-problem generalization of neural VRP solvers,

with a particular focus on reinforcement learning based autoregressive construction solvers due to their broad applicability across most VRP variants.

- **Generalizable neural VRP solvers across distributions.** At a high level, we frame the defence against adversarial perturbations in VRP instances as an effort to enhance the cross-distribution generalization of neural solvers. To this end, we introduce an ensemble-based Collaborative Neural Framework (CNF) that adversarially trains multiple models simultaneously, boosting robustness against adversarial attacks while also improving performance on clean instances. In CNF, an adversary perturbs the data distribution to generate challenging instances, compelling the models to adapt and thereby strengthening their cross-distribution generalization. Moreover, a global attack mechanism and a neural router are designed to enhance data diversity and effectively exploit model capacity. Extensive experiments on the traveling salesman problem (TSP) and capacitated vehicle routing problem (CVRP) demonstrate the effectiveness and versatility of CNF in enhancing cross-distribution generalization.

- **Generalizable neural VRP solvers across sizes and distributions.** We study a challenging yet realistic setting for neural VRP solvers by addressing both cross-size and cross-distribution generalization. Drawing on the insights from the No Free Lunch Theorems in Machine Learning, we propose Omni-VRP, a generic meta-learning framework that effectively trains an initial meta-model capable of rapidly adapting to new tasks only using limited data during inference. Each task corresponds to instances sharing a common problem scale and data distribution. In addition, we introduce an efficient approximation method to reduce the training overhead associated with second-order derivative calculations. Extensive experiments on synthetic and benchmark instances of TSP and CVRP demonstrate the effectiveness of our approach in achieving strong zero-shot and few-shot generalization.

- **Generalizable neural VRP solvers across problems.** We pioneer the study of cross-problem generalization, pushing the boundaries of neural VRP solvers in learning generalizable representations across diverse constraints. Methodologically, we propose a unified neural solver MVMoE to solve 16 VRPs simultaneously. The sole MVMoE can be trained on diverse VRP variants, and facilitate a strong zero-shot generalization capability on unseen VRPs. To enhance model

capacity efficiently, we employ a mixture-of-experts architecture with a hierarchical gating mechanism, striking a balance between empirical performance and computational overhead. Notably, hierarchical gating exhibits much stronger OOD generalization capability than base gating. Extensive experiments demonstrate that MVMoE significantly improves the zero-shot generalization against baselines on 10 unseen VRP variants, and achieves decent results on the few-shot setting and real-world instances. We further provide extensive studies on the effect of MoE configurations on the zero-shot generalization performance.

1.4 Organization

The remainder of this thesis is organized as follows. In Chapter 2, we review the literature on neural VRP solvers and generalization studies, along with related topics such as scalability, constraint handling, interpretability, and applications of large language models (LLMs) for solving VRPs. In Chapter 3, we introduce an adversarial trainingbased approach to improve the cross-distribution generalization of neural VRP solvers. Chapter 4 presents a generic meta-learning framework designed to enhance both cross-size and cross-distribution generalization. In Chapter 5, we pioneer cross-problem generalization by introducing a unified neural solver based on a mixture-of-experts architecture, capable of solving 16 VRP variants. Finally, Chapter 6 concludes the thesis and outlines directions for future work.

Chapter 2

Literature Review

In this chapter, we review the literature relevant to our work. We begin with a comprehensive survey of neural VRP solvers. Next, we discuss recent advancements in generalization, focusing on enhancing cross-size, cross-distribution, and cross-problem generalization of neural VRP solvers. Finally, we explore other key aspects of VRP research in NCO, including scalability, constraint handling, interpretability, and applications of LLMs for solving VRPs.

2.1 Neural VRP Solvers

Neural VRP solvers can be broadly categorized into three classes: autoregressive construction solvers, non-autoregressive construction solvers, and improvement solvers. Below, we provide a detailed discussion of each.

2.1.1 Autoregressive Construction Solvers

Autoregressive construction solvers build solutions sequentially by iteratively appending a feasible node to a growing partial solution. As one of the pioneering works in this area, Vinyals et al. [8] introduce the Pointer Network (Ptr-Net) to solve TSP using supervised learning. To alleviate the burden of obtaining labeled data, subsequent studies train Ptr-Net via reinforcement learning to address both

TSP [18] and CVRP [19]. Motivated by the success of the Transformer architecture [9], Kool et al. [11] propose the attention model (AM), which leverages attention mechanisms to tackle various VRPs, including TSP and CVRP. This approach has laid the groundwork for many subsequent contributions in the field. To enhance AM, Kwon et al. [12] introduce policy optimization with multiple optima (POMO), leveraging solution symmetries and instance augmentation to mitigate local optima. Expanding on this, Kim et al. [20] propose a general-purpose symmetric learning scheme suitable for various COPs and solutions to improve performance. Hottung et al. [21] introduce an efficient active search method that fine-tunes model parameters during inference, achieving strong performance on TSP and CVRP. Building upon AM and POMO, several works incorporate ensemble principles into training, either through a multi-decoder architecture [22, 23] or a conditional single-decoder approach [24]. In contrast to light decoder models like AM and POMO, recent studies have explored heavy-decoder architectures to improve scalability, employing supervised learning [25, 26] and self-improvement learning [27]. In addition, various adaptations have been developed to address different VRP variants, such as the asymmetric TSP (ATSP) [28] and heterogeneous routing problems [29]. For a comprehensive overview, we refer interested readers to Bogyrbayeva et al. [30], Berto et al. [31], Wu et al. [32]. In summary, autoregressive construction solvers can efficiently generate feasible solutions and naturally handle complex constraints through feasibility masking. However, effectively solving large-scale instances remains a challenge.

2.1.2 Non-autoregressive Construction Solvers

Non-autoregressive construction solvers generate high-quality solutions by predicting critical information (e.g., a matrix) that guides the solution construction process, typically in a single pass. A prominent direction in this category is heatmap prediction, which estimates the likelihood of edges appearing in an optimal solution. Joshi et al. [13] pioneer the use of GNN-based heatmaps combined with beam search for solving TSP. To extend heatmap-based methods to arbitrarily large instances, Fu et al. [14] introduce a divide-and-conquer approach with heatmap merging, leveraging Monte Carlo Tree Search (MCTS) to derive high-quality solutions. Their method significantly outperform previous approaches on large-scale

TSP instances. Recently, more advanced generative methods are proposed to improve efficiency and solution quality. Qiu et al. [33] introduce DIMES, an RL-based approach that optimizes over a compact continuous parameterization of candidate solutions. DIFUSCO [34] replace GNN-based heatmap prediction with diffusion models, further refined by Li et al. [35, 36]. In an unsupervised setting, UTSP [37] leverages a surrogate loss function to train GNNs, implicitly enforcing TSP constraints while optimizing for shorter paths. Despite these advancements, the effectiveness of learned heatmap has been challenged by Xia et al. [38], which highlights a fundamental misalignment between training and test objectives. Beyond heatmap-based approaches, some methods integrate learned guidance into traditional heuristics. The GLS solver [39] employs GNNs to enhance local search, while heuristic measures have been learned within ant colony optimization frameworks [40, 41]. In summary, non-autoregressive solvers offer scalability advantages, particularly for large instances. However, adapting these methods to handle problem variants with complex constraints remains an open challenge.

2.1.3 Improvement Solvers

In contrast to construction solvers, improvement solvers iteratively refine an existing solution until a termination criterion is met. These methods generally aim to enhance solution quality through learned refinement policies. NeuRewriter [15] introduces a neural rewriting framework that iteratively modifies solutions via node swaps. L2I [42] leverages RL to select hand-crafted local search heuristics from a predefined pool, surpassing LKH-3 in solution quality but suffering from heavy reliance on manual heuristics and high computational overhead. In another early approach, Deudon et al. [43] employ an RL model to generate initial solutions, subsequently improved by local search. The NLNS method [44] refines this idea by learning to control a ruin-and-repair process, where handcrafted operators partially destroy a solution before a neural model reconstructs it. Beyond local search, crossover-based strategies are explored in Kim et al. [45], where solution exchanges are guided by learned policies. A key research direction in learned improvement involves controlling k-opt heuristics for VRPs. Wu et al. [16] explore neural guidance for 2-opt, outperforming AM. This was later enhanced by Ma et al. [46], introducing Dual-Aspect Collaborative Attention (DAC-Att) and cyclic positional encoding. DAC-Att is further optimized into Synthesis Attention [47] to reduce

computational costs. Meanwhile, d O Costa et al. [48] propose an RNN-based policy for 2-opt, which Sui et al. [49] extend to 3-opt, improving efficiency. However, these approaches remained constrained by fixed k values. NeuOpt [50] address this limitation by enabling flexible k-opt exchanges and allowing exploration of both feasible and infeasible regions. In summary, although improvement solvers can outperform traditional solvers given sufficient runtime, they often struggle with scalability and computational efficiency.

2.2 Generalization Studies

Most neural VRP solvers are trained and tested on instances that share similar characteristics, yielding decent in-distribution generalization performance. However, these methods often exhibit significant performance degradation when faced with out-of-distribution data that differ from the training set [51, 52]. Recent research has sought to enhance the generalization capabilities of neural solvers. These approaches can be broadly categorized into three classes: cross-size, cross-distribution, and cross-problem generalization. It is worth noting that studies on robustness (e.g., guaranteeing worst-case performance) and uncertainty quantification can also be seen as efforts to improve the generalization of neural VRP solvers. However, these lines of research typically address different problem settings than those considered in this thesis.

2.2.1 Cross-size Generalization

Cross-size generalization addresses the challenge of scaling models from small training instances to larger test instances. Researchers have tackled this issue using several innovative approaches. For example, curriculum learning [53] and meta-learning [54] techniques have been employed to train models on instances of various sizes, enabling gradual adaptation and improved performance across scales. Advances in attention-based architectures have also been pivotal. Kim et al. [55] introduce a scale-conditioned network to adapt the model's behavior based on instance size, while Bdeir et al. [56] develop a sparse dynamic attention mechanism that adjusts focus depending on the scale of the problem. These modifications help the models dynamically adjust their processing as instance size increases. Another

promising direction is the incorporation of distance-based biases during the decoding phase. Studies by Jin et al. [57], Son et al. [58], Wang et al. [59] have integrated distance cues to steer the model towards more efficient exploration of the solution space. In parallel, Gao et al. [60] employ a local policy network that captures and utilizes distance information during compatibility calculations, and Li et al. [61] further enhance this approach by refining node embeddings with distance-related features. Recent work has also demonstrated that SL-based heavy decoder methods [25, 26] can enhance performance on large-scale instances. Meanwhile, Huang et al. [62] revisit light decoder methods and propose RELD, which introduces simple yet effective modifications to narrow the performance gap between light and heavy decoder paradigms.

2.2.2 Cross-distribution Generalization

Cross-distribution generalization focuses on maintaining performance when the underlying data distribution shifts. Early approaches in this area seek to proactively expose models to challenging scenarios. Xin et al. [63] introduce a GAN-based framework that generates hard instances, effectively pushing the model to learn from more difficult examples. Wang et al. [64] adopt a game-theoretic approach with the Policy Space Response Oracle (PSRO) framework, which is further enhanced by Wang et al. [65]. Zhang et al. [66] develop a hardness-adaptive generator for TSP instances and employ curriculum learning to incrementally increase problem difficulty. Alternative strategies have focused on making the model inherently more sensitive to distributional variations. Jiang et al. [67] leverage a CNN to capture distribution-aware features and apply group distributionally robust optimization to enhance model resilience. In contrast, Bi et al. [68] present AMDKD, a knowledge distillation framework where multiple teacher models, each trained on different exemplar distributions, guide a lightweight student model toward becoming a more generalist solver. Other advanced training algorithms have also been explored to bolster generalization. Both Geisler et al. [69] and Lu et al. [70] integrate adversarial training to improve model robustness against distribution shifts. Manchanda et al. [54] apply meta-learning to obtain a well-initialized model that can quickly adapt to diverse data distributions. Jiang et al. [71] propose an ensemble learning method that builds a group of diverse sub-policies, each capable

of handling different distributional characteristics. Collectively, these studies illustrate a broad spectrum of strategies aimed at mitigating the impact of distribution shifts, ultimately enhancing cross-distribution generalization of neural VRP solvers.

2.2.3 Cross-problem Generalization

Cross-problem generalization extends the applicability of a model to different but related VRP variants. Formally, if a problem falls within the VRP family but differs in its constraints or objectives, it is regarded as a VRP variant. Several studies pioneer the exploration of variations from a problem perspective by training generalized neural solvers capable of handling multiple problems with varying constraints and objectives. Wang and Yu [72] employ multi-armed bandits to address multiple VRPs simultaneously, while Lin et al. [73] adapt a model pretrained on a base VRP to new target variants via efficient fine-tuning. However, both approaches struggle with zero-shot generalization due to their reliance on architectures tailored to predetermined problem variants. To overcome these limitations, Liu et al. [74] propose a compositional zero-shot learning framework that views VRP variants as different combinations of fundamental attributes. By using a shared network to learn these representations, their approach enhances generalizability across unseen problem types. Building on this foundation, subsequent studies such as Berto et al. [75] and Li et al. [76] have further refined these methods to achieve better cross-problem generalization. Expanding the scope beyond VRPs, other works [77, 78, 79, 80, 81, 82] have tackled a diverse set of combinatorial optimization domains, including routing, scheduling, packing, and graph-based problems. These multi-task neural solvers demonstrate the potential to learn universal representations, paving the way for the development of future foundation models that can adapt to a broad spectrum of optimization challenges.

2.3 Further Related Works

We briefly discuss several VRP research directions that, while not central to this thesis, merit attention: scalability, constraint handling, interpretability, and applications of LLMs for solving VRPs.

Scalability. This research stream focuses on solving very large-scale instances by leveraging divide-and-conquer and decomposition strategies. Li et al. [83] propose a learning to delegate (L2D) approach that delegates the improvement of subtours to LKH3. Zong et al. [84] introduce the Rewriting-by-Generating (RBG) framework, which alternates between learning-based merging and rule-based decomposition to tackle large problem instances effectively. Hou et al. [85] introduce the Two-stage Divide Method (TAM), a real-time neural solver for solving large-scale CVRP. In this approach, a dividing model autoregressively partitions the CVRP into multiple sub-TSP instances, which are subsequently solved by sub-solvers such as AM and LKH3. Building on this concept, Kim et al. [86] introduce local subtour reconstruction for TSP, first generating a diverse set of initial solutions and then iteratively decomposing and refining them. Similarly, Pan et al. [87] propose H-TSP, a hierarchical solver that alternates between forming open-loop TSP using upper-level policies and refining the solutions. More recently, Cheng et al. [88] introduce a select-and-optimize framework that employs a TSP solution pipeline akin to that presented by Ye et al. [89].

Constraint Handling. Most neural VRP solvers address constraints by employing feasibility masking techniques that prevent selection of actions leading to infeasible solutions during either construction or iteration search. However, this approach presupposes the existence of accurate masks and does little to foster constraint-awareness during training, an assumption that becomes problematic when dealing with complex constraints. To mitigate this, Tang et al. [90] and Zhang et al. [91] introduce methods that relax hard constraints into softer ones via relaxation techniques and redefinition of the problem, yet these strategies often result in near-feasible solutions accompanied by high infeasibility rates in more complex VRP settings. Ma et al. [50] demonstrate that allowing temporary constraint violations could actually benefit neural iterative solvers. More recently, Chen et al. [92] develop the multi-step look-ahead (MUSLA) method, specifically tailored for TSPTW, which integrates problem-specific features and leverages a large supervised learning dataset to improve constraint handling. In contrast, Bi et al. [93] propose a more flexible and generic PIP framework that combines preventative infeasibility masking, learnable decoders, and adaptive strategies, enabling broader neural solvers to advance without relying on labelled training data.

Interpretability. Only a few studies have focused on understanding neural VRP

solvers. Zhang et al. [94] investigate interpretability by analyzing the embeddings learned across various architectures through three probing tasks. Another line of research leverages LLMs to elucidate decision-making processes. For example, Li et al. [95] employ LLMs to generate human-friendly explanations for supply chain optimization solutions, making the results more accessible to stakeholders. Kikuta et al. [96] introduce RouteExplainer, a post-hoc framework that uses counterfactual analysis to quantify the influence of each edge in a generated route. Given that neural solvers inherently operate as black boxes, it is essential to develop interpretability mechanisms to enhance their reliability and foster greater interactivity in real-world VRP applications.

Applications of LLMs for Solving VRPs. The application of LLMs to VRP problems can be broadly categorized into three directions: using LLMs as solution generators, heuristic generators, and formulation generators.

1) *Solution Generator:* This approach focuses on integrating diverse sources of information into prompts so that LLMs can directly generate solutions. Early work by Yang et al. [97] demonstrates that LLMs can iteratively refine TSP solutions, albeit with a strong dependence on prompt quality. Later studies enhance solution quality by incorporating additional information, such as visual representations [98]. Another promising strategy is to leverage LLMs for executing key algorithmic components. For instance, Liu et al. [99] utilize LLMs to carry out selection, crossover, and mutation operations via evolutionary computation prompting. To tackle larger-scale problems, Iklassov et al. [100] introduce an LLM-based task decomposition approach in which the model assesses the difficulty of the input and recursively breaks down complex tasks into manageable subproblems.

2) *Heuristic Generator:* Recent research has increasingly exploited LLMs to create heuristics for solving VRPs. In a manner similar to Romera-Paredes et al. [101], studies by Liu et al. [102, 103] have employed LLMs to generate key algorithmic components within an evolutionary computation framework. Building on this foundation, further advancements have integrated sophisticated techniques, including reflective feedback [104], diversity-aware strategies [105], and Monte Carlo Tree Search (MCTS) [106, 107], to boost performance and extend applicability to settings like multi-objective optimization[108]. More recently, Li et al. [109] introduce the Automatic Routing Solver (ARS), which leverages LLMs to automatically design constraint-aware heuristics for VRP variants that involve complex, real-world

constraints. In general, these approaches tend to deliver superior results compared to those based solely on solution generators.

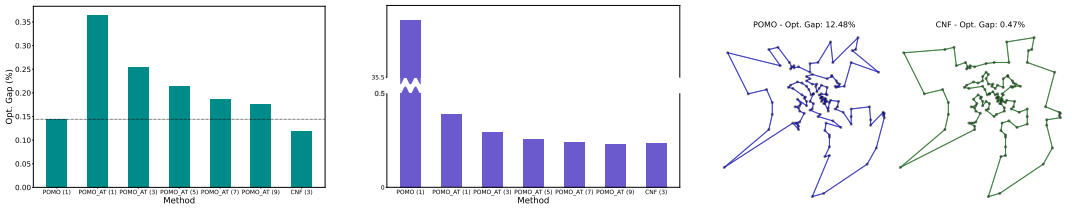
3) *Formulation Generator:* Recent studies [110, 111, 112, 113] have started to utilize LLMs to translate natural language descriptions of optimization problems into formulations that can be tackled by operations research solvers. However, few works have focused specifically on VRPs. Jiang et al. [114] introduce DRoC, a retrieval-augmented generation (RAG)-based pipeline designed to enhance LLMs' ability to model VRPs with a diverse array of complex constraints. DRoC accomplishes this by decomposing VRP constraints, retrieving externally relevant information for each constraint, and dynamically switching between RAG and self-debugging modes to significantly improve modeling accuracy.

Chapter 3

Generalizable Neural VRP Solvers Across Distributions

We approach cross-distribution generalization through the lens of adversarial robustness, where an adversary perturbs data distributions to degrade the performance of neural solvers. Although recent studies [66, 69, 70] have investigated this issue, defensive strategies for forging robust neural VRP solvers remain underexplored. Existing efforts primarily concentrate on the attack side by proposing various perturbation models to generate adversarial instances. On the defense side, most work simply relies on standard adversarial training (AT) [116]. In this approach, the problem is formulated as a min-max optimization task: first, adversarial instances that maximally impair model performance are generated, and then the empirical loss on these instances is minimized. However, vanilla AT is known to incur an undesirable trade-off [117, 118] between standard generalization (i.e., performance on clean instances) and adversarial robustness (i.e., performance on adversarial instances). As shown in Fig. 3.1, while vanilla AT enhances the adversarial robustness, it does so at the expense of standard generalization (e.g., POMO (1) vs. POMO_AT (1)). A key factor is that the trained model may not be sufficiently expressive. Nonetheless, it remains an open question how to effectively synergize multiple models to achieve favorable overall performance on both clean and adversarial instances within a reasonable computational budget.

This chapter is adapted from the author's publication [115].



(a) Performance on Clean Instances (b) Performance on Adversarial Instances (c) Illustration of Robustness Issue

FIGURE 3.1: An illustrative example of the motivation.

In this Chapter, we introduce an ensemble-based AT method to concurrently enhance both the standard generalization and adversarial robustness. Instead of separately training multiple models, we propose a Collaborative Neural Framework (CNF) to exert AT on multiple models in a collaborative manner. Specifically, in the inner maximization optimization of CNF, we synergize multiple models to further generate the global adversarial instance for each clean instance by attacking the best-performing model, rather than only leveraging each model to independently generate its own local adversarial instances. In doing so, the generated adversarial instances are diverse and strong in benefiting the policy exploration and attacking the models, respectively. In the outer minimization optimization of CNF, we train an attention-based neural router to forward instances to models for effective training, which helps achieve satisfactory load balancing and collaborative efficacy. The resulting models demonstrate strong robustness against adversarial perturbations and thus improved cross-distribution generalization.

3.1 Preliminaries

3.1.1 VRPs on Graphs

Problem Definition. Without loss of generality, we define a VRP instance x over a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V} = \{v_i\}_{i=1}^{n}$ represents the node set, and $(v_i, v_j) \in \mathcal{E}$ represents the edge set with $v_i \neq v_j$. The solution τ to a VRP instance is a tour, i.e., a sequence of nodes in $\mathcal{V}$. The cost function $c(\cdot)$ computes the total length of a given tour. The objective is to seek an optimal tour τ^* with the minimal cost: $\tau^* = \arg\min_{\tau \in \Phi} c(\tau|x)$, where Φ is the set of all feasible tours which obey the problem-specific constraints. For example, a feasible tour in TSP should visit

each node exactly once, and return to the starting node in the end. For CVRP, each customer node in $\mathcal{V}$ is associated with a demand δ_i, and a depot node v_0 is additionally added into $\mathcal{V}$ with $\delta_0 = 0$. Given the capacity Q for each vehicle, a tour in CVRP consists of multiple sub-tours, each of which represents a vehicle starting from v_0, visiting a subset of nodes in $\mathcal{V}$ and returning to v_0. It is feasible if each customer node in $\mathcal{V}$ is visited exactly once, and the total demand in each sub-tour is upper bounded by Q. The optimality gap $\frac{c(\tau)-c(\tau^*)}{c(\tau^*)} \times 100\%$ is used to measure how far a solution is from the optimal solution.

Autoregressive Construction Methods. Popular neural solvers [11, 12] construct a solution to a VRP instance following Markov Decision Process (MDP), where the policy is parameterized by a neural network with parameters θ. The policy takes the states as inputs, which are instantiated by features of the instance and the partially constructed solution. Then, it outputs the probability distribution of valid nodes to be visited next, from which an action is taken by either greedy rollout or sampling. After a complete tour τ is constructed, the probability of the tour can be factorized via the chain rule as $p_\theta(\tau|x) = \prod_{s=1}^{S} p_\theta(\pi_\theta^s|\pi_\theta^{<s}, x)$, where π_θ^s and $\pi_\theta^{<s}$ represent the selected node and the partial solution at the s_{th} step, and S is the number of total steps. Typically, the reward is defined as the negative length of a tour $-c(\tau|x)$. The policy network is commonly trained with REINFORCE [119]. With a baseline function $b(\,\cdot\,)$ to reduce the gradient variance and stabilize the training, it estimates the gradient of the expected reward as:

$$\nabla_\theta \mathcal{L}(\theta|x) = \mathbb{E}_{p_\theta(\tau|x)}[(c(\tau) - b(x))\nabla_\theta \log p_\theta(\tau|x)]. \tag{3.1}$$

3.1.2 Adversarial Training

Adversarial training is one of the most effective and practical techniques to equip deep learning models with adversarial robustness against crafted perturbations on the clean instance. In the supervised fashion, where the clean instance x and ground truth (GT) label y are given, AT is commonly formulated as a min-max optimization problem:

$$\min_\theta \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(y, f_\theta(\tilde{x}))], \text{ with } \tilde{x} = \arg\max_{\tilde{x}_i \in \mathcal{N}_\epsilon[x]} [\ell(y, f_\theta(\tilde{x}_i))], \tag{3.2}$$

where $\mathcal{D}$ is the data distribution; ℓ is the loss function; $f_\theta(\cdot)$ is the model prediction with parameters θ; $\mathcal{N}_\epsilon[x]$ is the neighborhood around x, with its size constrained by the attack budget ϵ. The solution to the inner maximization is typically approximated by projected gradient descent:

$$x^{(t+1)} = \Pi_{\mathcal{N}_\epsilon[x]}[x^{(t)} + \alpha \cdot \texttt{sign}(\nabla_{x^{(t)}} \ell(y, f_\theta(x^{(t)})))], \tag{3.3}$$

where α is the step size; Π is the projection operator that projects the adversarial instance back to the neighborhood $\mathcal{N}_\epsilon[x]$; $x^{(t)}$ is the adversarial instance found at step t; and the `sign` operator is used to take the gradient direction and carefully control the attack budget. $x^{(0)}$ is initialized by the clean instance or randomly perturbed instance with small Gaussian or Uniform noises. The adversarial instance is updated iteratively towards loss maximization until a stopping criterion is satisfied.

AT for VRPs. Most ML research on adversarial robustness focuses on the continuous image domain [116, 120]. We would like to highlight two main differences in the context of discrete VRPs (or COPs). 1) *Imperceptible perturbation:* The adversarial instance $\tilde{x}$ is typically generated within a small neighborhood of the clean instance x, so that the adversarial perturbation is imperceptible to human eyes. For example, the adversarial instance in image-related tasks is typically bounded by $\mathcal{N}_\epsilon[x] : \|x - \tilde{x}\|_p \leq \epsilon$ under the l_p norm threat model. When the attack budget ϵ is small enough, $\tilde{x}$ retains the GT label of x. However, it is not the case for VRPs due to the nature of discreteness. The optimal solution can be significantly changed even if only a small part of the instance is modified. Therefore, the subjective imperceptible perturbation is not a realistic goal in VRPs, and we do not exert such an explicit imperceptible constraint on the perturbation model. In this chapter, we set the attack budget within a reasonable range based on the attack methods. 2) *Accuracy-robustness trade-off:* The standard generalization and adversarial robustness seem to be conflicting goals in image-related tasks. With increasing adversarial robustness the standard generalization tends to decrease, and a number of works intend to mitigate such a trade-off in the image domain [118, 121, 122]. By contrast, a recent work [69] claims the existence of neural solvers with high accuracy and robustness in COPs. They state that a *sufficiently expressive* model does not suffer from the trade-off given the problem-specific *efficient and sound* perturbation model, which guarantees the correct GT label of the perturbed instance.

However, by following the vanilla AT, we empirically observe that the undesirable trade-off may still exist in VRPs, which is mainly due to the insufficient model capacity under the specific perturbation model. Furthermore, similar to combinatorial optimization, although the language and graph domains are also discrete, their methods cannot be trivially adapted to VRPs.

3.1.3 Attack Methods in VRPs

We first give a formal definition of adversarial instance in VRPs, and then present details of existing attack methods for neural VRP solvers, including perturbing input attributes [66], inserting new nodes [69], and lowering the cost of a partial problem instance to ensure no-worse theoretical optimum [70]. The details of the latter two methods are provided in Appendix A. They can generate instances that are underperformed by the current model.

We define an adversarial instance as the instance that 1) is obtained by the perturbation model within the neighborhood of the clean instance, and 2) is underperformed by the current model. Formally, given a clean VRP instance $x = \{x_c, x_d, \sigma\}$, where $x_c \in \mathcal{N}_c$ is the continuous variable (e.g., node coordinates) within the valid range $\mathcal{N}_c$; $x_d \in \mathcal{N}_d$ is the discrete variable (e.g., node demand) within the valid range $\mathcal{N}_d$; σ is the constraint, the adversarial instance $\tilde{x} = \{\tilde{x}_c, \tilde{x}_d, \tilde{\sigma}\}$ is found by the perturbation model G around the clean instance x, on which the current model may be vulnerable. Technically, the adversarial instance $\tilde{x}$ can be constructed by adding crafted perturbations γ to the corresponding attribute of the clean instance, and then project them back to the valid domain, e.g., $\tilde{x}_c = \Pi_{\mathcal{N}_c}(x_c + \alpha \cdot \gamma_{x_c})$, where Π denotes the projection operator (e.g., min-max normalization); α denotes the attack budget. The crafted perturbations can be obtained by various perturbation models, such as the one in Eq. (3.4), $\gamma_{x_c} = \nabla_{x_c} \ell(x; \theta)$, where θ is the model parameters; ℓ is the loss function. We omit the attack step t for notation simplicity. Below, we detail each attack method.

Attribute Perturbation. The attack generator from Zhang et al. [66] is applied to attention-based models [11, 12] by perturbing attributes of input instances. It generates adversarial instances by directly maximizing the reinforcement loss variant (so called the hardness measure in Zhang et al. [66]). We take the perturbation on the node coordinate as an example. Suppose given the clean instance x (i.e.,

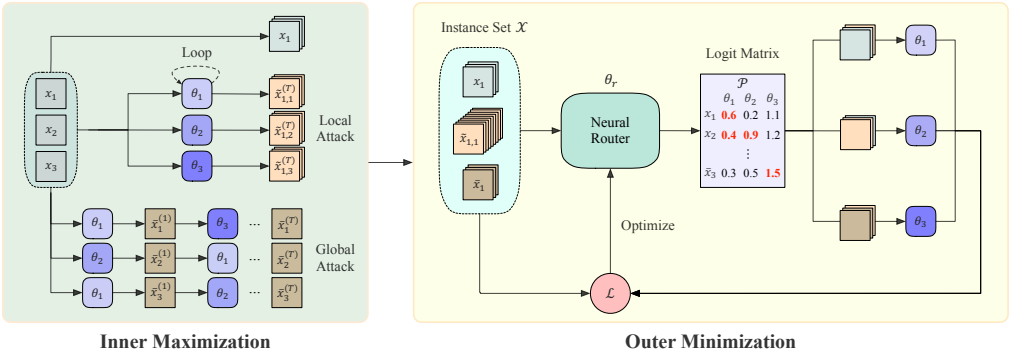


FIGURE 3.2: An overview of CNF.

$\tilde{x}^{(0)} = x$) and model parameter θ, the solution to the inner maximization can be approximated as follows:

$$\tilde{x}^{(t+1)} = \Pi_{\mathcal{N}_c}[\tilde{x}^{(t)} + \alpha \cdot \nabla_{\tilde{x}^{(t)}} \ell(\tilde{x}^{(t)}; \theta^{(t)})], \tag{3.4}$$

where $\tilde{x}^{(t)}$ is the (local) adversarial instances and $\theta^{(t)}$ is the model parameters, at step t. Here we use $\mathcal{N}_c$ to represent the valid domain of node coordinates (i.e., unit square $U(0,1)$) for simplicity. After each iteration, it checks whether $\hat{x}^{(t)} = \tilde{x}^{(t)} + \alpha \cdot \nabla_{\tilde{x}^{(t)}} \ell(\tilde{x}^{(t)}; \theta^{(t)})$ is within the valid domain $\mathcal{N}_c$ or not. If it is out of $\mathcal{N}_c$, the projection operator (i.e., min-max normalization) is applied as follows:

$$\Pi_{\mathcal{N}_c}(\hat{x}^{(t)}) = \frac{\hat{x}^{(t)} - \min \hat{x}^{(t)}}{\max \hat{x}^{(t)} - \min \hat{x}^{(t)}} (\max \mathcal{N}_c - \min \mathcal{N}_c). \tag{3.5}$$

Note that it originally only focuses on TSP, where the node coordinates are perturbed. We further adapt it to CVRP by perturbing both the node coordinates and node demands. The implementation is straightforward, except that we set the valid domain of node demands as $\mathcal{N}_d = \{1, \ldots, 9\}$. For the perturbations on node demands, the projection operator applies another round operation as follows:

$$\Pi_{\mathcal{N}_d}(\hat{x}^{(t)}) = \lceil \frac{\hat{x}^{(t)} - \min \hat{x}^{(t)}}{\max \hat{x}^{(t)} - \min \hat{x}^{(t)}} (\max \mathcal{N}_d - \min \mathcal{N}_d) \rceil. \tag{3.6}$$

3.2 Methodology

In this section, we first present the motivation and overview of the proposed framework, and then introduce the technical details. Overall, we propose a collaborative

neural framework to synergistically promote adversarial robustness among multiple models, while boosting standard generalization. Since conducting AT for deep learning models from scratch is computationally expensive due to the extra inner maximization steps, we use the model pretrained on clean instances as a warm-start for subsequent AT steps. An overview is illustrated in Fig. 3.2.

Motivation. Motivated by the empirical observations that 1) existing neural VRP solvers suffer from severe adversarial robustness issues; 2) undesirable trade-off between adversarial robustness and standard generalization may exist when following the vanilla AT, we propose to adversarially train multiple models in a *collaborative* manner to mitigate the above-mentioned issues within a reasonable computational budget. It then raises the research question on how to effectively and efficiently train multiple models under the AT framework, involving a pair of inner maximization and outer minimization, which will be detailed in the following parts. Note that despite the accuracy-robustness trade-off being a well-known research problem in the literature of adversarial ML, most works focus on the image domain. Due to the needs for GT labels or the dependence on the imperceptible perturbation model, their methods (e.g., TRADES [118], Robust Self-Training with AT [121]) cannot be directly leveraged to solve this trade-off in VRPs.

Overview. Given a pretrained model θ_p, CNF deploys the AT on its M copies (i.e., $\Theta^{(0)} = \{\theta_j^{(0)}\}_{j=1}^{M} \leftarrow \theta_p$) in a collaborative manner. Concretely, at each training step, it first solves the inner maximization optimization to synergistically generate the local ($\tilde{x}$) and global ($\bar{x}$) adversarial instances, on which the current models underperform. Then, in the outer minimization optimization, we jointly train an attention-based neural router θ_r with all models Θ by RL in an end-to-end manner. By adaptively distributing instances to different models for training, the neural router can reasonably exploit the overall capacity of models and thus achieve satisfactory load balancing and collaborative efficacy. During inference, we discard the neural router θ_r and use the trained models Θ to solve each instance. The best solution among them is returned to reflect the final *collaborative performance*. We present the pseudocode of CNF in Algorithm 1, and elaborate each part in the following subsections.

Algorithm 1 Collaborative Neural Framework for VRPs

Input: training steps: E, number of models: M, attack steps: T, batch size: B, pretrained model: θ_p;

Output: robust model set $\Theta^{(E)} = \{\theta_j^{(E)}\}_{j=1}^{M}$;

1: Initialize $\Theta^{(0)} = \{\theta_1^{(0)}, \cdots, \theta_M^{(0)}\} \leftarrow \theta_p,\ \theta_r^{(0)}$
2: **for** e $= 1, \ldots, E$ **do**
3: $\quad \{x_i\}_{i=1}^{B} \leftarrow$ Sample a batch of clean instances
4: $\quad$ Initialize $\tilde{x}_{i,j}^{(0)}, \bar{x}_i^{(0)} \leftarrow x_i, \quad \forall i \in [1, B],\ \forall j \in [1, M]$
5: $\quad$ **for** t $= 1, \ldots, T$ **do** ▷ *Inner Maximization*
6: $\quad\quad \tilde{x}_{i,j}^{(t)} \leftarrow$ Approximate solutions to $\max \ell(\tilde{x}_{i,j}^{(t-1)}; \theta_j^{(e-1)}), \quad \forall i \in [1, B],\ \forall j \in [1, M]$
7: $\quad\quad \theta_{b_i}^{(e-1)} \leftarrow$ Choose the best-performing model for $\bar{x}_i^{(t-1)}$ from $\Theta^{(e-1)}, \quad \forall i \in [1, B]$
8: $\quad\quad \bar{x}_i^{(t)} \leftarrow$ Approximate solutions to $\max \ell(\bar{x}_i^{(t-1)}; \theta_{b_i}^{(e-1)}), \quad \forall i \in [1, B]$
9: $\quad$ **end for**
10: $\quad \mathcal{X} \leftarrow \left\{ \{x_i, \tilde{x}_{i,j}^{(T)}, \bar{x}_i^{(T)}\}, \quad \forall i \in [1, B],\ \forall j \in [1, M] \right\}$ ▷ *Outer Minimization*
11: $\quad \mathcal{R} \leftarrow$ Evaluate $\mathcal{X}$ on $\Theta^{(e-1)}$
12: $\quad \tilde{\mathcal{P}} \leftarrow \texttt{Softmax}(f_{\theta_r^{(e-1)}}(\mathcal{X}, \mathcal{R}))$
13: $\quad \Theta^{(e)} \leftarrow$ Train $\theta_j^{(e-1)} \in \Theta^{(e-1)}$ on $\texttt{Top}\mathcal{K}(\tilde{\mathcal{P}}_{\cdot j})$ instances, $\quad \forall j \in [1, M]$
14: $\quad \mathcal{R}' \leftarrow$ Evaluate $\mathcal{X}$ on $\Theta^{(e)}$
15: $\quad \theta_r^{(e)} \leftarrow$ Update neural router $\theta_r^{(e-1)}$ with the gradient $\nabla_{\theta_r^{(e-1)}} \mathcal{L}$ using Eq. (3.9)
16: **end for**

3.2.1 Inner Maximization

The inner maximization aims to generate adversarial instances for the training in the outer minimization, which should be 1) effective in attacking the framework; 2) diverse to benefit the policy exploration for VRPs. Typically, an iterative attack method generates local adversarial instances for each model only based on its own parameter (e.g., the same θ in Eq. (3.3) is repetitively used throughout the generation). Such *local attack* (line 6) only focuses on degrading each individual model, failing to consider the ensemble effect of multiple models. Due to the existence of multiple models in CNF, we are motivated to further develop a general form of local attack – *global attack* (line 7-8), where each adversarial instance can be generated using different model parameters within T generation steps. Concretely, given each clean instance x, we generate the global adversarial instance $\bar{x}$ by attacking the corresponding *best-performing model* in each iteration of the inner maximization. In doing so, compared with the sole local attack, the generated adversarial instances are more diverse to successfully attack the models Θ. Without loss of generality, we take the attacker from Zhang et al. [66] as an example, which

directly maximizes the variant of the reinforcement loss as follows:

$$\ell(x;\theta) = \frac{c(\tau)}{b(x)} \log p_\theta(\tau|x), \tag{3.7}$$

where $b(\cdot)$ is the baseline function (as shown in Eq. (3.1)). On top of it, we generate the global adversarial instance $\bar{x}$ such that:

$$\bar{x}^{(t+1)} = \Pi_{\mathcal{N}}[\bar{x}^{(t)} + \alpha \cdot \nabla_{\bar{x}^{(t)}} \ell(\bar{x}^{(t)}; \theta_b^{(t)})], \quad \theta_b^{(t)} = \arg\min_{\theta \in \Theta} c(\tau|\bar{x}^{(t)}; \theta), \tag{3.8}$$

where $\bar{x}^{(t)}$ is the global adversarial instance and $\theta_b^{(t)}$ is the best-performing model (w.r.t. $\bar{x}^{(t)}$) at step t. If $\bar{x}^{(t)}$ is out of the range, it would be projected back to the valid domain $\mathcal{N}$ by Π, such as the min-max normalization for continuous variables or the rounding operation for discrete variables. We discard the `sign` operator in Eq. (3.3) to relax the imperceptible constraint.

In summary, the local attack is a special case of the global attack, where the same model is chosen as θ_b in each iteration. While the local attack aims to degrade a single model θ, the global attack can be viewed as explicitly attacking the collaborative performance of all models Θ, which takes into consideration the ensemble effect by attacking θ_b. In CNF, we involve adversarial instances that are generated by both the local and global attacks to pursue better adversarial robustness, while also including clean instances to preserve standard generalization.

3.2.2 Outer Minimization

After the adversarial instances are generated by the inner maximization, we collect a set of instances $\mathcal{X}$ with $|\mathcal{X}| = N$, which includes clean instances x, local adversarial instances $\tilde{x}$ and global adversarial instances $\bar{x}$, to train M models. Here a key problem is that *how are the instances distributed to models for their training, so as to achieve satisfactory load balancing (training efficiency) and collaborative performance (effectiveness)?* To solve this, we design an attention-based neural router, and jointly train it with all models Θ to maximize the improvement of collaborative performance.

Concretely, we first evaluate each model on $\mathcal{X}$ to obtain a cost matrix $\mathcal{R} \in \mathbb{R}^{N \times M}$. The attention-based neural router θ_r takes as inputs the instances $\mathcal{X}$ and $\mathcal{R}$, and

outputs a logit matrix $f_{\theta_r}(\mathcal{X}, \mathcal{R}) = \mathcal{P} \in \mathbb{R}^{N \times M}$, where f is the decision function. Then, we apply `Softmax` function along the first dimension of $\mathcal{P}$ to obtain the probability matrix $\tilde{\mathcal{P}}$, where the entity $\tilde{\mathcal{P}}_{ij}$ represents the probability of the i_{th} instance being selected for the outer minimization of the j_{th} model. For each model, the neural router distributes the instances with Top$\mathcal{K}$-largest predicted probabilities as a batch for its training (line 10-13). In doing so, all models have the same amount ($\mathcal{K}$) of training instances, which explicitly ensures the *load balancing*. We also discuss other strategies of instance distributing, such as sampling, instance-based choice, etc. More details can be found in Section 3.3.

After all models Θ are trained with the distributed instances, we further evaluate each model on $\mathcal{X}$, obtaining a new cost matrix $\mathcal{R}' \in \mathbb{R}^{N \times M}$. To pursue desirable *collaborative performance*, it is expected that the neural router θ_r can reasonably exploit the overall capacity of models. Since the action space of θ_r is huge and the models Θ are changing throughout the training, we resort to reinforcement learning (based on trial-and-error) to optimize parameters of the neural router θ_r (line 14-15). Specifically, we set $(\min \mathcal{R} - \min \mathcal{R}')$ as the reward signal, and update θ_r by gradient ascent to maximize the expected return with the following approximation:

$$\nabla_{\theta_r} \mathcal{L}(\theta_r | \mathcal{X}) = \mathbb{E}_{j \in [1,M], i \in \text{Top}\mathcal{K}(\tilde{\mathcal{P}}_{\cdot\, j}), \tilde{\mathcal{P}}}[(\min \mathcal{R} - \min \mathcal{R}')_i \nabla_{\theta_r} \log \tilde{\mathcal{P}}_{ij}], \tag{3.9}$$

where the min operator is applied along the last dimension of $\mathcal{R}$ and $\mathcal{R}'$, since we would like to maximize the improvement of collaborative performance after training with the selected instances. Intuitively, if an entity in $(\min \mathcal{R} - \min \mathcal{R}')$ is positive, it means that, after training with the selected instances, the collaborative performance of all models on the corresponding instance is increased. Thus, the corresponding action taken by the neural router should be reinforced, and vice versa. For example, in Fig. 3.2, if the reward entity for the first instance x_1 is positive, the probability of this action (i.e., the red one in the first row of $\mathcal{P}$) will be reinforced after optimization. Note that the unselected (e.g., black ones) will be masked out in Eq. (3.9). In doing so, the neural router learns to effectively distribute instances and reasonably exploit the overall model capacity, such that the collaborative performance of all models can be enhanced after optimization.

TABLE 3.1: Performance evaluation over 1K test instances.

		$n = 100$						$n = 200$					
		Uniform (100)		Fixed Adv. (100)		Adv. (100)		Uniform (200)		Fixed Adv. (200)		Adv. (200)	
		Gap	Time	Gap	Time	Gap	Time	Gap	Time	Gap	Time	Gap	Time
TSP	Concorde	0.000%	0.3m	0.000%	0.3m	–	–	0.000%	0.6m	0.000%	0.6m	–	–
	LKH3	0.000%	1.3m	0.002%	2.1m	–	–	0.000%	3.9m	0.005%	5.8m	–	–
	POMO (1)	0.144%	0.1m	35.803%	0.1m	35.803%	0.1m	0.736%	0.5m	63.477%	0.5m	63.477%	0.5m
	POMO_AT (1)	0.365%	0.1m	0.390%	0.1m	0.330%	0.1m	2.151%	0.5m	1.248%	0.5m	1.154%	0.5m
	POMO_AT (3)	0.255%	0.3m	0.295%	0.3m	0.243%	0.3m	1.884%	1.5m	1.090%	1.5m	1.011%	1.5m
	POMO_HAC (3)	0.135%	0.3m	0.344%	0.3m	0.316%	0.3m	0.683%	1.5m	1.308%	1.5m	1.273%	1.5m
	POMO_DivTrain (3)	0.255%	0.3m	0.297%	0.3m	0.254%	0.3m	1.875%	1.5m	1.093%	1.5m	1.026%	1.5m
	CNF_Greedy (3)	0.187%	0.3m	0.314%	0.3m	0.280%	0.3m	0.868%	1.5m	1.108%	1.5m	1.096%	1.5m
	CNF (3)	**0.118%**	0.3m	**0.236%**	0.3m	**0.217%**	0.3m	**0.614%**	1.5m	**0.954%**	1.5m	**0.952%**	1.5m
CVRP	HGS	0.000%	6.6m	0.000%	14.6m	–	–	0.000%	0.4h	0.000%	1.2h	–	–
	LKH3	0.538%	18.1m	0.344%	23.0m	–	–	1.116%	0.5h	0.761%	0.6h	–	–
	POMO (1)	1.209%	0.1m	3.983%	0.1m	3.983%	0.1m	2.122%	0.6m	16.173%	0.8m	16.173%	0.8m
	POMO_AT (1)	1.456%	0.1m	0.882%	0.1m	0.935%	0.1m	3.249%	0.6m	1.384%	0.6m	1.435%	0.6m
	POMO_AT (3)	1.256%	0.3m	0.767%	0.3m	0.809%	0.3m	2.919%	1.8m	1.253%	1.8m	1.296%	1.8m
	POMO_HAC (3)	1.085%	0.3m	0.829%	0.3m	0.848%	0.3m	1.974%	1.8m	1.374%	1.8m	1.367%	1.8m
	POMO_DivTrain (3)	1.254%	0.3m	0.754%	0.3m	0.809%	0.3m	2.946%	1.8m	1.220%	1.8m	1.302%	1.8m
	CNF_Greedy (3)	1.112%	0.3m	0.785%	0.3m	0.821%	0.3m	**1.969%**	1.8m	1.316%	1.8m	1.353%	1.8m
	CNF (3)	**1.073%**	0.3m	**0.730%**	0.3m	**0.769%**	0.3m	2.031%	1.8m	**1.193%**	1.8m	**1.198%**	1.8m

For traditional solvers, *Adv.* is not shown since the test adversarial dataset is different for each neural solver.

3.3 Experiments

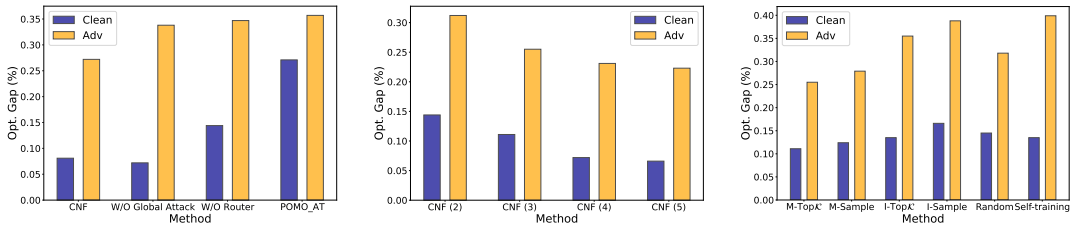
In this section, we empirically verify the effectiveness and versatility of CNF against attacks specialized for VRPs, and conduct further analyses to provide the underlying insights. Specifically, our experiments focus on two attack methods [66, 70], since the accuracy-robustness trade-off exists when conducting vanilla AT to defend against them. We conduct the main experiments on POMO [12] with the attacker in Zhang et al. [66], and further demonstrate the versatility of the proposed framework on MatNet [28] with the attacker in Lu et al. [70]. All experiments are conducted on a machine with NVIDIA V100S-PCIE cards and Intel Xeon Gold 6226 CPU at 2.70GHz.

Baselines. 1) *Traditional solvers:* we solve TSP instances by Concorde and LKH3 [5], and CVRP instances by hybrid genetic search (HGS) [6] and LKH3. 2) *Neural solvers:* we compare our method with the pretrained base model POMO ($\sim$1M parameters), and its variants training with various defensive methods, such as the vanilla adversarial training (POMO_AT), the defensive method proposed by the attacker [66] (POMO_HAC), and the diversity training [123] from the literature of ensemble-based adversarial ML (POMO_DivTrain). Specifically, POMO_AT adversarially trains the models by first generating local adversarial instances in the inner maximization, and then minimizing their empirical risks in the outer minimization. POMO_HAC further improves the outer minimization by optimizing a hardness-aware instance-reweighted loss function on a mixed dataset, including

both clean and local adversarial instances. POMO_DivTrain improves the ensemble diversity by minimizing the cosine similarity between the gradients of models w.r.t. the input. Furthermore, we also compare our method with CNF_Greedy by replacing the neural router with a heuristic greedy selection method.

Training Setups. CNF starts with a pretrained model, and then adversarially trains its M copies in a collaborative way. We consider two scales of training instances $n \in \{100, 200\}$. For the pretraining stage, the model is trained on clean instances following the uniform distribution. We use the open-source pretrained POMO for $n = 100$, and retrain the model for $n = 200$. Following the original training setups from [12], Adam optimizer is used with the learning rate of $1e-4$, the weight decay of $1e-6$ and the batch size of $B = 64$. To achieve full convergence, we pretrain the model on 300M and 100M clean instances for TSP200 and CVRP200, respectively. After obtaining the pretrained model, we use it to initialize $M = 3$ models, and further adversarially train them on 5M and 2.5M instances for $n = 100$ and $n = 200$, respectively. To save the GPU memory, we reduce the batch size to $B = 32$ for $n = 200$. The optimizer setting is the same as the one employed in the pretraining stage, except that the learning rate is decayed by 10 for the last 40% training instances. For the mixed data collection, we collect B clean instances, MB local adversarial instances and B global adversarial instances in each step.

Inference Setups. For neural solvers, we use the greedy rollout with x8 instance augmentations following [12]. We report the average gap over the test dataset containing 1K instances. Concretely, the gap is computed w.r.t. the traditional VRP solvers (i.e., Concorde for TSP, and HGS for CVRP). If multiple models exist, we report the collaborative performance, where the best gap among all models is recorded for each instance. The reported time is the total time to solve the entire dataset. We consider three evaluation metrics: 1) *Uniform (standard generalization):* the performance on clean instances whose distributions are the same as the pretraining ones; 2) *Fixed Adv. (adversarial robustness):* the performance on adversarial instances generated by attacking the pretrained model. It mimics the black-box setting, where the attacker generates adversarial instances using a surrogate model due to the inaccessibility to the current model and the transferability of adversarial instances; 3) *Adv. (adversarial robustness):* the performance on adversarial instances generated by attacking the current model. For a neural solver



(a) Components (b) Hyperparameters (c) Routing Strategies

FIGURE 3.3: Ablation studies on TSP100.

with multiple (M) models, it generates MK adversarial instances, from which we randomly sample 1K instances to construct the test dataset. It is the conventional white-box metric used to evaluate adversarial robustness in the literature of AT.

3.3.1 Performance Evaluation

The results are shown in Table 3.1. For all neural solvers, we report the inference time on a single GPU. Moreover, for neural solvers with multiple (M) models (e.g., CNF (3)), we develop an implementation of parallel evaluation on multiple GPUs, which can further reduce their inference time by almost M times. From the results, we observe that 1) traditional VRP solvers are relatively more robust than neural solvers against crafted perturbations, demonstrating the importance and necessity of improving adversarial robustness for neural solvers; 2) the evaluation metrics of Fixed Adv. and Adv. are almost consistent in the context of VRPs; 3) our method consistently outperforms baselines, and achieves high standard generalization and adversarial robustness concurrently. For CNF, we show the performance of each model on TSP100 in Fig. A.1. Although not all models excel well on both clean and adversarial instances, the collaborative performance is quite good, demonstrating diverse expertise of models and the capability of CNF in reasonably exploiting the overall model capacity.

3.3.2 Ablation Study

We conduct extensive ablation studies on TSP100 to demonstrate the effectiveness and sensitivity of our method. Note that the setups are slightly different from the

training ones (e.g., half training instances). The detailed results and setups are presented in Fig. 3.3 and Appendix A.3, respectively.

Ablation on Components. We investigate the role of each component in CNF by removing them separately. As demonstrated in Fig. 3.3(a), despite both components contribute to the collaborative performance, the neural router exhibits a bigger effect due to its favorable potentials to elegantly exploit training instances and model capacity, especially in the presence of multiple models.

Ablation on Hyperparameters. We investigate the effect of the number of trained models, which is a key hyperparameter of our method, on the collaborative performance. The results are shown in Fig. 3.3(b), where we observe that increasing the number of models can further improve the collaborative performance. However, we use $M = 3$ in the main experiments due to the trade-off between empirical performance and computational complexity.

Ablation on Routing Strategies. We further discuss different routing strategies, including neural and heuristic ones. Specifically, given the logit matrix $\mathcal{P}$ predicted by the neural router, there are various ways to distribute instances: 1) *Model choice with Top$\mathcal{K}$ (M-Top$\mathcal{K}$):* each model chooses potential instances with Top$\mathcal{K}$-largest logits, which is the default strategy ($\mathcal{K} = B$) in CNF; 2) *Model choice with sampling (M-Sample):* each model chooses potential instances by sampling from the probability distribution (i.e., scaled logits); 3-4) *Instance choice with Top$\mathcal{K}$/sampling (I-Top$\mathcal{K}$/I-Sample):* in contrast to the model choice, each instance chooses potential model(s) either by Top$\mathcal{K}$ or sampling. The probability matrix $\tilde{\mathcal{P}}$ is obtained by taking `Softmax` along the first and last dimension of $\mathcal{P}$ for model choice and instance choice, respectively. Unlike the model choice, instance choice cannot guarantee load balancing. For example, the majority of instances may choose a dominant model (if exists), leaving the remaining models underfitting and therefore weakening the ensemble effect and collaborative performance; 5) *Random:* instances are randomly distributed to each model; 6) *Self-training:* each model is trained on adversarial instances generated by itself without instance distributing. The results in Fig. 3.3(c) show that M-Top$\mathcal{K}$ performs the best.

TABLE 3.2: Generalization evaluation on synthetic TSP datasets.

	Cross-Distribution				*Cross-Size*				*Cross-Size & Distribution*			
	Rotation (100)		Explosion (100)		Uniform (50)		Uniform (200)		Rotation (200)		Explosion (200)	
	Gap	Time	Gap	Time	Gap	Time	Gap	Time	Gap	Time	Gap	Time
Concorde	0.000%	0.3m	0.000%	0.3m	0.000%	0.2m	0.000%	0.6m	0.000%	0.6m	0.000%	0.6m
LKH3	0.000%	1.2m	0.000%	1.2m	0.000%	0.4m	0.000%	3.9m	0.000%	3.3m	0.000%	3.5m
POMO (1)	0.471%	0.1m	0.238%	0.1m	0.064%	0.1m	1.658%	0.5m	2.936%	0.5m	2.587%	0.5m
POMO_AT (1)	0.640%	0.1m	0.364%	0.1m	0.151%	0.1m	2.667%	0.5m	3.462%	0.5m	2.989%	0.5m
POMO_AT (3)	0.508%	0.3m	0.263%	0.3m	0.085%	0.1m	2.362%	1.5m	3.176%	1.5m	2.688%	1.5m
POMO_HAC (3)	0.204%	0.3m	0.107%	0.3m	0.038%	0.1m	1.414%	1.5m	2.184%	1.5m	1.718%	1.5m
POMO_DivTrain (3)	0.502%	0.3m	0.255%	0.3m	0.078%	0.1m	2.356%	1.5m	3.176%	1.5m	2.707%	1.5m
CNF (3)	**0.193%**	0.3m	**0.084%**	0.3m	**0.036%**	0.1m	**1.383%**	1.5m	**2.055%**	1.5m	**1.672%**	1.5m

3.3.3 Out-of-Distribution Generalization

In contrast to other domains (e.g., vision), the set of valid problems is not just a low-dimensional manifold in a high-dimensional space, and hence the manifold hypothesis [124] does not apply to VRPs (or COPs). Therefore, it is critical for neural solvers to perform well on adversarial instances when striving for a broader out-of-distriburion (OOD) generalization in VRPs. In this section, we further evaluate the OOD generalization performance on unseen instances from both synthetic and benchmark datasets. All neural solvers are only trained on $n = 100$. The empirical results demonstrate that raising robustness against attacks through CNF can favorably promotes various forms of generalization, indicating the potential existence of neural VRP solvers with high generalization and robustness concurrently.

Synthetic Datasets. We consider three generalization settings. The results are shown in Table 3.2, where we observe that simply conducting the vanilla AT somewhat hurts the OOD generalization, while CNF can significantly improve it. Since adversarial robustness is known as a kind of local generalization property [116, 120], the improvements in OOD generalization can be viewed as a byproduct of defending against adversarial attacks and balancing the accuracy-robustness trade-off.

Benchmark Datasets. We further evaluate all neural solvers on the real-world benchmark datasets, such as TSPLIB [125] and CVRPLIB [126]. We choose representative instances within the range of $n \in [100, 1002]$, capturing diverse variations in problem scales and data distributions encountered in real-world scenarios. The results, presented in Tables A.1 and A.2, demonstrate that our method performs well across most instances.

TABLE 3.3: Performance evaluation against ROCO over 1K ATSP instances.

	Clean				Fixed Adv.			
	(x1) Gap	Time	(x16) Gap	Time	(x1) Gap	Time	(x16) Gap	Time
LKH3	0.000%	1s	0.000%	1s	0.000%	1s	0.000%	1s
Nearest Neighbour	30.481%	–	30.481%	–	31.595%	–	31.595%	–
Farthest Insertion	3.373%	–	3.373%	–	3.967%	–	3.967%	–
MatNet (1)	0.784%	0.5s	0.056%	5s	0.931%	0.5s	0.053%	5s
MatNet_AT (1)	0.817%	0.5s	0.072%	5s	0.827%	0.5s	0.046%	5s
MatNet_AT (3)	0.299%	1.5s	0.028%	15s	0.319%	1.5s	0.023%	15s
CNF (3)	**0.246%**	1.5s	**0.022%**	15s	**0.278%**	1.5s	**0.015%**	15s

3.3.4 Versatility Study

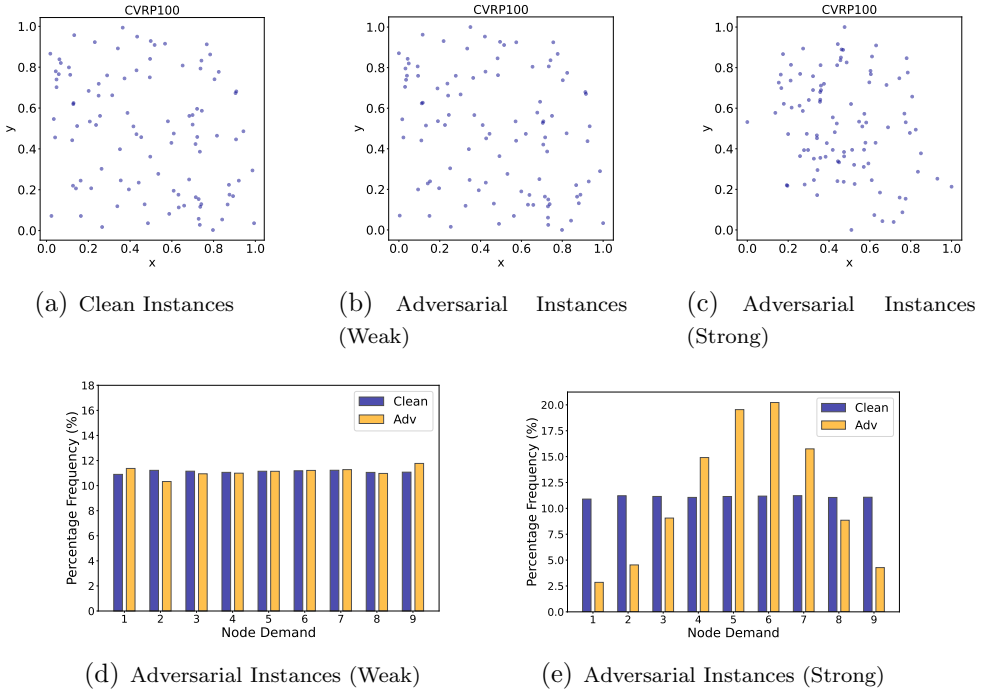
To demonstrate the versatility of CNF, we extend its application to MatNet [28] to defend against another attacker in Lu et al. [70]. Specifically, it constructs the adversarial instance by lowering the cost of a partial clean asymmetric TSP (ATSP) instance. The results are shown in Table 3.3, where we evaluate all methods on 1K ATSP instances. For neural solvers, we use the sampling with x1 and x16 instance augmentations following [28, 70]. The gaps are computed w.r.t. LKH3. Based on the results, we observe that 1) CNF is effective in mitigating the undesirable accuracy-robustness trade-off; 2) together with the main results, CNF is versatile to defend against various attacks among different neural VRP solvers.

3.3.5 Visualization

The distribution of generated adversarial instances depends on the attack method and its strength. Here, we take the attacker from Zhang et al. [66] as an example. We visualize clean instances and their corresponding adversarial instances in Fig. 3.4, where we show the spatial distribution of node locations and the percentage frequency distribution of node demands over the entire CVRP100 test dataset.

3.4 Chapter Summary

This chapter studies cross-distribution generalization through adversarial defense for neural VRP solvers, filling the gap in the current literature on this topic. We propose an ensemble-based collaborative neural framework to concurrently enhance



(a) Clean Instances (b) Adversarial Instances (Weak) (c) Adversarial Instances (Strong)

(d) Adversarial Instances (Weak) (e) Adversarial Instances (Strong)

FIGURE 3.4: Visualization of clean instances and corresponding adversarial instances generated by weak and strong attack.

performance on clean and adversarial instances. Extensive experiments demonstrate the effectiveness and versatility of our method, highlighting its potential to defend against attacks while promoting various forms of generalization of neural VRP solvers. Our work offers new insights into developing more robust and generalizable neural VRP solutions in practice.

The limitation of this work is the increased computational complexity due to the need to synergistically train multiple models. Fortunately, based on experimental results, CNF with just three models has already achieved commendable performance. It even surpasses vanilla AT trained with nine models, demonstrating a better trade-off between empirical performance and computational complexity.

Chapter 4

Generalizable Neural VRP Solvers Across Sizes and Distributions

While the previous chapter focused on cross-distribution generalization, we argue that it is more realistic to address generalization across both instance size and distribution (i.e., omni-generalization), as real-world VRP instances often vary along both dimensions. As a promising work, Manchanda et al. [54] handles this challenging setting by exploiting a meta-learning technique, i.e., Reptile [128], which only relies on the first-order derivatives for training. However, it is still far from satisfactory in that, 1) Reptile is less sample efficient as it typically needs multiple inner-loop updates to incorporate information from higher-order derivatives to achieve desirable performance. Moreover, theoretical and empirical evidence of its effectiveness are only demonstrated on the few-shot supervised learning setting [128] rather than the reinforcement learning setting, the latter of which is more favored by the neural solvers for VRPs; 2) it simply selects the training task by randomly sampling from the task set in each iteration, which overlooks the training dynamics and fails to fully make use of the diverse data information.

In this chapter, we tackle the omni-generalization issue of neural VRP solvers by training on diverse tasks, each of which relates to a unique size and distribution.

This chapter is adapted from the author's publication [127].

According to the *No Free Lunch Theorems of Machine Learning* [129], it is unrealistic to train a one-size-fits-all model that could perform well on any task. Therefore, we also resort to meta-learning [130, 131] to learn a good initialized model for fine-tuning afterwards, while bypassing the limitations in Manchanda et al. [54]. Specifically, we propose Omni-VRP, a generic meta-learning framework, which is model-agnostic and compatible with any model trained with gradient updates. It learns a good initialization of model parameters by performing meta-training on tasks, which are adaptively selected from the training task set via a hierarchical scheduler. The trained model is able to efficiently adapt to new tasks only using limited data during inference. Although effective, Omni-VRP needs the second-order derivatives to perform meta-model updates, making it computationally expensive when training on instances of large sizes. Therefore, we further develop a simple yet efficient approximation method by early stopping the usage of second-order derivatives, and only leveraging the first-order derivatives afterwards.

4.1 Preliminaries

We introduce the problem statement for VRPs and the Markov Decision Process (MDP) formulation for constructing solutions to VRP instances. Without loss of generality, we define a VRP instance of size n over a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $v_i \in \mathcal{V}$ ($\mathcal{V} = \{v_i\}_{i=1}^{n}$) represents the node (e.g., customer in TSP) and $e(v_i, v_j) \in \mathcal{E}$ represents the edge between node v_i and v_j. The solution (i.e., tour) τ is represented as a sequence of nodes in $\mathcal{V}$. In this chapter, we consider the Euclidean VRPs with the cost function $c(\cdot)$ defined as the total length of the tour. The objective of VRPs is to find the optimal tour τ^* with the *minimal* cost:

$$\tau^* = \arg\min_{\tau \in S} c(\tau|\mathcal{G}), \tag{4.1}$$

where $\mathcal{S}$ is the discrete search space that contains all the feasible tours, subject to the problem-specific constraints. Specifically, a feasible tour in TSP should visit each (customer) node exactly once and return to the starting node in the end. On top of TSP, a depot node v_0 is introduced in CVRP, where each customer node is featured by a demand δ_i, and a capacity limit Q is set for each vehicle. A tour in CVRP consists of multiple sub-tours, each of which represents a vehicle starting from the depot, visiting a subset of nodes in $\mathcal{V}$ and returning to the depot. It is

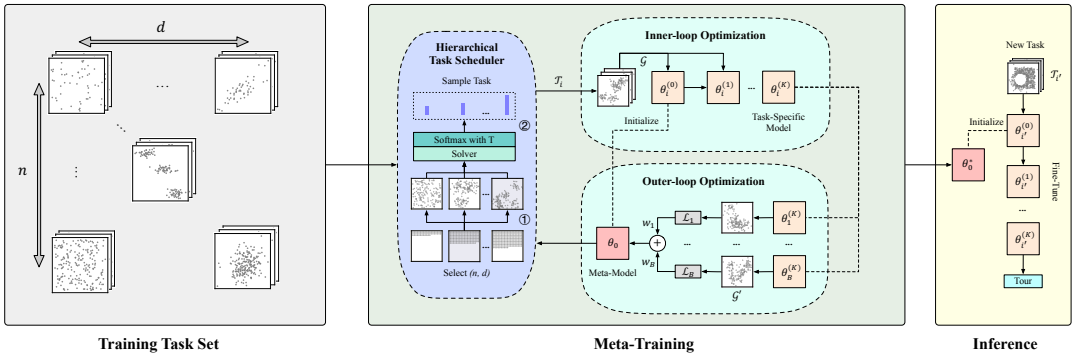


FIGURE 4.1: An overview of Omni-VRP.

feasible if each customer node is visited exactly once and the total demand in each sub-tour does not exceed the capacity limit Q.

Neural construction methods formulate the solving process of a VRP instance $\mathcal{G}$ as a MDP, where they typically parameterize the policy by an encoder–decoder structured neural network to learn node selection for constructing a solution. The encoder in the policy network outputs a global representation of the instance, which, with the representation of the context (e.g., the partial tour in construction), captures the current state. The decoder takes the global and context representations as inputs to compute the probabilities of nodes (i.e., actions) to be visited. The node is selected sequentially until a complete tour τ is constructed. Hence, the probability of the tour is factorized via the chain rule as:

$$p_\theta(\tau|\mathcal{G}) = \prod_{t=1}^{T} p_\theta(\pi_\theta(t)|\pi_\theta(<t), \mathcal{G}), \tag{4.2}$$

where $\pi_\theta(t)$ and $\pi_\theta(<t)$ are the selected node and the current partial solution at time step t, respectively; T denotes the number of total steps. The reward is defined as the negative cost of a tour, i.e., $\mathcal{R} = -c(\tau|\mathcal{G})$. To train the policy network, REINFORCE algorithm [119] is commonly used to estimate the gradient of the expected reward $\mathcal{L}(\theta|\mathcal{G}) = \mathbb{E}_{p_\theta(\tau|\mathcal{G})} c(\tau)$ such that:

$$\nabla_\theta \mathcal{L}(\theta|\mathcal{G}) = \mathbb{E}_{p_\theta(\tau|\mathcal{G})}[(c(\tau) - b(\mathcal{G}))\nabla_\theta \log p_\theta(\tau|\mathcal{G})], \tag{4.3}$$

where $b(\cdot)$ is a baseline function to reduce gradient variance.

4.2 Methodology

We introduce a generic meta-learning framework to tackle the omni-generalizable issue of neural solvers across both size and distribution in VRPs. We further develop a simple yet efficient approximation method to promote the training efficiency. Without loss of generality, we present our method by taking the meta-training of POMO [12] on TSP as an example.

4.2.1 Meta-Learning Framework

We define a task $\mathcal{T}(n, d) \sim p(\mathcal{T})$ as a class of instances with the same size $n \in \mathcal{N}$ and distribution $d \in \mathcal{D}$, where $\mathcal{N} = [n_{\min}, n_{\max}]$ is the range of problem sizes; $\mathcal{D} = \{\mathcal{D}_j\}_{j=1}^{|\mathcal{D}|}$ is a set of distributions; $p(\mathcal{T})$ is the underlying task distribution. For notation simplicity, we use $\mathcal{T}_i$ to represent a task. Inspired by MAML [132], we propose a generic meta-learning framework to improve the generalization capability of neural solvers for VRPs, which is model-agnostic and compatible with any model trained with gradient updates. The framework is illustrated in Fig. 4.1, and we elaborate its key components below.

Meta-Training. In the meta-learning framework, we aim to train a meta-model θ_0, which as a good initialized model can be efficiently adapted to new tasks during inference. Formally, we define the meta-objective as follows:

$$\theta_0^* = \arg\min_{\theta_0} \mathbb{E}_{\mathcal{T}_i \sim p(\mathcal{T})} \mathbb{E}_{\mathcal{G} \sim \mathcal{T}_i} \mathcal{L}_i(\theta_i^{(K)} | \mathcal{G}), \tag{4.4}$$

where $\theta_i^{(K)}$ is the fine-tuned model after K gradient updates of θ_0 on the task $\mathcal{T}_i$; $\mathcal{L}_i$ is the loss function on the task $\mathcal{T}_i$. In this chapter, we use the same loss function (e.g., reinforcement loss) for different tasks. To *directly* optimize this objective, the meta-training procedure comprises the inner-loop and outer-loop optimization at each iteration. The meta-training pseudocode is presented in Algorithm 2.

Inner-loop optimization: It optimizes a task-specific model iteratively, which is similar to the fine-tuning stage during inference. Specifically, given a task $\mathcal{T}_i \sim p(\mathcal{T})$, we initialize the task-specific model by the meta-model, i.e., $\theta_i^{(0)} \leftarrow \theta_0$ (line 5), and adapt it to $\mathcal{T}_i$ by performing K gradient update steps on the training instances

(line 6-11). The gradient of the loss function $\mathcal{L}_i$ at the k_{th} step is computed as:

$$\nabla_{\theta_i^{(k-1)}} \mathcal{L}_i(\theta_i^{(k-1)}) \leftarrow \frac{1}{M} \sum_{m=1}^{M} \nabla_{\theta_i^{(k-1)}} \mathcal{L}_i(\theta_i^{(k-1)} | \mathcal{G}_m), \tag{4.5}$$

where $\nabla_{\theta_i^{(k-1)}} \mathcal{L}_i(\theta_i^{(k-1)} | \mathcal{G}_m)$ can be estimated by the REINFORCE as in Eq. (4.3).

Outer-loop optimization: It optimizes the meta-model with the objective in Eq. (4.4). Concretely, for each task $\mathcal{T}_i$, the few-shot generalization performance of the task-specific model $\theta_i^{(K)}$ is evaluated on the validation instances. The meta-gradient (line 13) is obtained as follows:

$$\nabla_{\theta_0} \mathcal{L}_i(\theta_i^{(K)}) \leftarrow \frac{1}{M} \sum_{m=1}^{M} \nabla_{\theta_i^{(K)}} \mathcal{L}_i(\theta_i^{(K)} | \mathcal{G}'_m) \cdot \frac{\partial \theta_i^{(K)}}{\partial \theta_0}. \tag{4.6}$$

After conducting the inner-loop optimization on a batch of tasks $\{\mathcal{T}_i\}_{i=1}^{B}$, the meta-model θ_0 is then updated once (as in line 16). Intuitively, the inner-loop optimization serves as the task adaption stage, imitating the fine-tuning process during inference, while the outer-loop optimization updates the meta-model with the objective of maximizing the few-shot generalization performance of the task-specific model $\theta_i^{(K)}$. Therefore, after meta-training, we can get a good initialized model θ_0^* with the capability of efficient adaptation to new tasks only using limited data. Note that Eq. (4.6) needs the second-order derivatives since we expect to get the gradient direction with respect to the meta-model θ_0. We will discuss and analyze its first-order approximation methods later.

Hierarchical Task Scheduler. The aim of a task scheduler is to guide the task selection for the meta-training process, so as to improve the optimization performance. Most existing works [54, 132, 133, 134] randomly sample training tasks from the task set with a uniform probability, which assumes all tasks are equally important. It may overlook the training dynamics and cannot make full use of diverse data information. In this chapter, we propose a simple yet effective hierarchical task scheduler for VRPs.

During training, we first gradually increase the size of training task following a linear scheduler. At the e_{th} iteration of meta-training, we select tasks of the size $n_e = \lfloor n_{\min} + \min(\frac{e}{E_s}, 1) \cdot (n_{\max} - n_{\min}) \rfloor$, where E_s is the working duration of the scheduler. Then, a probability distribution over the tasks in $\{\mathcal{T}(n_e, d_j)\}_{j=1}^{|\mathcal{D}|}$ is

Algorithm 2 Meta-Training for VRPs

Input: distribution over tasks $p(\mathcal{T})$, number of tasks in a mini-batch B, number of inner-loop updates K, batch size M, step sizes of inner-loop and outer-loop optimization α, β;
Output: meta-model θ_0^*;

Initialize meta-model θ_0
while not done **do**
  $\{\mathcal{T}_i, w_i\}_{i=1}^{B}, \leftarrow$ Hierarchical task scheduler
  for $i = 1, \ldots, B$ **do**
    Initialize task-specific model $\theta_i^{(0)} \leftarrow \theta_0$
    for $k = 1, \ldots, K$ **do** ▷ *Inner-loop optimization*
      Sample training instances $\{\mathcal{G}_m\}_{m=1}^{M}$ from task $\mathcal{T}_i$
      Obtain $\nabla_{\theta_i^{(k-1)}} \mathcal{L}_i(\theta_i^{(k-1)})$ using Eq. (4.5)
      $\theta_i^{(k)} \leftarrow \theta_i^{(k-1)} - \alpha \nabla_{\theta_i^{(k-1)}} \mathcal{L}_i(\theta_i^{(k-1)})$
    end for
    Sample validation instances $\{\mathcal{G}'_m\}_{m=1}^{M}$ from task $\mathcal{T}_i$
    Obtain $\nabla_{\theta_0} \mathcal{L}_i(\theta_i^{(K)})$ using Eq. (4.6)
  end for ▷ *Outer-loop optimization*
  $\theta_0 \leftarrow \theta_0 - \beta \sum_{i=1}^{B} w_i \nabla_{\theta_0} \mathcal{L}_i(\theta_i^{(K)})$
end while

generated for training task selection, based on their hardness. The optimality gap is an appropriate metric to measure the hardness of each task. However, it is intractable to obtain optimal solutions due to the NP-hardness. Here we use LKH3 [5] to efficiently[1] obtain near-optimal solutions for a validation set of instances in a one-shot manner. The validation instances are fixed throughout the meta-training process, and hence we only need to run LKH3 once. To avoid overfitting, we only sample M instances from the validation set to calculate the relative gap $g_i = \frac{1}{M} \sum_{m=1}^{M} \frac{c(\tau_m) - c(\bar{\tau}_m)}{c(\bar{\tau}_m)}$ for each task $\mathcal{T}_i$, where τ_m and $\bar{\tau}_m$ are solutions to the m_{th} instance constructed by the current meta-model and LKH3, respectively. Accoringly, the probability of selecting each task $\mathcal{T}_i$ is defined as:

$$w_i = \frac{\exp(g_i/\eta)}{\sum_{j=1}^{|\mathcal{D}|} \exp(g_j/\eta)}, \tag{4.7}$$

where η is the temperature to control the entropy of probability distribution, from which the task is sampled. The initial probability distribution is uniform, and is updated by Eq. (4.7) periodically. After a batch of tasks $\{\mathcal{T}_i\}_{i=1}^{B}$ is sampled, we

[1]We set the maximum trials of LKH3 to 100 for efficiency.

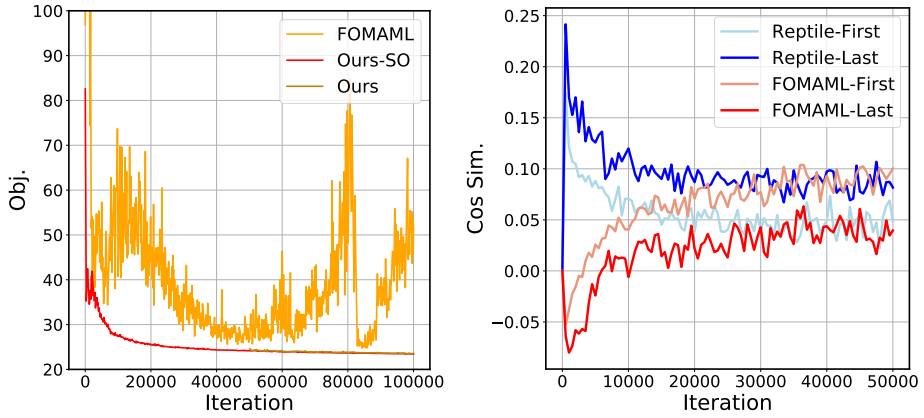


FIGURE 4.2: *Left panel:* the meta-training curves; *Right panel:* the cosine similarity between the second-order derivative and its first-order approximation at the first and last layer of the model.

normalize their weights such that $\sum_{i=1}^{B} w_i = 1$. Then, the meta-model is optimized with the weighted sum of their losses $\{w_i \mathcal{L}_i(\theta_i^{(K)})\}_{i=1}^{B}$ (line 16).

Inference. During inference, given instances sampled from a new task $\mathcal{T}_{i'}$, the trained meta-model θ_0^* could be used to approximate solutions in several ways: 1) *zero-shot:* the solution is directly constructed using the learned policy $\pi_{\theta_0^*}$ with efficient search strategies (e.g., greedy rollout); 2) *few-shot:* similar to the inner-loop optimization as shown in line 6-11 of Algorithm 2, the task-specific model $\theta_{i'}$ is initialized by θ_0^*, and is adapted to the new task $\mathcal{T}_{i'}$ by taking K gradient steps on a small set of instances (different from test instances) sampled from $\mathcal{T}_{i'}$. Then the solution is constructed using the adapted policy; 3) *active search:* the model is adapted to each test instance by learning instance-dependent parameters. Bello et al. [18] first proposes the general active search that iteratively adjusts the model parameters with the objective of increasing the likelihood of constructing high-quality solutions for each instance. However, it is extremely computationally expensive. Hottung et al. [21] proposes the efficient active search (EAS) by introducing extra instance-specific parameters (e.g., a MLP layer) for each test instance while fixing the original model parameters. We mainly consider the zero-shot and few-shot settings in our experiments, and only use EAS when evaluating on benchmark instances.

4.2.2 First-Order Approximation

The meta-gradient in Eq. (4.6) involves a gradient through a gradient (i.e., second-order derivative), and therefore it is computationally expensive to obtain due to the calculation of Hessian-vector products. To tackle this issue, Finn et al. [132] proposes a first-order approximation method (i.e., FOMAML) which simply drops the second-order term. The empirical evidence of its effectiveness has been verified in the few-shot supervised learning, while lacking in the more complex reinforcement learning. Specifically, the first-order approximation of meta-model update can be expressed as follows:

$$\theta_0 \leftarrow \theta_0 - \beta \sum_{i=1}^{B} w_i \nabla_{\theta_i} \mathcal{L}_i(\theta_i^{(K)}). \tag{4.8}$$

However, as shown in the left panel of Fig. 4.2, we empirically observe that meta-training POMO from scratch with Eq. (4.8) may induce fluctuating validation performance. The unstable meta-training may be attributed to the deviation of the first-order approximation (Eq. (4.8)) from the (ground-truth) gradient direction of the second-order term (line 16 in Algorithm 2). Intuitively, $\text{sign}(\nabla_{\theta_i} \mathcal{L}_i(\theta_i^{(K)}))$ is the (steepest) descent direction for the task-specific model $\theta_i^{(K)}$, but not necessarily the descent direction for the meta-model θ_0, especially at the *early stage* of meta-training when the optimization tends to be unstable. To justify our hypothesis, we show the cosine similarities of the gradient directions for meta-model updates between our method and others in the right panel of Fig. 4.2. The detailed experimental setups are presented in Appendix B.3. We observe that $\text{sign}(\nabla_{\theta_i} \mathcal{L}_i(\theta_i^{(K)}))$ cannot approximate $\text{sign}(\nabla_{\theta_0} \mathcal{L}_i(\theta_i^{(K)}))$ well (i.e., with a negative cosine similarity) at the early stage of meta-training, while gradually having the positive correlation as the training progresses. Therefore, in order to reduce the computational cost and stabilize the meta-training, we develop a simple yet efficient method by early stopping the usage of second-order derivatives. Specifically, we start the meta-model updates with second-order derivatives, and switch to the first-order ones (i.e., by replacing line 16 with Eq. (4.8)) when the optimization tends to be stable. Recently, Manchanda et al. [54] leverages another first-order method called Reptile [128], with the form of the meta-model update as follows:

$$\theta_0 \leftarrow \theta_0 + \beta \sum_{i=1}^{B} w_i (\theta_i^{(K)} - \theta_0). \tag{4.9}$$

However, it needs multiple inner-loop updates to effectively incorporate information from higher-order derivatives of the loss function so as to achieve satisfactory performance (as shown in Appendix B.1), making it less sample efficient. Moreover, similar to Nichol et al. [128] which observes negative results after applying Reptile to the reinforcement learning setting, we also empirically find its weak performance (see Meta-POMO in Section 4.3). In contrast, our method could achieve decent performance only running a single inner-loop update.

4.3 Experiments

To demonstrate the effectiveness of the proposed framework, we apply it to POMO [12], which is a strong construction-based neural solver. We consider two representative VRP problems (i.e., TSP and CVRP). Moreover, we also evaluate the generalizability of our method on L2D [83] as shown in Section 4.3.3.

Baselines. 1) *Traditional VRP solvers:* we employ Concorde [135] and LKH3 [5] for solving TSP, and the hybrid genetic search (HGS) [6] and LKH3 for CVRP. 2) *Neural solvers:* we compare our method with POMO-based methods, including the original POMO [12], AMDKD-POMO [68] and Meta-POMO [54] for TSP and CVRP. AMDKD-POMO is a recent method that improves the cross-distribution generalization of POMO using knowledge distillation. Meta-POMO uses Reptile to improve the generalization performance across both size and distribution. For a fair comparision, we re-train all methods following our training setups. Note that the setting of Meta-POMO is the most relevant to ours. We also show the results of their open-sourced pretrained models (i.e., POMO* and AMDKD-POMO*) in Table 4.1, with the aim of demonstrating the severe generalization issue of current neural solvers rather than the direct comparison. Specifically, POMO* is trained on instances with a fixed size and distribution (i.e., $n = 100$ with the uniform distribution), and AMDKD-POMO* is adaptively distilling from teacher models trained on fixed-sized instances following different distributions (i.e., $n = 100$ with the uniform, cluster and mixed distributions). More implementation details are provided in Appendix B.3.

Training Setups. We follow most of the setups in Kwon et al. [12]. For our method, Adam optimizer [136] is used in both inner-loop and outer-loop optimization, with the weight decay of $1e-6$. The step sizes (learning rates) are $\alpha=\beta=1e-4$, and decayed by 10 in the last 10% iterations to achieve a faster convergence. The batch size is $M=64$ ($M=32$ for instances with sizes larger than 150). The training task set consists of hundreds of (i.e., 341) tasks, with diverse sizes $\mathcal{N}=[50,200]$ and distributions (i.e., uniform (U) and gaussian mixture (GM) distributions). More details about the generation of training and test data are presented in Appendix B.2. Similar to Finn et al. [132], we simply set $B=K=1$ and empirically observe strong performance. As suggested by Kwon et al. [12], most of the training is already completed by 200 epochs (i.e., 20M instances) for POMO. We give more instances due to our complicated problem setting. Specifically, we re-train all methods for roughly the same number of instances (i.e., 32M) sampled from our training task set. For example, we re-train POMO for roughly 500K iterations (i.e., gradient updates). For our method, one iteration of meta-training consists of a pair of inner-loop and out-loop optimization, which needs two batches of instances. Therefore, for a fair comparison, we train our method for roughly 250K iterations. For the hierarchical task scheduler, we set $\eta=1$ and $E_s=225$K. It evaluates the hardness and updates the weight of each task every 100 iterations. Due to the training efficiency, we regard meta-training with the first-order approximation (i.e., *Ours*) as the default method, which uses the second-order derivatives in the first 50K iterations, and switch to the first-order ones afterwards. In specific, the meta-training with full second-order derivatives (i.e., *Ours-SO*) needs roughly 5 days and 53GB GPU memory for TSP (6 days and 71GB GPU memory for CVRP), while *Ours* needs 2.5 days and 17GB GPU memory for TSP (3 days and 25GB GPU memory for CVRP).

Inference Setups. For all neural solvers, we use the greedy rollout with x8 instance augmentations following Kwon et al. [12]. We report the average results over the test dataset containing 1K instances. The reported time is the total time to solve the entire test dataset. The reported gaps are computed with respect to the traditional VRP solvers (i.e., Concorde for TSP, and HGS for CVRP). Specifically, we evaluate the effectiveness of our method on the *zero-shot* and *few-shot* (FS) settings. For the zero-shot setting, the trained model is directly used to construct the solutions. We further evaluate meta-learning based methods (i.e., Meta-POMO, Ours-SO and Ours) on the few-shot setting, where we fine-tune the

TABLE 4.1: Evaluation on cross-size or distribution generalization.

	Method	**Cross-Distribution Generalization** (1K ins.)						**Cross-Size Generalization** (1K ins.)					
		$(200, GM_2^5)$		$(200, R)$		$(200, E)$		$(300, U)$		$(300, GM_3^{10})$		$(300, GM_7^{50})$	
		Obj. (Gap)	Time	Obj. (Gap)	Time	Obj. (Gap)	Time	Obj. (Gap)	Time	Obj. (Gap)	Time	Obj. (Gap)	Time
TSP	Concorde	8.78 (0.00%)	0.6m	8.20 (0.00%)	0.5m	8.09 (0.00%)	0.5m	12.95 (0.00%)	1.4m	9.47 (0.00%)	1.2m	5.63 (0.00%)	1.0m
	LKH3	8.78 (0.00%)	3.3m	8.20 (0.00%)	3.3m	8.09 (0.00%)	3.5m	12.95 (0.00%)	5.9m	9.47 (0.00%)	12.5m	5.63 (0.01%)	18.5m
	POMO*	9.36 (6.67%)	0.5m	8.41 (2.66%)	0.5m	8.28 (2.35%)	0.5m	13.82 (6.70%)	1.5m	10.73 (13.38%)	1.5m	6.54 (16.04%)	1.5m
	AMDKD-POMO*	9.05 (2.97%)	0.5m	8.41 (2.57%)	0.5m	8.30 (2.61%)	0.5m	13.97 (7.83%)	1.5m	10.25 (8.22%)	1.5m	6.25 (11.00%)	1.5m
	POMO	**9.01 (2.56%)**	0.5m	8.37 (2.13%)	0.5m	8.24 (1.85%)	0.5m	13.54 (4.51%)	1.5m	9.88 (4.27%)	1.5m	5.83 (3.46%)	1.5m
	AMDKD-POMO	9.10 (3.56%)	0.5m	8.47 (3.32%)	0.5m	8.38 (3.55%)	0.5m	13.74 (6.08%)	1.5m	9.97 (5.30%)	1.5m	6.00 (6.44%)	1.5m
	Meta-POMO	9.03 (2.78%)	0.5m	8.39 (2.31%)	0.5m	8.25 (2.00%)	0.5m	13.50 (4.23%)	1.5m	9.89 (4.38%)	1.5m	5.80 (2.94%)	1.5m
	Ours-SO	9.01 (2.59%)	0.5m	**8.36 (1.99%)**	0.5m	**8.23 (1.72%)**	0.5m	**13.37 (3.22%)**	1.5m	**9.82 (3.72%)**	1.5m	**5.78 (2.61%)**	1.5m
	Ours	9.02 (2.71%)	0.5m	8.37 (2.14%)	0.5m	8.24 (1.86%)	0.5m	13.40 (3.42%)	1.5m	9.84 (3.89%)	1.5m	5.79 (2.75%)	1.5m
	Meta-POMO+FS ($K = 1$)	9.02 (2.74%)	2.0m	8.38 (2.24%)	2.0m	8.25 (1.92%)	2.0m	13.46 (3.87%)	6.8m	9.86 (4.11%)	6.8m	5.78 (2.64%)	6.8m
	Meta-POMO+FS ($K = 10$)	9.02 (2.67%)	15.7m	8.38 (2.17%)	15.7m	8.24 (1.83%)	15.7m	13.42 (3.58%)	0.9h	9.84 (3.91%)	0.9h	**5.77 (2.46%)**	0.9h
	Ours-SO+FS ($K = 1$)	**9.01 (2.53%)**	2.0m	**8.36 (1.95%)**	2.0m	**8.22 (1.63%)**	2.0m	**13.35 (3.05%)**	6.8m	**9.81 (3.57%)**	6.8m	**5.77 (2.47%)**	6.8m
	Ours+FS ($K = 1$)	9.01 (2.60%)	2.0m	8.37 (2.05%)	2.0m	8.23 (1.74%)	2.0m	13.37 (3.19%)	6.8m	9.82 (3.69%)	6.8m	5.78 (2.52%)	6.8m
CVRP	HGS	18.89 (0.00%)	0.7h	19.36 (0.00%)	0.6h	19.45 (0.00%)	0.5h	25.61 (0.00%)	1.0h	22.20 (0.00%)	1.6h	22.11 (0.00%)	0.9h
	LKH3	19.09 (1.06%)	0.6h	19.55 (0.99%)	0.6h	19.66 (1.04%)	0.5h	25.97 (1.39%)	0.6h	22.46 (1.19%)	0.7h	22.24 (0.59%)	0.6h
	POMO*	19.93 (5.60%)	0.6m	20.45 (5.74%)	0.6m	20.54 (5.69%)	0.6m	28.72 (12.32%)	1.8m	24.81 (12.00%)	1.8m	24.33 (10.22%)	1.8m
	AMDKD-POMO*	20.29 (7.59%)	0.6m	20.89 (8.07%)	0.6m	20.96 (7.92%)	0.6m	30.49 (19.17%)	1.8m	25.65 (15.92%)	1.8m	24.41 (10.60%)	1.8m
	POMO	19.47 (3.12%)	0.6m	19.99 (3.32%)	0.6m	20.12 (3.49%)	0.6m	27.07 (5.74%)	1.8m	23.25 (4.80%)	1.8m	22.80 (3.16%)	1.8m
	AMDKD-POMO	19.58 (3.69%)	0.6m	20.07 (3.72%)	0.6m	20.20 (3.93%)	0.6m	26.94 (5.20%)	1.8m	23.28 (4.92%)	1.8m	22.82 (3.27%)	1.8m
	Meta-POMO	19.48 (3.19%)	0.6m	20.01 (3.40%)	0.6m	20.15 (3.65%)	0.6m	26.87 (4.94%)	1.8m	23.09 (4.07%)	1.8m	22.75 (2.93%)	1.8m
	Ours-SO	**19.38 (2.66%)**	0.6m	**19.91 (2.87%)**	0.6m	**20.05 (3.13%)**	0.6m	26.67 (4.15%)	1.8m	**22.93 (3.33%)**	1.8m	**22.60 (2.23%)**	1.8m
	Ours	19.39 (2.69%)	0.6m	19.91 (2.88%)	0.6m	20.07 (3.21%)	0.6m	**26.65 (4.10%)**	1.8m	22.93 (3.35%)	1.8m	22.61 (2.27%)	1.8m
	Meta-POMO+FS ($K = 1$)	19.43 (2.92%)	2.4m	19.96 (3.13%)	2.4m	20.10 (3.39%)	2.4m	26.71 (4.32%)	8.2m	22.99 (3.64%)	8.2m	22.70 (2.72%)	8.2m
	Meta-POMO+FS ($K = 10$)	19.41 (2.83%)	18.8m	19.94 (3.03%)	18.8m	20.08 (3.27%)	18.8m	26.65 (4.07%)	1.1h	22.95 (3.43%)	1.1h	22.67 (2.55%)	1.1h
	Ours-SO+FS ($K = 1$)	**19.38 (2.65%)**	2.4m	**19.90 (2.81%)**	2.4m	**20.03 (3.00%)**	2.4m	26.61 (3.93%)	8.2m	**22.90 (3.21%)**	8.2m	**22.58 (2.16%)**	8.2m
	Ours+FS ($K = 1$)	19.38 (2.66%)	2.4m	19.90 (2.83%)	2.4m	20.04 (3.05%)	2.4m	**26.61 (3.92%)**	8.2m	22.91 (3.23%)	8.2m	22.59 (2.20%)	8.2m

meta-model for K iterations only using extra 1K instances sampled from the test task (0.003% of instances used for meta-training). The instances are augmented following Kwon et al. [12]. Note that the instances for fine-tuning are different from the test ones. The Adam optimizer is used with the learning rate of $\alpha = 1e-5$ and the weight decay of $1e-6$. Moreover, we further combine our method with EAS [21] when evaluating on benchmark instance.

4.3.1 Performance Evaluation

Below, we demonstrate the effectiveness of our method on synthetic and real-world datasets. For the synthetic data, we evaluate the generalization performance across size, distribution and the both.

Cross-Size or Distribution Generalization. We first consider a simple setting where either the cross-size or distribution generalization is evaluated. For the cross-distribution setting, we test on instances of size $n = 200 \in [50, 200]$, while following diverse distributions that are unseen during training. Note that we do not strictly choose the test tasks sampled from the presumed training task distribution $p(\mathcal{T})$ since it only covers a small part of the entire problem space. Therefore,

TABLE 4.2: Evaluation on cross-size and distribution generalization.

	Method	**Cross-Size and Distribution Generalization** (1K ins.)											
		$(300, R)$		$(300, E)$		$(500, R)$		$(500, E)$		$(1000, R)$		$(1000, E)$	
		Obj. (Gap)	Time	Obj. (Gap)	Time	Obj. (Gap)	Time	Obj. (Gap)	Time	Obj. (Gap)	Time	Obj. (Gap)	Time
TSP	Concorde	9.79 (0.00%)	1.2m	9.48 (0.00%)	1.5m	12.39 (0.00%)	5.0m	11.73 (0.00%)	5.8m	17.09 (0.00%)	0.7h	15.66 (0.00%)	0.9h
	LKH3	9.79 (0.00%)	6.0m	9.48 (0.00%)	6.8m	12.39 (0.00%)	11.8m	11.73 (0.00%)	13.8m	17.09 (0.00%)	0.4h	15.66 (0.00%)	0.5h
	POMO	10.23 (4.43%)	1.5m	9.88 (4.20%)	1.5m	13.63 (10.00%)	6.0m	12.89 (9.88%)	6.0m	20.74 (21.38%)	0.8h	18.94 (20.97%)	0.8h
	AMDKD-POMO	10.35 (5.69%)	1.5m	10.06 (6.15%)	1.5m	13.74 (10.85%)	6.0m	13.08 (11.52%)	6.0m	20.73 (21.25%)	0.8h	19.08 (21.85%)	0.8h
	Meta-POMO	10.22 (4.37%)	1.5m	9.87 (4.14%)	1.5m	13.56 (9.41%)	6.0m	12.84 (9.44%)	6.0m	20.51 (19.97%)	0.8h	18.77 (19.88%)	0.8h
	Ours-SO	**10.14 (3.54%)**	1.5m	**9.78 (3.13%)**	1.5m	**13.39 (8.07%)**	6.0m	**12.64 (7.73%)**	6.0m	**20.37 (19.20%)**	0.8h	**18.59 (18.74%)**	0.8h
	Ours	10.16 (3.74%)	1.5m	9.80 (3.35%)	1.5m	13.42 (8.30%)	6.0m	12.66 (7.90%)	6.0m	20.40 (19.36%)	0.8h	18.60 (18.80%)	0.8h
	Meta-POMO+FS ($K = 1$)	10.18 (3.96%)	6.8m	9.83 (3.70%)	6.8m	13.34 (7.60%)	0.5h	12.63 (7.66%)	0.5h	19.58 (14.52%)	6.5h	17.92 (14.48%)	6.5h
	Meta-POMO+FS ($K = 10$)	10.16 (3.69%)	0.9h	9.80 (3.41%)	0.9h	13.23 (6.75%)	4.1h	12.54 (6.84%)	4.1h	–	–	–	–
	Ours-SO+FS ($K = 1$)	**10.12 (3.32%)**	6.8m	**9.76 (2.91%)**	6.8m	**13.19 (6.45%)**	0.5h	**12.45 (6.11%)**	0.5h	**19.53 (14.28%)**	6.5h	17.79 (13.65%)	6.5h
	Ours+FS ($K = 1$)	10.13 (3.41%)	6.8m	9.77 (3.05%)	6.8m	13.20 (6.52%)	0.5h	12.51 (6.64%)	0.5h	19.53 (14.30%)	6.5h	**17.75 (13.38%)**	6.5h
CVRP	HGS	22.40 (0.00%)	1.3h	23.02 (0.00%)	1.3h	26.62 (0.00%)	4.5h	26.89 (0.00%)	4.6h	32.36 (0.00%)	30.9h	32.01 (0.00%)	37.7h
	LKH3	22.68 (1.28%)	0.7h	23.32 (1.28%)	0.7h	27.06 (1.69%)	0.9h	27.32 (1.61%)	0.9h	33.16 (2.51%)	1.6h	32.78 (2.43%)	1.6h
	POMO	23.56 (5.30%)	1.8m	24.20 (5.30%)	1.8m	29.06 (9.48%)	6.9m	29.29 (9.29%)	6.9m	39.33 (22.44%)	1.0h	38.63 (21.73%)	1.0h
	AMDKD-POMO	23.54 (5.18%)	1.8m	24.24 (5.39%)	1.8m	29.06 (9.32%)	6.9m	29.33 (9.29%)	6.9m	39.72 (23.17%)	1.0h	38.86 (21.90%)	1.0h
	Meta-POMO	23.39 (4.54%)	1.8m	24.08 (4.71%)	1.8m	28.53 (7.34%)	6.9m	28.80 (7.32%)	6.9m	37.46 (16.09%)	0.9h	36.85 (15.52%)	0.9h
	Ours-SO	23.24 (3.83%)	1.8m	**23.93 (4.07%)**	1.8m	28.34 (6.60%)	6.7m	28.63 (6.69%)	6.7m	37.30 (15.62%)	0.8h	36.61 (14.83%)	0.8h
	Ours	**23.23 (3.79%)**	1.8m	23.94 (4.08%)	1.8m	**28.29 (6.41%)**	6.7m	**28.60 (6.56%)**	6.7m	**37.02 (14.73%)**	0.8h	**36.40 (14.15%)**	0.8h
	Meta-POMO+FS ($K = 1$)	23.29 (4.05%)	8.2m	23.96 (4.20%)	8.2m	28.13 (5.80%)	0.6h	28.43 (5.90%)	0.6h	36.14 (11.93%)	7.5h	35.78 (12.07%)	7.5h
	Meta-POMO+FS ($K = 10$)	23.23 (3.79%)	1.1h	23.90 (3.92%)	1.1h	**27.95 (5.14%)**	4.9h	**28.24 (5.19%)**	4.7h	–	–	–	–
	Ours-SO+FS ($K = 1$)	23.19 (3.61%)	8.2m	**23.87 (3.78%)**	8.2m	28.03 (5.41%)	0.6h	28.33 (5.52%)	0.6h	35.69 (10.52%)	7.4h	35.40 (10.92%)	7.4h
	Ours+FS ($K = 1$)	**23.19 (3.59%)**	8.2m	23.87 (3.79%)	8.2m	**28.01 (5.34%)**	0.6h	**28.31 (5.44%)**	0.6h	**35.60 (10.26%)**	7.4h	**35.25 (10.45%)**	7.4h

we also evaluate all methods on several complex distributions, e.g., rotation (R) and explosion (E) distributions [137]. For the cross-size setting, we evaluate on instances of the size $n = 300 \notin [50, 200]$ following distributions used in training. Besides the zero-shot setting, we further compare with another meta-learning based method (i.e., Meta-POMO) on the few-shot setting, where we fine-tune the learned model for K steps only using limited data. The detailed results are shown in Table 4.1, where we observe that our method can achieve superior performance on both settings. The inferior performance of AMDKD-POMO may be attributed to its design for the trivial problem setting and sample inefficiency. While it is specialized for the cross-distribution generalization, its original problem setting is much easier than ours, with only three distributions on the fixed size (i.e., 100) considered during training. To achieve satisfactory performance, a good pretrained model for each training task is needed, which requires a huge amount of training instances.

Cross-Size and Distribution Generalization. We further evaluate all methods on a much more complex setting, where the generalization across both size and distribution is considered. Specifically, we choose the test task with the unseen size $n \in [300, 500, 1000]$ and distribution $d \in [R, E]$ during training. The results are presented in Table 4.2, where we observe our method has consistently better performance than baselines. Notably, our method achieves superior results on the

TABLE 4.3: Ablation study on Components.

	$(200, GM_2^5)$	$(300, U)$	$(500, R)$	$(1000, E)$
POMO	3.12%	5.74%	9.48%	21.73%
+ task scheduler	**2.67%**	4.44%	7.53%	19.03%
+ meta-training	3.08%	4.79%	7.02%	15.39%
Ours	2.69%	**4.10%**	**6.41%**	**14.15%**

large-scale CVRP1000 task with totally unseen distributions, showing a strong omni-generalization capability.

Results on Benchmark Datasets. We further evaluate all methods on the well-konwn benchmark datasets TSPLIB [125] and CVRPLIB (Set-X [126] and Set-XML100 [138]). Detailed results can be found in Tables B.1 and B.2, where we observe our method performs well in most cases.

4.3.2 Analyses

In this section, we conduct further analyses, including the ablation studies and few-shot experiments, to demonstrate the effectiveness of the proposed framework.

Ablation Study on Components. In Section 4.3.1, we have shown the effect of the first-order approximation. Compared with the full second-order method, it could achieve similar or even better zero-shot and few-shot performance, and meanwhile greatly reduce the training complexity. Here, following the training setups presented in Section 4.3, we further conduct the ablation study on CVRP to demonstrate the benefit of each component in our framework. The results are shown in Table 4.3, where we observe that the meta-training significantly improves POMO (zero-shot) performance on the large-scale instances, and the task scheduler can further boost the overall performance of the meta-training.

Efficient Adaptation. As shown in Table 4.1 and 4.2, given the same amount of training instances, our method achieves strong zero-shot performance, which enables efficient adaptation to a new task afterwards. To demonstrate it, we further conduct two experiments on the adaptation to CVRP (500, R) only using 100 and 1000 instances. As shown in Fig. 4.3, our methods can be efficiently adapted to the new task, while Meta-POMO needs to run multiple steps to achieve similar

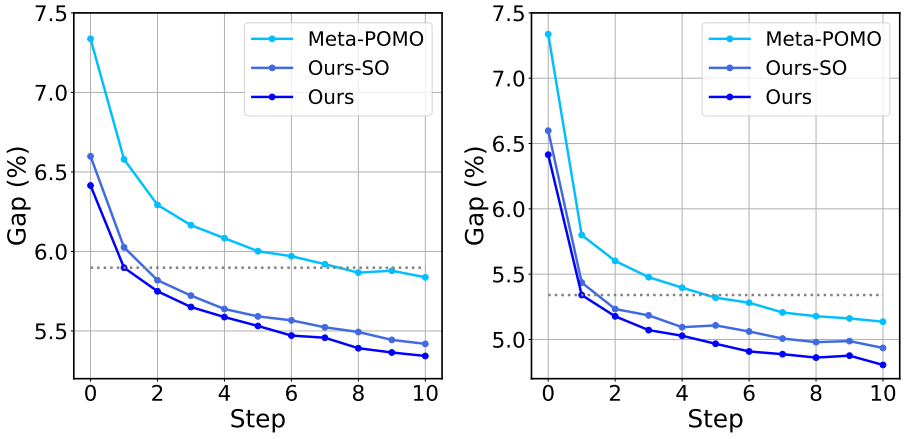


FIGURE 4.3: Adaptation to CVRP (500, R) test task using (a) 100 instances; (b) 1000 instances.

few-shot performance (e.g., the dotted line for $K = 1$) to our method. Moreover, we observe that the number of instances is crucial to the few-shot performance in the reinforcement learning. Meta-POMO may need more instances in order to achieve strong few-shot performance.

4.3.3 Generalizability

To evaluate the generalizability of the proposed framework, we further apply it to L2D [83], which is an improvement-based method outperforming LKH3 on large-scale CVRP instances. Specifically, it decomposes the large-scale problem instance into several subproblems, which are selected by a (supervised) learned policy, and uses an existing solver (e.g., LKH3 or HGS) to solve each subproblem. We train the model on the omni-generalization setting, where the training task set consists of various sizes and distributions. Concretely, we train a regression model (rather than a classification model) since: (1) the training of the regression model is more efficient (i.e., around 6 hrs); (2) it has better flexibility in training multiple sizes and distributions, which is quite suitable for the omni-generalization setting. We use the datasets provided by Li et al. [83] to construct the training task set. Concretely, it contains six mixed CVRP distributions with $n \in \{500, 1000\} \times d \in \{3, 5, 7\}$, where n is the problem size and d is the cluster center. We use HGS as the subsolver, and keep the other settings the same as Li et al. [83]. During the evaluation, we set the number of runs to 1 for each instance, and set the time limit for solving each subproblem to 1s. For a fair comparison, we retrain the regression model (i.e., L2D

TABLE 4.4: Performance evaluation on L2D.

	In-Distribution			*Cross-Size*			*Cross-Distribution*	*Cross-Size & Distribution*
	mixed_d3_n1000	mixed_d5_n1000	mixed_d7_n1000	mixed_d3_n2000	mixed_d5_n2000	mixed_d7_n2000	cluster_n1000	cluster_n2000
L2D (512)	142.53 (2)	119.33 (2)	**102.96 (5)**	287.64 (2)	245.55 (3)	201.84 (1)	149.43 (2)	194.70 (2)
L2D (2048)	140.90 (1)	161.61 (2)	208.95 (1)	312.07 (0)	194.84 (3)	300.44 (0)	**81.21 (4)**	265.26 (1)
Ours (512)	**80.95 (7)**	**95.53 (6)**	124.95 (4)	**101.28 (8)**	**139.44 (4)**	**96.03 (9)**	91.96 **(4)**	**97.18 (7)**

with the batch size of 512 and 2048), and meta-train a regression model (i.e., Ours with the batch size of 512) on the training task set. We show the average cost over 10 instances on each test dataset, and the number of achieved best solutions (in brackets) among all methods. As shown in Table 4.4, our method could further improve the generalization of L2D when meta-training with diverse tasks in terms of sizes and distributions, demonstrating the effectiveness and generalizability of the proposed framework.

4.4 Chapter Summary

This chapter studies the omni-generalization issue of neural solvers across both problem size and distribution in VRPs. We propose a generic meta-learning framework to tackle this issue, which is model-agnostic and compatible with any model trained with gradient updates. We further provide analyses of the first-order approximation methods on the reinforcement learning setting, and propose a simple yet efficient method to reduce the meta-training complexity.

The limitations of this work are the training efficiency and scalability. However, they heavily depend on the base model and meta-learning algorithm. If a pretrained model exists, it would be better to conduct meta-training on it. We leave advanced algorithms and other neural solvers to future work.

Chapter 5

Generalizable Neural VRP Solvers Across Problems

The previous two chapters investigate cross-distribution and cross-size generalization within a single problem type. However, prevailing neural solvers require network structures tailored and trained independently for each specific VRP, instigating prohibitive training overhead and less practicality when facing multiple VRPs. In this chapter, we aim to develop a unified neural solver, which can be trained for solving a range of VRP variants simultaneously, and has decent zero-shot generalization capability on unseen VRPs. A few recent works explore similar problem settings. Wang and Yu [72] applies multi-armed bandits to solve multiple VRPs, while Lin et al. [73] adapts the model pretrained on one base VRP to target VRPs by efficient fine-tuning. They fail to achieve zero-shot generalization to unseen VRPs due to the dependence on networks structured for predetermined problem variants. Liu et al. [74] empowers the neural solver with such generalizability by the compositional zero-shot learning [140], which treats VRP variants as different combinations of a set of underlying attributes and uses a shared network to learn their representations. However, it still leverages existing network structure proposed for simple VRPs, which is limited by its model capacity.

In this chapter, motivated by the recent advancement of LLMs [141, 142, 143], we propose a multi-task VRP solver with mixture-of-experts (MVMoE). Typically, a mixture-of-expert (MoE) layer replaces a feed-forward network (FFN) with several

This chapter is adapted from the author's publication [139].

'experts' in a Transformer-based model, which are a group of FFNs with respective trainable parameters. An input to the MoE layer is routed to specific expert(s) by a gating network, and only parameters in selected expert(s) are activated (i.e., conditional computation [144, 145]). In this manner, partially activated parameters can effectively enhance the model capacity without a proportional increase in computation, making the training and deployment of LLMs viable. Therefore, towards a more generic and powerful neural solver, we first propose an MoE-based neural VRP solver, and present a hierarchical gating mechanism for a good trade-off between empirical performance and computational complexity. We choose the setting from Liu et al. [74] as a test bed due to its potential to solve an exponential number of new VRP variants as any combination of the underlying attributes.

5.1 Preliminaries

We first present the definition of CVRP, and then introduce its variants featured by additional constraints. Afterwards, we delineate recent construction-based neural solvers for VRPs [11, 12].

VRP Variants. We define a CVRP instance of size n over a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V}$ includes a depot node v_0 and customer nodes $\{v_i\}_{i=1}^{n}$, and $\mathcal{E}$ includes edges $e(v_i, v_j)$ between node v_i and v_j $(i \neq j)$. Each customer node is associated with a demand δ_i, and a capacity limit Q is set for each vehicle. The solution (i.e., tour) τ is represented as a sequence of nodes, consisting of multiple sub-tours. Each sub-tour represents that a vehicle starts from the depot, visits a subset of customer nodes and returns to the depot. The solution is feasible if each customer node is visited exactly once, and the total demand in each sub-tour does not exceed the capacity limit Q. We consider the Euclidean space with the cost function $c(\cdot)$ defined as the total length of the tour. The objective is to find the optimal tour τ^* with the minimal cost: $\tau^* = \arg\min_{\tau \in \Phi} c(\tau|\mathcal{G})$, where Φ is the discrete search space that contains all feasible tours.

On top of CVRP (featured by the capacity constraint *(C)*), several VRP variants involve additional practical constraints. 1) *Open Route (O):* The vehicle does not need to return to the depot v_0 after visiting customers; 2) *Backhaul (B):* The demand δ_i is positive in CVRP, representing a vehicle unloads goods at the

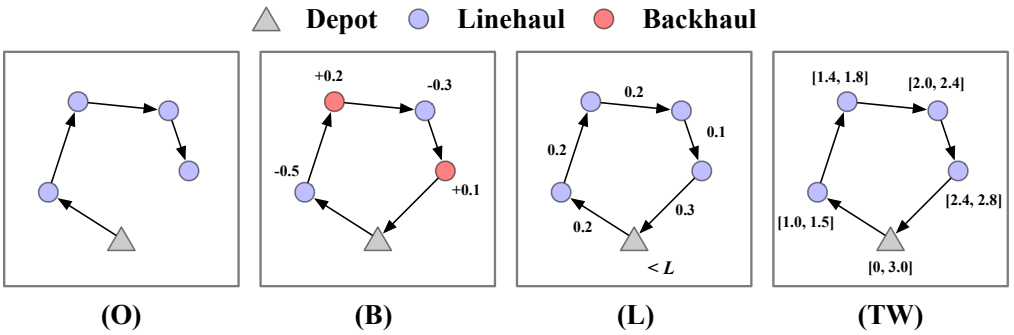


FIGURE 5.1: Illustrations of sub-tours with various constraints: open route (O), backhaul (B), duration limit (L), and time window (TW).

customer node. In practice, a customer can have a negative demand, requiring a vehicle to load goods. We name the customer nodes with $\delta_i > 0$ as linehauls and the ones with $\delta_i < 0$ as backhauls. Hence, VRP with backhaul allows the vehicle traverses linehauls and backhauls in a mixed manner, without strict precedence between them; 3) *Duration Limit (L):* To maintain a reasonable workload, the cost (i.e., length) of each route is upper bounded by a predefined threshold; 4) *Time Window (TW):* Each node $v_i \in \mathcal{V}$ is associated with a time window $[e_i, l_i]$ and a service time s_i. A vehicle must start serving customer v_i in the time slot from e_i to l_i. If the vehicle arrives earlier than e_i, it has to wait until e_i. All vehicles must return to the depot v_0 no later than l_0. The aforementioned constraints are illustrated in Fig. 5.1. By combining them, we can obtain 16 typical VRP variants, which are summarized in Table C.1. Note that the combination is not a trivial addition of different constraints. For example, when the open route is coupled with the time window, the vehicle does not need to return to the depot, and hence the constraint imposed by l_0 at the depot is relaxed. We present more details of VRP variants and the associated data generation process in Appendix C.1.

Learning to Solve VRPs. Typical neural solvers [11, 12] parameterize the solution construction policy by an attention-based neural network π_θ, which is trained to generate a solution in an autoregressive way. The feasibility of the generated solution is guaranteed by the masking mechanism during decoding. Without loss of generality, we consider RL training paradigm, wherein the solution construction process is formulated as a Markov Decision Process. Given an input instance, the encoder processes it and attains all node embeddings, which, with the context representation of the constructed partial tour, represent the current state. The decoder takes them as inputs and outputs the probabilities of valid nodes (i.e.,

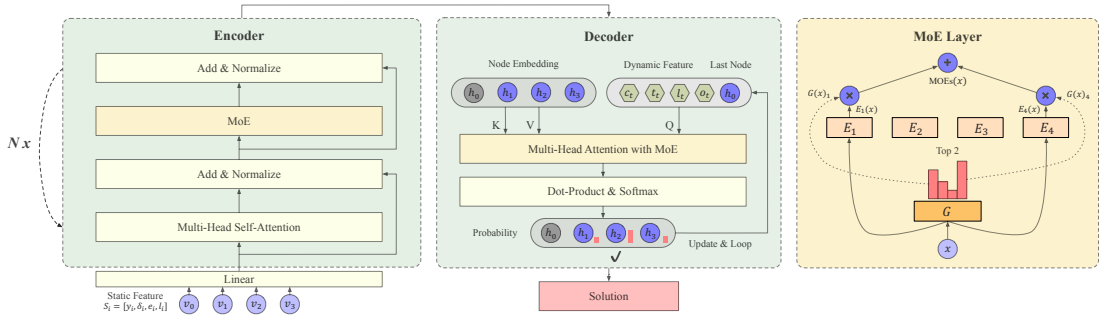


FIGURE 5.2: An overview of MVMoE.

actions) to be selected. After a complete solution τ is constructed, its probability can be factorized via the chain rule such that $p_\theta(\tau|\mathcal{G}) = \prod_{t=1}^{T} p_\theta(\pi_\theta^{(t)}|\pi_\theta^{(<t)}, \mathcal{G})$, where $\pi_\theta^{(t)}$ and $\pi_\theta^{(<t)}$ denote the selected node and constructed partial tour at step t, and T is the number of total steps. The reward is defined as the negative tour length, i.e., $\mathcal{R} = -c(\tau|\mathcal{G})$. Given a baseline function $b(\cdot)$ for training stability, the policy network π_θ is often trained by REINFORCE [119] algorithm, which applies estimated gradients of the expected reward to optimize the policy as below,

$$\nabla_\theta \mathcal{L}_a(\theta|\mathcal{G}) = \mathbb{E}_{p_\theta(\tau|\mathcal{G})}[(c(\tau) - b(\mathcal{G}))\nabla_\theta \log p_\theta(\tau|\mathcal{G})]. \tag{5.1}$$

5.2 Methodology

In this section, we present the multi-task VRP solver with MoEs, and introduce the gating mechanism. Without loss of generality, we train a construction-based neural solver [11, 12] for tackling VRP variants with the five constraints introduced in Section 5.1. The structure of MVMoE is illustrated in Fig. 5.2.

5.2.1 Multi-Task VRP Solver with MoEs

Multi-Task VRP Solver. Given an instance of a specific VRP variant, the *static* features of each node v_i are expressed by $\mathcal{S}_i = \{y_i, \delta_i, e_i, l_i\}$, where y_i, δ_i, e_i, l_i denote the coordinate, demand, start and end time of the time window, respectively. The encoder takes these static node features as inputs, and outputs d-dimensional node embeddings h_i. At the t_{th} decoding step, the decoder takes as input the node embeddings and a context representation, including the embedding of the last

selected node and *dynamic* features $\mathcal{D}_t = \{c_t, t_t, l_t, o_t\}$, where c_t, t_t, l_t, o_t denote the remaining capacity of the vehicle, the current time, the length of the current partial route, and the presence indicator of the open route, respectively. Thereafter, the decoder outputs the probability distribution of nodes, from which a valid node is selected and appended to the partial solution. A complete solution is constructed in an autoregressive manner by iterating the decoding process.

In each training step, we randomly select a VRP variant, and train the neural network to solve associated instances in a batch. In this way, MVMoE is able to learn a unified policy that can tackle different VRP tasks. If only a subset of static or dynamic features are involved in the current selected VRP variant, the other features are padded to the default values (e.g., zeros). For example, given a CVRP instance, the static features of the i_{th} customer node are $\mathcal{S}_i^{(C)} = \{y_i, \delta_i, 0, 0\}$, and the dynamic features at the t_{th} decoding step are $\mathcal{D}_t^{(C)} = \{c_t, 0, l_t, 0\}$. In summary, motivated by the fact that different VRP variants may include some common attributes (e.g., coordinate, demand), we define the static and dynamic features as the union set of attributes that exist in all VRP variants. By training on a few VRP variants with these attributes, the policy network has the potential to solve unseen variants, which are characterized by different combinations of these attributes, i.e., the zero-shot generalization capability [74].

Mixture-of-Experts. An MoE layer consists of 1) m experts $\{E_1, E_2, \ldots, E_m\}$, each of which is a linear layer or FFN with independent trainable parameters, and 2) a gating network G parameterized by W_G, which decides how the inputs are distributed to experts. Given a single input x, $G(x)$ and $E_j(x)$ denote the output of the gating network (i.e., an m-dimensional vector), and the output of the j_{th} expert, respectively. In light of this, the output of an MoE layer is calculated as,

$$\text{MoE}(x) = \sum_{j=1}^{m} G(x)_j E_j(x). \tag{5.2}$$

Intuitively, a sparse vector $G(x)$ only activates a small subset of experts with partial model parameters, and hence saves the computation. Typically, a Top$K$ operator can achieve such sparsity by only keeping the K-largest values while setting others as the negative infinity. In this case, the gating network calculates the output as $G(x) = \text{Softmax}(\text{Top}K(x \cdot W_G))$. Given the fact that larger sparse models do not always lead to better performance [146], it is crucial yet tricky to *design*

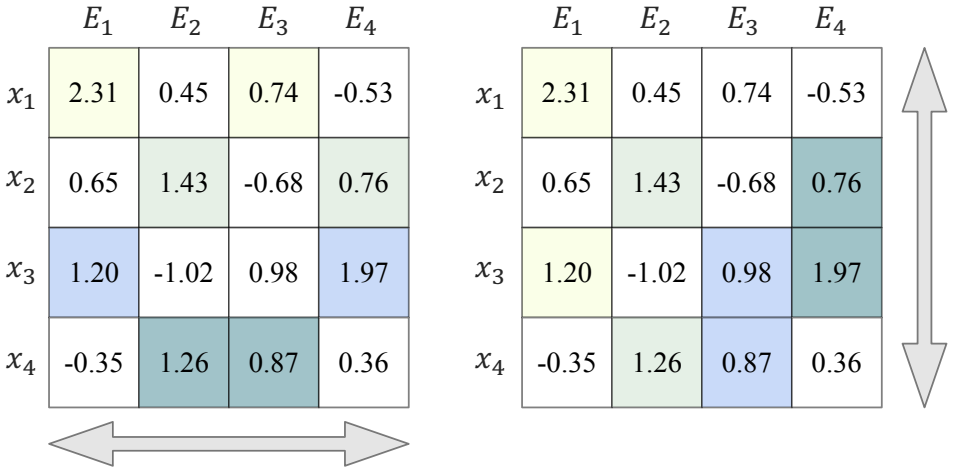


FIGURE 5.3: An illustration of the score matrix and gating algorithm. *Left panel:* Input-choice gating. *Right panel:* Expert-choice gating.

effective and efficient gating mechanisms to endow each expert being sufficiently trained, given enough training data. To this effect, some works have been put forward in language and vision domains, such as designing an auxiliary loss [147] or formulating it as a linear assignment problem [148] in pursuit of the load balancing.

MVMoE. By integrating the above parts, we obtain the multi-task VRP solver with MoEs. The overall model structure is shown in Fig. 5.2, where we employ MoEs in both the encoder and decoder. In specific, we substitute MoEs for the FFN layer in the encoder, and substitute MoEs for the final linear layer of multi-head attention in the decoder. We refer more details of the structure of MVMoE to Appendix C.2. We empirically find our design is effective in generating high-quality solutions, and especially employing MoEs in the decoder tends to exert a greater influence on performance (see Section 5.3). We jointly optimize all trainable parameters Θ, with the objective formulated as follows,

$$\min_{\Theta} \mathcal{L} = \mathcal{L}_a + \alpha \mathcal{L}_b, \tag{5.3}$$

where $\mathcal{L}_a$ denotes the original loss function of the VRP solver (e.g., the REINFORCE loss used to train the policy for solving VRP variants in Eq. (3.1)), $\mathcal{L}_b$ denotes the loss function associated with MoEs (e.g., the auxiliary loss used to ensure load balancing in Eq. (C.16) in Appendix C.2), and α is a hyperparameter to control its strength.

5.2.2 Gating Mechanism

We mainly consider the node-level (or token-level) gating, by which each node is routed independently to experts.[1] In each MoE layer, the extra computation originates from the forward pass of the gating network and the distribution of nodes to the selected experts. While employing MoEs in the decoder can significantly improve the performance, the number of decoding steps T increases as the problem size n scales up. It suggests that compared to the encoder with a fixed number of gating steps N ($\ll T$), applying MoEs in the decoder may substantially increase the computational complexity. In light of this, we propose a hierarchical gating mechanism to make the better use of MoEs in the decoder for gaining a good trade-off between empirical performance and computational complexity. Next, we detail the node-level and hierarchical gating mechanism.

Node-Level Gating. The node-level gating routes inputs at the granularity of nodes. Let d denote the hidden dimension and $W_G \in \mathbb{R}^{d\times m}$ denote trainable parameters of the gating network in MVMoE. Given a batch of inputs $X \in \mathbb{R}^{I\times d}$, where I is the total number of nodes (i.e., batch size B $\times$ problem scale n), each node is routed to the selected experts based on the score matrix $H = (X \cdot W_G) \in \mathbb{R}^{I\times m}$ predicted by the gating network. We illustrate an example of the score matrix in Fig. 5.3, where x_i denotes the i_{th} node, and E_j denotes the j_{th} expert in the node-level gating.

In this chapter, we mainly consider two popular gating algorithms [147, 149]: 1) *Input-choice gating:* Each node selects Top$K$ experts based on H. Typically, K is set to 1 or 2 to retain a reasonable computational complexity. The input-choice gating is illustrated in the left panel of Fig. 5.3, where each node is routed to two experts with the largest scores (i.e., Top2). However, this method cannot guarantee load balancing. An expert may receive much more nodes than the others, resulting in a dominant expert while leaving others underfitting. To address this issue, most works employ an auxiliary loss to equalize quantities of nodes sent to different experts during training. Here we use the importance & load loss [147] as $\mathcal{L}_b$ in Eq. (5.3) to mitigate load imbalance (see Appendix C.2). 2) *Expert-choice gating:* Each expert selects Top$K$ nodes based on H. Typically, K is set to $\frac{I\times\beta}{m}$, where β is the capacity factor reflecting the average number of experts utilized by a node.

[1] In addition, we also investigate another two gating levels, i.e., instance-level and problem-level gating, which are presented in Section 5.3 and Appendix C.2.

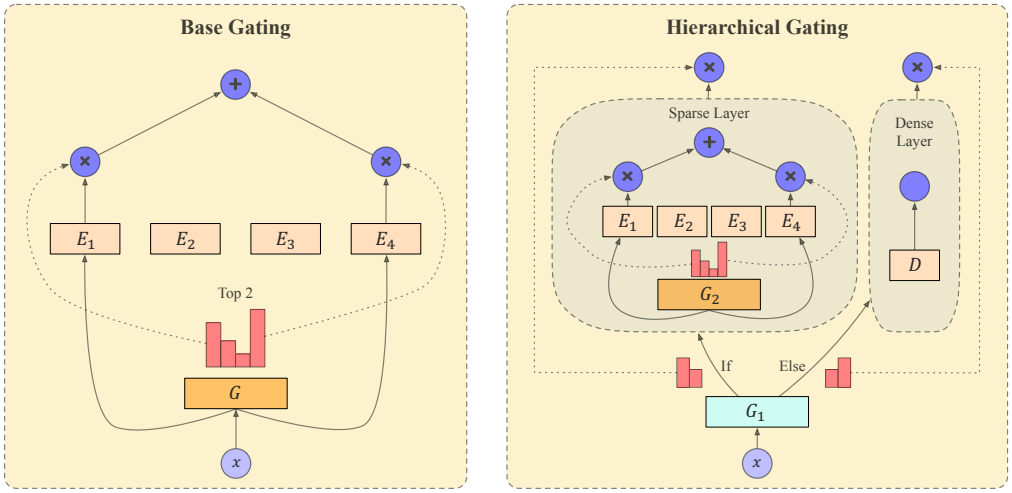


FIGURE 5.4: A base gating (i.e., the input-choice gating with $K = 2$) and its hierarchical gating counterpart. In the latter, the gating network G_1 routes inputs to the sparse layer ($\{G_2, E_1, E_2, E_3, E_4\}$) or the dense layer D.

The expert-choice gating is illustrated in the right panel of Fig. 5.3, where each expert selects two nodes with the largest scores given $\beta = 2$. While this gating algorithm explicitly ensures load balancing, some nodes may not be chosen by any expert. We refer more details of the above gating algorithms to Appendix C.2.

Hierarchical Gating. In the VRP domain, it is computationally expensive to employ MoEs in each decoding step, since 1) the number of decoding steps T increases as the problem size n rises; 2) the problem-specific feasibility constraints must be satisfied during decoding. To tackle the challenges, we propose to employ MoEs only in *partial* decoding steps. Accordingly, we present a hierarchical gating, which learns to effectively and efficiently utilize MoEs during decoding.

We illustrate the proposed hierarchical gating in Fig. 5.4. An MoE layer with the hierarchical gating includes two gating networks $\{G_1, G_2\}$, m experts $\{E_1, E_2, \ldots, E_m\}$, and a dense layer D (e.g., a linear layer). Given a batch of inputs $X \in \mathbb{R}^{I \times d}$, the hierarchical gating routes them in two stages. In the first stage, G_1 decides to distribute inputs X to either the sparse or dense layer according to the problem-level representation X_1. In specific, we obtain X_1 by applying the mean pooling along the first dimension of X, and process it to obtain the score matrix $H_1 = (X_1 \cdot W_{G_1}) \in \mathbb{R}^{1 \times 2}$. Then, we route the batch of inputs X to the sparse or dense layer by sampling from the probability distribution $G_1(X) = \text{Softmax}(H_1)$. Here we employ the problem-level gating in G_1 for the generality and efficiency of the hierarchical gating. In the second stage, if X is routed to the sparse layer, the

gating network G_2 is activated to route nodes to experts on the node-level by using aforementioned gating algorithms (e.g., the input-choice gating). Otherwise, X is routed to the dense layer D and transformed into $D(X) \in \mathbb{R}^{I \times d}$. In summary, the hierarchical gating learns to output $G_1(X)_0 \sum_{j=1}^{m} G_2(X)_j E_j(X)$ or $G_1(X)_1 D(X)$ based on both problem-level and node-level representations.

Overall, the hierarchical gating improves the computational efficiency with a minor loss on the empirical performance. To balance the efficiency and performance of MVMoE, we use the base gating in the encoder and its hierarchical gating counterpart in the decoder. Note that the hierarchical gating is applicable to different gating algorithms, such as the input-choice gating [147] and expert-choice gating [149]. We also explore a more advanced gating algorithm [150] for reducing the number of routed nodes and thus the computational complexity. But its empirical performance is unsatisfactory in the VRP domain.

5.3 Experiments

In this section, we empirically verify the superiority of the proposed MVMoE, and provide insights into the application of MoEs to solve VRPs. We consider 16 VRP variants with five constraints. All experiments are conducted on a machine with NVIDIA Ampere A100-80GB GPU cards and AMD EPYC 7513 CPU at 2.6GHz.

Baselines. *Traditional solvers:* We employ HGS [6] to solve CVRP and VRPTW instances with default hyperparameters (i.e., the maximum number of iterations without improvement is 20000). We run LKH3 [5] to solve CVRP, OVRP, VRPL and VRPTW instances with 10000 trails and 1 run. OR-Tools [151] is an open source solver for complex optimization problems. It is more versatile than LKH and HGS, and can solve all 16 VRP variants considered in this chapter. We use the parallel cheapest insertion as the first solution strategy, and use the guided local search as the local search strategy in OR-Tools. For $n = 50/100$, we set the search time limit as 20s/40s to solve an instance, and also provide its results given 200s/400s (i.e., OR-Tools (x10)). For all traditional solvers, we use them to solve 32 instances in parallel on 32 CPU cores following Kool et al. [11].

Neural solvers: We compare our method to POMO [12] and POMO-MTL [74]. While POMO is trained on each single VRP, POMO-MTL is trained on multiple

VRPs by multi-task learning. Note that POMO-MTL is the dense model counterpart of MVMoE, which is structured by dense layers (e.g., FFNs) rather than sparse MoEs. In specific, POMO-MTL and MVMoE/4E possess 1.25M and 3.68M parameters, but they activate a similar number of parameters for each single input.

Training. We follow most setups in Kwon et al. [12]. 1) *For all neural solvers:* Adam optimizer is used with the learning rate of $1e-4$, the weight decay of $1e-6$, and the batch size of 128. The model is trained for 5000 epochs, with each containing 20000 training instances (i.e., 100M training instances in total). The learning rate is decayed by 10 for the last 10% training instances. We consider two problem scales $n \in \{50, 100\}$ during training, according to Liu et al. [74]. 2) *For multi-task solvers:* The training problem set includes CVRP, OVRP, VRPB, VRPL, VRPTW, and OVRPTW (see Appendix C.3 for further discussions). In each batch of training, we randomly sample a problem from the set and generate its instances. Please refer to Appendix C.1 for details of the generation procedure. 3) *For our method:* We employ $m = 4$ experts with $K = \beta = 2$ in each MoE layer, and set the the weight α of the auxiliary loss $\mathcal{L}_b$ as 0.01. The default gating mechanism of MVMOE/4E is the node-level input-choice gating in both the encoder and decoder layers. MVMoE/4E-L is a computationally light version that replaces the input-choice gating with its hierarchical gating counterpart in the decoder.

Inference. For all neural solvers, we use greedy rollout with x8 instance augmentation following Kwon et al. [12]. We report the average results (i.e., objective values and gaps) over the test dataset containing 1K instances, and the total time to solve the entire test dataset. The gaps are computed with respect to the results of the best-performing traditional VRP solvers (i.e., * in Tables 5.1 and 5.2).

5.3.1 Empirical Results

Performance on Trained VRPs. We evaluate all methods on 6 trained VRPs and gather all results in Table 3.1. The single-task neural solver (i.e., POMO) achieves better performance than multi-task neural solvers on each single problem, since it is restructured and retrained on each VRP independently. However, its average performance over all trained VRPs is quite inferior as shown in Table C.2 in Appendix C.3, since each trained POMO is overfitted to a specific VRP. For example, the average performance of POMO solely trained on CVRP is 16.815%, while

TABLE 5.1: Performance on 1K test instances of trained VRPs.

	Method	$n = 50$			$n = 100$				Method	$n = 50$			$n = 100$		
		Obj.	Gap	Time	Obj.	Gap	Time			Obj.	Gap	Time	Obj.	Gap	Time
CVRP	HGS	10.334	*	4.6m	15.504	*	9.1m	VRPTW	HGS	14.509	*	8.4m	24.339	*	19.6m
	LKH3	10.346	0.115%	9.9m	15.590	0.556%	18.0m		LKH3	14.607	0.664%	5.5m	24.721	1.584%	7.8m
	OR-Tools	10.540	1.962%	10.4m	16.381	5.652%	20.8m		OR-Tools	14.915	2.694%	10.4m	25.894	6.297%	20.8m
	OR-Tools (x10)	10.418	0.788%	1.7h	15.935	2.751%	3.5h		OR-Tools (x10)	14.665	1.011%	1.7h	25.212	3.482%	3.5h
	POMO	10.418	0.806%	3s	15.734	1.488%	9s		POMO	14.940	2.990%	3s	25.367	4.307%	11s
	POMO-MTL	10.437	0.987%	3s	15.790	1.846%	9s		POMO-MTL	15.032	3.637%	3s	25.610	5.313%	11s
	MVMoE/4E	**10.428**	**0.896%**	4s	**15.760**	**1.653%**	11s		MVMoE/4E	**14.999**	**3.410%**	4s	**25.512**	**4.903%**	12s
	MVMoE/4E-L	10.434	0.955%	4s	15.771	1.728%	10s		MVMoE/4E-L	15.013	3.500%	3s	25.519	4.927%	11s
OVRP	LKH3	6.511	0.198%	4.5m	9.828	*	5.3m	VRPL	LKH3	10.571	0.790%	7.8m	15.771	*	16.0m
	OR-Tools	6.531	0.495%	10.4m	10.010	1.806%	20.8m		OR-Tools	10.677	1.746%	10.4m	16.496	4.587%	20.8m
	OR-Tools (x10)	6.498	*	1.7h	9.842	0.122%	3.5h		OR-Tools (x10)	10.495	*	1.7h	16.004	1.444%	3.5h
	POMO	6.609	1.685%	2s	10.044	2.192%	8s		POMO	10.491	-0.008%	2s	15.785	0.093%	9s
	POMO-MTL	6.671	2.634%	2s	10.169	3.458%	8s		POMO-MTL	10.513	0.201%	2s	15.846	0.479%	9s
	MVMoE/4E	**6.655**	**2.402%**	3s	**10.138**	**3.136%**	10s		MVMoE/4E	**10.501**	**0.092%**	3s	**15.812**	**0.261%**	10s
	MVMoE/4E-L	6.665	2.548%	3s	10.145	3.214%	9s		MVMoE/4E-L	10.506	0.131%	3s	15.821	0.323%	10s
VRPB	OR-Tools	8.127	0.989%	10.4m	12.185	2.594%	20.8m	OVRPTW	OR-Tools	8.737	0.592%	10.4m	14.635	1.756%	20.8m
	OR-Tools (x10)	8.046	*	1.7h	11.878	*	3.5h		OR-Tools (x10)	8.683	*	1.7h	14.380	*	3.5h
	POMO	8.149	1.276%	2s	11.993	0.995%	7s		POMO	8.891	2.377%	3s	14.728	2.467%	10s
	POMO-MTL	8.182	1.684%	2s	12.072	1.674%	7s		POMO-MTL	8.987	3.470%	3s	15.008	4.411%	10s
	MVMoE/4E	**8.170**	**1.540%**	3s	**12.027**	**1.285%**	9s		MVMoE/4E	**8.964**	**3.210%**	4s	**14.927**	**3.852%**	11s
	MVMoE/4E-L	8.176	1.605%	3s	12.036	1.368%	8s		MVMoE/4E-L	8.974	3.322%	4s	14.940	3.941%	10s

POMO-MTL and MVMoE/4E achieve 2.102% and 1.925%, respectively. Notably, our neural solvers consistently outperform POMO-MTL. MVMoE/4E performs slightly better than MVMoE/4E-L at the expense of more computation. Despite that, MVMoE/4E-L exhibits stronger out-of-distribution generalization capability than MVMoE/4E (see Tables C.4 and C.5 in Appendix C.3).

Generalization on Unseen VRPs. We evaluate multi-task solvers on 10 unseen VRP variants. 1) *Zero-shot generalization:* We directly test the trained solvers on unseen VRPs. The results in Table 5.2 reveal that the proposed MVMoE significantly outperforms POMO-MTL across all VRP variants. 2) *Few-shot generalization:* We also consider the few-shot setting on $n = 50$, where a trained solver is fine-tuned on the target VRP using 10K instances (0.01% of total training instances) in each epoch. Without loss of generality, we conduct experiments on VRPBLTW and OVRPBLTW following the training setups. The results in Fig. 5.5 showcase MVMoE generalizes more favorably than POMO-MTL.

TABLE 5.2: Zero-shot generalization on 1K test instances of unseen VRPs.

	Method	$n = 50$			$n = 100$			Method	$n = 50$			$n = 100$		
		Obj.	Gap	Time	Obj.	Gap	Time		Obj.	Gap	Time	Obj.	Gap	Time
OVRPB	OR-Tools	5.764	0.332%	10.4m	8.522	1.852%	20.8m	OVRPL	6.522	0.480%	10.4m	9.966	1.783%	20.8m
	OR-Tools (x10)	5.745	*	1.7h	8.365	*	3.5h		6.490	*	1.7h	9.790	*	3.5h
	POMO-MTL	6.116	6.430%	2s	8.979	7.335%	8s		6.668	2.734%	2s	10.126	3.441%	9s
	MVMoE/4E	**6.092**	**5.999%**	3s	**8.959**	**7.088%**	9s		MVMoE/4E	**6.650**	**2.454%**	3s	**10.097**	**3.148%**	10s
	MVMoE/4E-L	6.122	6.522%	3s	8.972	7.243%	9s		MVMoE/4E-L	6.659	2.597%	3s	10.106	3.244%	9s
VRPBL	OR-Tools	8.131	1.254%	10.4m	12.095	2.586%	20.8m	VRPBTW	OR-Tools	15.053	1.857%	10.4m	26.217	2.858%	20.8m
	OR-Tools (x10)	8.029	*	1.7h	11.790	*	3.5h		OR-Tools (x10)	14.771	*	1.7h	25.496	*	3.5h
	POMO-MTL	8.188	1.971%	2s	11.998	1.793%	8s		POMO-MTL	16.055	8.841%	3s	27.319	7.413%	10s
	MVMoE/4E	**8.172**	**1.776%**	3s	**11.945**	**1.346%**	9s		MVMoE/4E	**16.022**	**8.600%**	4s	**27.236**	**7.078%**	11s
	MVMoE/4E-L	8.180	1.872%	3s	11.960	1.473%	9s		MVMoE/4E-L	16.041	8.745%	4s	27.265	7.190%	10s
VRPLTW	OR-Tools	14.815	1.432%	10.4m	25.823	2.534%	20.8m	OVRPBL	OR-Tools	5.771	0.549%	10.4m	8.555	2.459%	20.8m
	OR-Tools (x10)	14.598	*	1.7h	25.195	*	3.5h		OR-Tools (x10)	5.739	*	1.7h	8.348	*	3.5h
	POMO-MTL	14.961	2.586%	3s	25.619	1.920%	12s		POMO-MTL	6.104	6.306%	2s	8.961	7.343%	8s
	MVMoE/4E	**14.937**	**2.421%**	4s	**25.514**	**1.471%**	13s		MVMoE/4E	**6.076**	**5.843%**	3s	**8.942**	**7.115%**	9s
	MVMoE/4E-L	14.953	2.535%	4s	25.529	1.545%	12s		MVMoE/4E-L	6.104	6.310%	3s	8.957	7.300%	9s
OVRPBTW	OR-Tools	8.758	0.927%	10.4m	14.713	2.268%	20.8m	OVRPLTW	OR-Tools	8.728	0.656%	10.4m	14.535	1.779%	20.8m
	OR-Tools (x10)	8.675	*	1.7h	14.384	*	3.5h		OR-Tools (x10)	8.669	*	1.7h	14.279	*	3.5h
	POMO-MTL	9.514	9.628%	3s	15.879	10.453%	10s		POMO-MTL	8.987	3.633%	3s	14.896	4.374%	11s
	MVMoE/4E	**9.486**	**9.308%**	4s	**15.808**	**9.948%**	11s		MVMoE/4E	**8.966**	**3.396%**	4s	**14.828**	**3.903%**	12s
	MVMoE/4E-L	9.515	9.630%	3s	15.841	10.188%	10s		MVMoE/4E-L	8.974	3.488%	4s	14.839	3.971%	10s
VRPBLTW	OR-Tools	14.890	1.402%	10.4m	25.979	2.518%	20.8m	OVRPBLTW	OR-Tools	8.729	0.624%	10.4m	14.496	1.724%	20.8m
	OR-Tools (x10)	14.677	*	1.7h	25.342	*	3.5h		OR-Tools (x10)	8.673	*	1.7h	14.250	*	3.5h
	POMO-MTL	15.980	9.035%	3s	27.247	7.746%	11s		POMO-MTL	9.532	9.851%	3s	15.738	10.498%	10s
	MVMoE/4E	**15.945**	**8.775%**	4s	**27.142**	**7.332%**	12s		MVMoE/4E	**9.503**	**9.516%**	4s	**15.671**	**10.009%**	11s
	MVMoE/4E-L	15.963	8.915%	4s	27.177	7.473%	11s		MVMoE/4E-L	9.518	9.682%	4s	15.706	10.263%	10s

5.3.2 Ablation on MoEs

Here we explore the effect of different MoE settings on the zero-shot generalization of neural solvers, and provide insights on how to effectively apply MoEs to solve VRPs. Due to the fast convergence, we reduce the number of epochs to 2500 on VRPs of the size $n = 50$, while leaving other setups unchanged. We set MVMoE/4E as the default baseline, and ablate on different components of MoEs below.

Position of MoEs. We consider three positions to apply MoEs in neural solvers: 1) *Raw feature processing (Raw):* The linear layer, which projects raw features into initial embeddings, are replaced by MoEs. 2) *Encoder (Enc):* The FFN in an encoder layer is replaced by MoEs. Typically, MoEs are widely used in *every-two* or *last-two* layers (i.e., every or last two layers with even indices $\ell \in [0, N-1]$) [152]. Besides, we further attempt to use MoEs in all encoder layers. 3) *Decoder (Dec):* The final linear layer of the multi-head attention is replaced by MoEs in the decoder. We show the average performance over 10 unseen VRPs in Fig. 5.6(a).

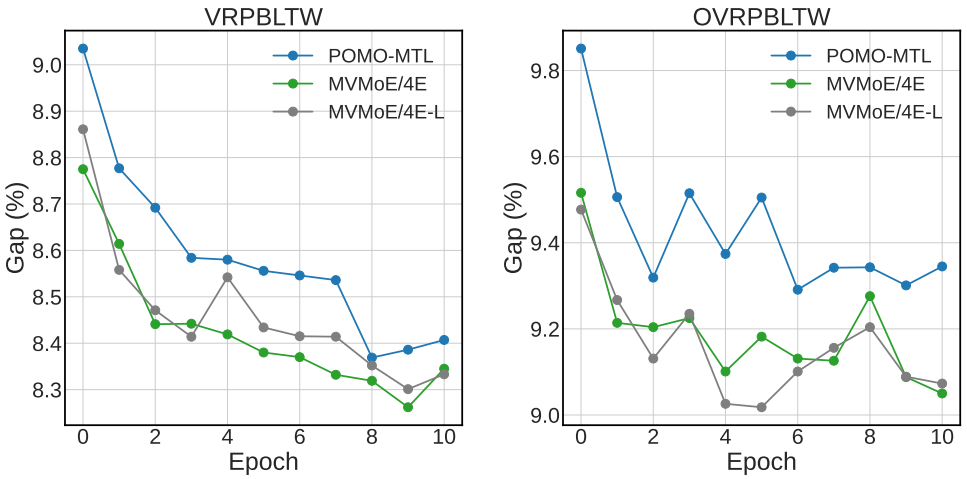


FIGURE 5.5: Few-shot generalization on unseen VRPs.

The results reveal that applying MoEs at the shallow layer (e.g., Raw) may worsen the model performance, while using MoEs in all encoder layers (Enc_All) or decoder (Dec) can benefit the zero-shot generalization. Therefore, in this chater, we employ MoEs in both encoder and decoder to pursue a strong unified model architecture to solve various VRPs.

Number of Experts. We increase the number of experts in each MoE layer to 8 and 16, and compare the derived MVMoE/8E/16E models to MVMoE/4E. We first train all models using the same number (50M) of instances. After that, we also train MVMoE/8E/16E with more data and computation to explore potential better results, based on the scaling laws [141]. In specific, we provide MVMoE/8E/16E with more data by using larger batch sizes, which linearly scale up against the number of experts (i.e., MVMoE/4E/8E/16E are trained on 50M/100M/200M instances with batch sizes 128/256/512, respectively). The results in Fig. 5.6(b) show that increasing the number of experts with more training data further unleashes the power of MVMoE, indicating the efficacy of MoEs in solving VRPs.

Gating Mechanism. We investigate the effect of different gating levels and algorithms, including three levels (i.e., node-level, instance-level and problem-level) and three algorithms (i.e., input-choice, expert-choice and random gatings), with their details presented in Appendix C.2. As shown in Fig. 5.6(c), the node-level input-choice gating performs the best, while the node-level expert-choice gating performs the worst. Interestingly, we observe that the expert-choice gating in the decoder makes MVMoE hard to be optimized. It may suggest that each gating algorithm could have its most suitable position to serve MoEs. However, after

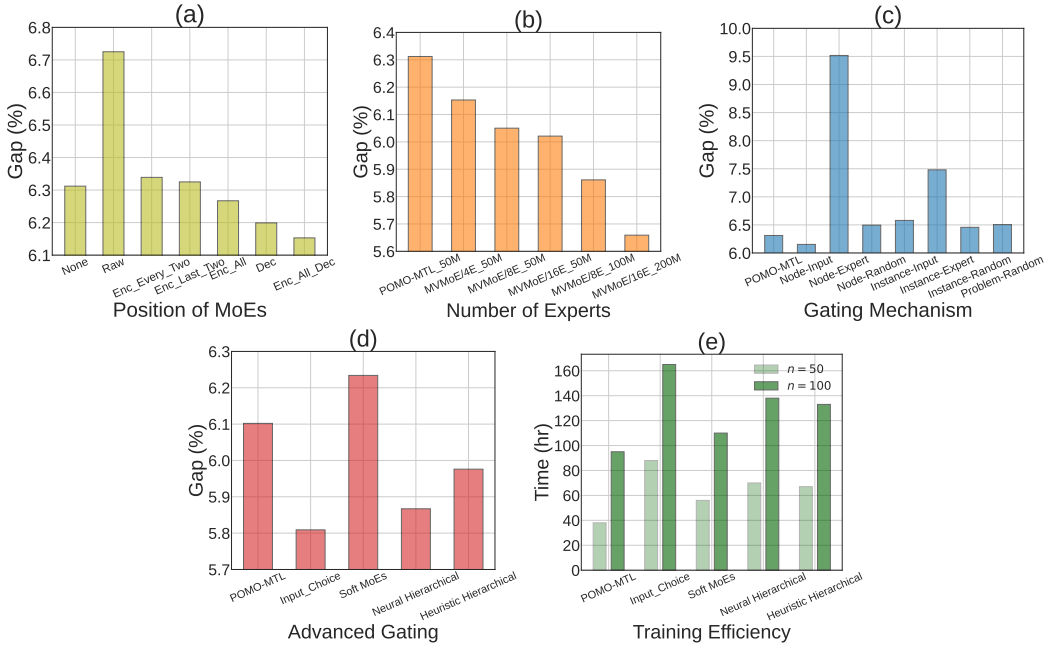


FIGURE 5.6: Ablation studies on MoE configurations.

an attempt to tune this configuration (i.e., by using MoEs only in the encoder), its performance is still inferior to the baseline, with an average gap of 7.190% on unseen VRPs.

5.3.3 Further Analyses

We further provide experiments and discussions on more advanced gating algorithms, training efficiency, benchmark performance, and scalability. We refer readers to more empirical results (e.g., sensitivity analyses) in Appendix C.3.

Advanced Gating. Besides the input-choice and expert-choice gating algorithms evaluated above, we further consider soft MoEs [150], which is a recent advanced gating algorithm. Specifically, it performs an implicit soft assignment by distributing K slots (i.e., convex combinations of all inputs) to each expert, rather than a hard assignment between inputs and experts as done by the conventional sparse and discrete gating networks. Since only K (e.g., 1 or 2) slots are distributed to each expert, it can save much computation. We train MVMoE on $n = 50$ by using node-level soft MoEs in the decoder, following training setups. We also show the result of employing heuristic (random) hierarchical gating in the decoder. However, their results are unsatisfactory as shown in Fig. 5.6(d).

Training Efficiency. Fig. 5.6(e) shows the training time of employing each gating algorithm in the decoder, combining with their results reported in Fig. 5.6(d), demonstrating the efficacy of the proposed hierarchical gating in reducing the training overhead with only minor losses in performance.

Benchmark Performance. We further evaluate the OOD generalization performance of all neural solvers on CVRPLIB benchmark instances. Detailed results can be found in Tables C.4 and C.5 in Appendix C.3. Surprisingly, we observe that MVMoE/4E performs poorly on large-scale instances (e.g., $n > 500$). It may be caused by the generalization issue of sparse MoEs when transferring to new distributions or domains, which is still an open question in the MoE literature [153]. In contrast, MVMoE/4E-L mostly outperforms MVMoE/4E, demonstrating more favourable potential of the hierarchical gating in promoting the OOD generalization capability. It is worth noting that all neural solvers are only trained on the simple uniformly distributed instances with the size $n = 100$. Embracing more varied problem sizes (cross-size) and attribute distributions (cross-distribution) into the multi-task training (cross-problem) may further consolidate their performance.

Scalability. Given that SL-based approaches appear to be more scalable than RL-based approaches in the current literature, we try to build upon a more scalable method, i.e., LEHD [26]. Concretely, we train a dense model LEHD and a light sparse model with 4 experts LEHD/4E-L on CVRP. The training setups are kept the same as Luo et al. [26], except that we train all models for only 20 epochs for the training efficiency. We use the hierarchical MoE in each decoder layer of LEHD/4E-L. The results are shown in Table C.5, which demonstrates the potential of MoE as a general idea that can further benefit recent scalable methods. Moreover, during the solution construction process, recent works [25, 60] typically constrain the search space within a neighborhood of the currently selected node, which is shown to be effective in handling large-scale instances. Integrating MVMoE with these simple yet effective techniques may further improve large-scale performance.

5.4 Chapter Summary

This chapter targets a more generic and powerful neural solver for VRPs, pushing the boundaries of neural VRP solvers in learning generalizable representations

across diverse constraints. Methodologically, we propose a multi-task vehicle routing solver with MoEs (MVMoE), which can solve a range of VRPs concurrently, even in a zero-shot manner. We provide valuable insights on how to apply MoEs in neural VRP solvers, and propose an effective and efficient hierarchical gating mechanism. Empirically, MVMoE demonstrates strong generalization capability on zero-shot, few-shot settings, and real-world benchmark.

Despite this chapter presents the first attempt towards a large VRP model, the scale of parameters is still far less than LLMs. We leave 1) the development of *scalable* MoE-based models in solving *large-scale VRPs*, 2) the venture of *generic* representations for different problems, 3) the exploration of *interpretability* of gating mechanisms, and 4) the investigation of *scaling laws* in MoEs to the future work. We hope this work contributes to the NCO community by advancing the development of foundation models for combinatorial optimization.

Chapter 6

Conclusions and Future Works

6.1 Conclusions

This thesis advances the frontier of neural VRP solvers by significantly improving their ability to learn generalizable representations across diverse contexts, including varying problem scales, data distributions, and constraints. To achieve this, we introduced three novel approaches aimed at enhancing cross-size, cross-distribution, and cross-problem generalization capabilities. These advancements systematically enhance the robustness and efficacy of neural VRP solvers, particularly when addressing out-of-distribution scenarios that commonly occur in real-world applications. Below, we provide a summary of these approaches, and conclude this thesis by emphasizing the key findings and insights garnered throughout the study.

In Chapter 3, we address cross-distribution generalization through the lens of adversarial robustness by introducing CNF, an ensemble-based adversarial training framework. We demonstrate that relaxing the conventional imperceptibility constraint on the attack budget, which is typically imposed in the vision domain, yields strong yet meaningful attacks on neural VRP solvers. In the inner maximization phase, CNF employs this attack as its backbone while incorporating a global attack mechanism to enhance data diversity. During the outer minimization phase, a neural router is trained to adaptively allocate training instances to each model, thereby fully leveraging the overall model capacity to achieve a more robust ensemble effect. Extensive experiments confirm the effectiveness and versatility of CNF

in defending against a variety of attacks across different neural VRP solvers, and highlight its impressive cross-distribution generalization on benchmark instances.

In Chapter 4, we further extend the capabilities of neural VRP solvers by tackling both cross-size and cross-distribution generalization (i.e., omni-generalization). We introduce Omni-VRP, a generic meta-learning framework that trains an initial meta-model capable of rapidly adapting to new tasks using only limited data during inference. A hierarchical task scheduler adaptively selects batches of tasks, each characterized by a unique problem scale and data distribution, based on the training dynamics. Additionally, we propose an efficient approximation method to reduce the training overhead associated with calculating second-order derivatives. In light of the No Free Lunch Theorems in Machine Learning, it is unrealistic to expect a one-size-fits-all model to excel on every task. Nevertheless, our meta-model effectively handles unseen tasks through efficient adaptation. Empirical results demonstrate that Omni-VRP achieves strong zero-shot and few-shot generalization performance on both synthetic and benchmark instances of TSP and CVRP, marking an advancement in addressing complex and dynamic real-world challenges.

In Chapter 5, we further push the boundaries of neural VRP solvers towards a multi-task setting, encouraging them to learn generalizable representations across problem variants with diverse constraints. We proposed MVMoE, a unified neural solver capable of addressing 16 VRP variants simultaneously. MVMoE is trained on a diverse set of VRP variants and exhibits strong zero-shot generalization on unseen VRPs. To enhance model capacity efficiently, we leverage a mixture-of-experts (MoE) architecture with a node-level gating mechanism. Our experiments demonstrate that incorporating an MoE layer within the decoder significantly improves solution quality, albeit at the cost of increased computational overhead. To strike a balance between empirical performance and computational efficiency, we introduce a hierarchical gating mechanism that adaptively allocates computation during decision-making. This hierarchical gating approach preserves in-distribution performance while offering substantially enhanced out-of-distribution generalization and computational efficiency compared to the base gating mechanism. Extensive experiments demonstrate that MVMoE significantly outperforms baselines in zero-shot generalization on 10 unseen VRP variants, and achieves robust results in both few-shot settings and real-world benchmark instances.

In summary, this thesis underscores the critical importance of developing generalizable neural VRP solvers. Our contributions through CNF, Omni-VRP, and MVMoE significantly enhance the ability of neural VRP solvers to generalize across varying contexts. CNF addresses generalization across diverse data distributions at the instance level. Omni-VRP extends this capability by achieving omni-generalization across both diverse problem scales and data distributions at the instance level. At a higher level, MVMoE targets generalization across diverse constraints at the problem level. The insights and methodologies proposed here lay a strong foundation for future research, opening promising avenues for the development of more resilient, interpretable, and practically applicable neural combinatorial optimization frameworks.

6.2 Future Works

Despite the above advancements, several challenges remain to be addressed to tackle real-world problems effectively. In this section, we outline several key directions for future research as follows.

Address Broad and Practical VRP Variants. A primary avenue for future work is the extension to a wider array of practical VRP variants. Although MVMoE can address 16 classical VRP variants, there remains a significant gap between neural solvers and traditional solvers (e.g., OR-Tools, LKH3, and HGS) in terms of both generality and empirical performance. Traditional solvers often require extensive trial-and-error and domain knowledge to adapt to diverse problem settings, whereas neural solvers offer greater flexibility in adaptation. Future research should focus on adapting neural solvers to tackle more challenging settings, such as dynamic or stochastic VRPs where uncertainties exist (e.g., in customer demands or travel times), and complex constrained VRPs that cannot be easily handled via feasibility masking. These directions offer exciting opportunities to enhance the applicability of neural VRP solvers in real-world scenarios.

Develop Foundation Models for VRPs. The current generation of multi-task solvers, exemplified by MVMoE, lays the groundwork for developing foundation models (FMs) for VRPs, which is a promising research direction that demands careful consideration of both generality and scalability.

Generality refers to a models ability to learn representations that effectively generalize across multiple dimensions within various perspectives in VRPs. Building on the unique characteristics of combinatorial optimization, we can categorize these dimensions into six, grouped under three perspectives, as outlined below: 1) *Instance perspective:* Neural solvers should capture common representations transferable across a class of instances, encompassing variations in problem *scales*, underlying *distributions*, and input *modalities*. 2) *Problem perspective:* VRPs primarily differ in their feasibility *constraints* and optimization *objectives*, reflecting unique structural characteristics and solution requirements that challenge the generalization of models. 3) *Algorithm perspective:* Algorithmic *paradigms* (e.g., autoregressive node selection) represent high-level strategies that neural solvers intrinsically learn and apply to solve VRPs. FMs for VRPs should account for variations across these dimensions to effectively learn generalizable representations.

Scaling is equally critical and should be addressed along multiple facets. This involves expanding the scope of training and inference along several key aspects: 1) *Data scale:* Leveraging large-scale, diverse datasets that capture extensive variations across dimensions (e.g., within both the instance and problem perspectives) to effectively reflect the complexity and diversity of real-world scenarios. 2) *Model scale:* Developing scalable model architectures with sufficient model capacity to learn generalizable representations, while maintaining efficiency for practical deployment. 3) *Inference scale:* Beyond scaling during training, scaling inference-time computation (e.g., through efficient post-hoc search) can significantly enhance the model's ability to tackle complex optimization tasks. Addressing these aspects is essential for developing models that can reach the capabilities of FMs.

Regarding generality, while this thesis has examined generalization in diverse contexts, including problem scales, underlying distributions, and constraints, additional dimensions remain to be explored to fully capture the complexity and diversity of real-world applications. Regarding scaling, although MVMoE advances model scaling to some extent, its parameter count (around 0.01B) is modest compared to the 10100B parameters or more used in LLMs. Bridging these gaps in future research could pave the way for the first generation of foundation models for VRPs that are both highly scalable and broadly generalizable, ultimately catalyzing transformative advances in solving real-world problems.

Enhance Scalability. Scalability remains a fundamental challenge in combinatorial optimization, and improving the capability of generalizable solvers to handle larger-scale problem instances is therefore a crucial direction for future research. Although our experiments demonstrated the feasibility of effectively solving benchmark instances with approximately 1K nodes, this scale is relatively modest compared to real-world applications, which frequently require handling instances of 10K nodes or more. To bridge this gap, promising avenues for future research include: 1) leveraging divide-and-conquer and decomposition techniques to break down large-scale problems into smaller, more manageable subproblems, as mentioned in Section 2.3; 2) integrating more efficient deep learning methods, such as linear attention mechanisms, or sparse attention to reduce computational complexity; 3) strategically pruning or constraining the search space during decision-making processes to significantly enhance computational efficiency. Addressing these scalability challenges will be critical for ensuring neural VRP solvers can meet the demanding requirements of large-scale, real-world VRP applications.

Enhance Interpretability. Beyond generalization, interpretability is another key consideration for real-world deployment. 0Classical heuristics often benefit from well-understood design principles and transparent decision-making processes, which contribute to their interpretability and trust in critical applications. Despite their promising capabilities, neural VRP solvers typically operate as black-box models, offering limited insights into their internal reasoning and decision-making processes. Enhancing their interpretability represents another significant research direction. Improved interpretability can foster trust and facilitate broader adoption among industry practitioners by enabling a clearer understanding of how neural solvers generate solutions and identify key decision factors. To achieve this, future work can explore: 1) designing interpretable model architectures with modular structures that transparently reveal key decision criteria; 2) developing explainability techniques specifically tailored to VRP contexts, including sensitivity analysis, feature importance assessments, and counterfactual reasoning methods; and 3) integrating human-in-the-loop approaches, where domain experts validate, refine, or provide feedback on the models decision-making process, thereby enhancing transparency and practical usability. Addressing interpretability challenges is critical for establishing neural VRP solvers as reliable, trustworthy tools for solving complex real-world optimization problems.

Appendix A

Appendix for Chapter 3

A.1 Other Attack Methods

Node Insertion. An efficient and sound perturbation model is proposed by [69], which, given the optimal solution y to the clean instance x sampled from the data distribution $\mathcal{D}$, guarantees to directly derive the optimal solution $\tilde{y}$ to the adversarial instance $\tilde{x}$ without running a solver. The attack is applied to the GCN [13], which is a non-autoregressive construction method for TSP. It learns the probability of each edge occurring in the optimal solution (i.e., heat-map) with supervised learning. Following the AT framework, the objective function can be written as follows:

$$\min_{\theta} \mathbb{E}_{(x,y)\sim\mathcal{D}} \max_{\tilde{x}} \ell(f_\theta(\tilde{x}), \tilde{y}), \text{ with } \tilde{x} \in G(x,y) \wedge \tilde{y} = h(\tilde{x}, x, y), \tag{A.1}$$

where ℓ is the cross-entropy loss; G is the perturbation model that describes the possible perturbed instances $\tilde{x}$ around the clean instance x; and h is used to derive the optimal solution $\tilde{y}$ based on $(\tilde{x}, x, y)$ without running a solver. In the inner maximization, the adversarial instance $\tilde{x}$ is generated by inserting several new nodes into x (below we take inserting one new node as an example), which adheres to below proposition borrowed from [69]:

Proposition 1. *Let $Z \notin \mathcal{V}$ be an additional node to be inserted, $w(\mathcal{E})$ is an edge weight, and P, Q are any two neighbouring nodes in the original optimal solution y. Then, the new optimal solution $\tilde{y}$ (including Z) is obtained from y through inserting*

Z between P and Q if $\nexists(A, B) \in \mathcal{E} \setminus \{(P, Q)\}$ *with* $A \neq B$ *s.t.* $w(A, Z) + w(B, Z) - w(A, B) \leq w(P, Z) + w(Q, Z) - w(P, Q)$.

Based on the proposition, the optimization of inner maximization involves: 1) obtaining the coordinates of additional node Z by gradient ascending (e.g., maximizing l such that the model prediction is maximally different from the derived optimal solution $\tilde{y}$); 2) penalizing if Z violates the constraint in Proposition 1. Unfortunately, the constraint is non-convex and hard to find a relaxation. Instead of optimizing the Lagrangian (which requires extra computation for evaluating f_θ), the vanilla gradient descent is leveraged with the constraint as the objective:

$$Z \leftarrow Z - \eta \nabla_Z [w(P, Z) + w(Q, Z) - w(P, Q) - (\min_{A,B} w(A, Z) + w(B, Z) - w(A, B))], \tag{A.2}$$

where η is the step size. After we find Z satisfying the constraint, the adversarial instance $\tilde{x}$ and the corresponding optimal solution $\tilde{y}$ can be constructed directly. Finally, the outer minimization takes $(\tilde{x}, \tilde{y})$ as inputs to train the robust neural solvers. This attack method can be easily adopted by our proposed framework, where θ is replaced by the best model θ_b when maximizing the cross-entropy loss (i.e., $\max_{\tilde{x}} \ell(f_{\theta_b}(\tilde{x}), \tilde{y})$) in the inner maximization optimization. However, due to the efficiency and soundness of the perturbation model, it does not suffer from the undesirable trade-off following the vanilla AT [69]. Therefore, we mainly focus on other attack methods [66, 70].

No-Worse Theoretical Optimum. The attack method specialized for graph-based COPs is proposed by [70]. It resorts to the black-box adversarial attack method by training a reinforcement learning based attacker, and thus can be used to generate adversarial instances for both differentiable (e.g., learning-based) and non-differentiable (e.g., heuristic or exact) solvers. In this paper, we only consider the learning-based neural solvers for VRPs. Specifically, it generates the adversarial instance $\tilde{x}$ by modifying the clean instance x (e.g., lowering the cost of a partial instance) under the no worse optimum condition, which requires $c(\tilde{y}) \leq c(y)$ if we are solving a minimization optimization problem. The attack is successful (w.r.t. the neural solver θ) if the output solution to $\tilde{x}$ is worse than the one to x (i.e., $c(\tilde{\tau}|\tilde{x}; \theta) > c(\tau|x; \theta) \geq c(y) \geq c(\tilde{y})$). The training of the attacker is hence

formulated as follows:

$$\begin{aligned} \max_{\tilde{x}} . \quad & c(\tilde{\tau}|\tilde{x};\theta) - c(\tau|x;\theta), \\ \text{s.t.} \quad & \tilde{x} = G(x, T;\theta),\ c(\tilde{y}) \leq c(y), \end{aligned} \tag{A.3}$$

where $G(x, T;\theta)$ represents the deployment of the attacker G trained on the given model (or solver) θ to conduct T modifications on the clean instance x. It is trained with the objective as in Eq. (A.3) using the RL algorithm (i.e., Proximal Policy Optimization (PPO)). Specifically, the attack process is modelled as a MDP, where, at step t, the state is the current instance $\tilde{x}^{(t)}$; the action is to select an edge whose weight is going to be half; and the reward is the increase of the objective: $c(\tau^{(t+1)}|\tilde{x}^{(t+1)};\theta) - c(\tau^{(t)}|\tilde{x}^{(t)};\theta)$. This process is iterative until T edges are modified. We use ROCO to represent this attack method in the remaining of this paper.

ROCO has been applied to attack MatNet [28] on asymmetric TSP (ATSP). As shown in the empirical results of [70], conducting the vanilla AT may suffer from the trade-off between standard generalization and adversarial robustness. To solve the problem, we further apply our method to defend against it. However, it is not straightforward to adapt ROCO to the inner maximization of CNF, since ROCO belongs to the black-box adversarial attack method, which does not directly rely on the parameters (or gradients) of the current model to generate adversarial instances. Concretely, we first train an attacker G_j using RL for each model θ_j, obtaining M attackers for M models ($\Theta = \{\theta_j\}_{j=0}^{M-1}$) in CNF. For the local attack, we simply use G_j to generate local adversarial instances for θ_j by $G_j(x, T;\theta_j)$. For the global attack, we decompose the generation process (i.e., modifying T edges) of a global adversarial instance $\bar{x}$ as follows:

$$\bar{x}^{(t+1)} = G_b^{(t)}(\bar{x}^{(t)}, 1;\theta_b^{(t)}), \quad \theta_b^{(t)} = \arg\min_{\theta\in\Theta} c(\tau|\bar{x}^{(t)};\theta), \tag{A.4}$$

where $\bar{x}^{(t)}$ is the global adversarial instance; $\theta_b^{(t)}$ is the best-performing model (w.r.t. $\bar{x}^{(t)}$); and $G_b^{(t)}$ is the attacker corresponding to $\theta_b^{(t)}$, at step $t \in [0, T-1]$. Since the model θ_j is updated throughout the optimization, to save the computation, we fix the attacker G_j and only update it every E epochs using the latest model.

A.2 Neural Router

Model Structure. Without loss of generality, we take TSP as an example, where an instance consists of coordinates of n nodes. The attention-based neural router takes as inputs N instances $X \in \mathbb{R}^{N\times n\times 2}$ and a cost matrix $\mathcal{R} \in \mathbb{R}^{N\times M}$, where M is the number of models in CNF. The neural router first embeds the raw inputs into h-dimensional (i.e., 128) features as follows:

$$F_I = \texttt{Mean}(W_1 X + b_1), \quad F_R = W_2 \mathcal{R} + b_2, \tag{A.5}$$

where $F_I \in \mathbb{R}^{N\times h}$ and $F_R \in \mathbb{R}^{N\times h}$ are features of instances and cost matrices, respectively; W_1, W_2 are weight matrices; b_1, b_2 are biases. The `Mean` operator is taken along the second dimension of inputs. Then, a single-head attention layer (i.e., *glimpse* [18]) is applied:

$$Q = W_Q([F_I, F_R]), \quad K = W_K(\texttt{Emb}(M)), \tag{A.6}$$

where $[\,\cdot\,,\,\cdot\,]$ is the horizontal concatenation operator; $Q \in \mathbb{R}^{N\times h}$ is the query matrix; $K \in \mathbb{R}^{M\times h}$ is the key matrix; W_Q, W_K are the weight matrices; $\texttt{Emb}(M) \in \mathbb{R}^{M\times h}$ is a learnable embedding layer representing the features of M models. The logit matrix $\mathcal{P} \in \mathbb{R}^{N\times M}$ is calculated as follows:

$$\mathcal{P} = C \cdot \texttt{tanh}(\frac{QK^T}{\sqrt{h}}), \tag{A.7}$$

where the result is clipped by the `tanh` function with $C = 10$ following [18]. When the neural router is applied to CVRP, we only modify Q by further concatenating it with the features of the depot and node demands, while keeping others the same.

Learned Routing Policy. We attempt to briefly interpret the learned routing policy. We first show the performance of each model $\theta_j \in \Theta$ in CNF in the left panel of Fig. A.1, which is trained on TSP100. Although not all models perform well on both clean and adversarial instances, the collaborative performance of Θ is quite good, demonstrating the diverse expertise of each model and the capability of CNF in reasonably exploiting the overall model capacity. We further give a demonstration (i.e., attention map) of the learned routing policy in the right panel of Fig. A.1. The horizontal axis is the index of the training instance. Concretely, 0-2: clean instances x; 3-11: local adversarial instances $\tilde{x}$; 12-14: global adversarial

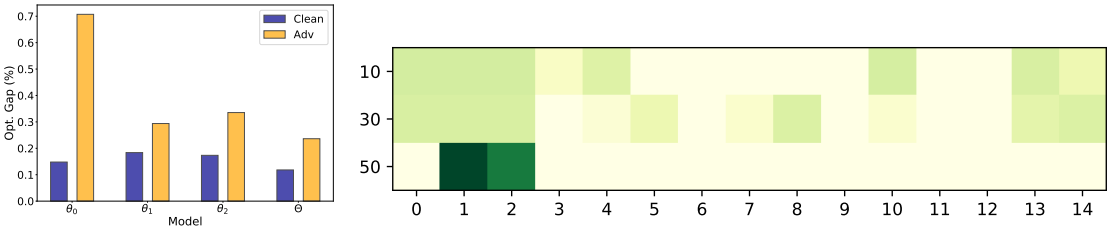


FIGURE A.1: *Left panel:* Performance of each model $\theta_j \in \Theta$ in CNF ($M = 3$), and the overall collaboration performance of Θ. *Right panel:* A demonstration of the learned routing policy for θ_0. A deeper color represents a higher probability.

instances $\bar{x}$. The vertical axis is the epoch of the checkpoint. We take the first model θ_0 as an example, whose expertise lies in clean instances. For simplicity and readability of the results, the batch size is set to $B = 3$, and thus the number of input instances $\mathcal{X}$ for the neural router is 15. The neural router then distributes $B = 3$ instances to each model (if using model choice routing strategies). Note that the instances for different epochs are not the same, while the types remain the same. From the results, we observe that 1) the learned policy tends to distribute all types of instances to each model in a balanced way at the beginning of training, when the model is vulnerable to adversarial instances; 2) clean instances are more likely to be selected at the end of training, when the model is relatively robust to adversarial instances while trying to mitigate the accuracy-robustness trade-off.

A.3 Additional Experiments

Setups for Baselines. We compare our method with several strong traditional and neural VRP solvers. Following the conventional setups in the community [11, 12, 21], for specialized heuristic solvers such as Concorde, LKH3 and HGS, we run them on 32 CPU cores for solving TSP and CVRP instances in parallel, while running neural solvers on one GPU card. Below, we provide the implementation details of baselines. 1) Concorde: We use Concorde Version 03.12.19 with the default setting, to solve TSP instances. 2) LKH3 [5]: We use LKH3 Version 3.0.8 to solve TSP and CVRP instances. For each instance, we run LKH3 with 10000 trails and 1 run. 3) HGS [6]: We run HGS with the default hyperparameters to solve CVRP instances. The maximum number of iterations without improvement is set to 20000. 4) For POMO [12], beyond the open-source pretrained model, we further train it using the vanilla AT framework (POMO_AT). Specifically, following the training

setups presented in Section 3.3, we use the pretrained model to initialize M models, and train them individually using local adversarial instances generated by themselves. 5) POMO_HAC [66] further improves upon the vanilla AT. It constructs a mixed dataset with both clean instances and local adversarial instances for training afterwards. In the outer minimization, it optimizes an instance-reweighted loss function based on the instance hardness. Following their setups, the weight for each instance x_i is defined as: $w_i = \exp(\mathcal{F}(\mathcal{H}(x_i))/\mathcal{T})/\sum_j \exp(\mathcal{F}(\mathcal{H}(x_j))/\mathcal{T})$, where $\mathcal{F}$ is the transformation function (i.e., tanh); $\mathcal{T}$ is the temperature controlling the weight distribution. It starts from 20 and decreases linearly as the epoch increasing. The hardness $\mathcal{H}$ is computed following Eq. (3.7). 6) POMO_DivTrain is adapted from the diversity training [123], which studies the ensemble-based adversarial robustness in the image domain by proposing a novel method to train an ensemble of models with uncorrelated loss functions. Specifically, it improves the ensemble diversity by minimizing the cosine similarity between the gradients of (sub-)models w.r.t. the input. Its loss function is formulated as: $\mathcal{L} = \ell + \lambda \log(\sum_{1 \le a < b \le M} \exp(\frac{<\nabla_x \ell_a, \nabla_x \ell_b>}{|\nabla_x \ell_a||\nabla_x \ell_b|}))$, where ℓ is the original loss function; M is the number of models; $\nabla_x \ell_a$ is the gradient of the loss function (on the a_{th} model) w.r.t. the input x; $\lambda = 0.5$ is a hyperparameter controlling the importance of gradient alignment during training. 7) CNF_Greedy: the neural router is simply replaced by the heuristic method, where each model selects $\mathcal{K}$ hardest instances. We use [66] as the attack method in Section 3.3. For training efficiency, we set $T = 1$ in the main experiments. The step size α is randomly sampled from 1 to 100.

Setups for Ablation Study. We conduct extensive ablation studies on components, hyperparameters and routing strategies as shown in Section 3.3. For simplicity, we slightly modified the training setups. We train all methods using 2.5M TSP100 instances. The learning rate is decayed by 10 for the last 20% training instances. For the ablation on components (Fig. 3.3(a)), we set the attack steps as $T = 2$, and remove each component separately to demonstrate the effectiveness of each component in our proposed framework. For the ablation on hyperparameters (Fig. 3.3(b)), we train multiple models with $M \in \{2, 3, 4, 5\}$. For the ablation on routing strategies (Fig. 3.3(c)), we set $\mathcal{K} = B$ for M-Top$\mathcal{K}$, where $B = 64$ is the batch size, and $\mathcal{K} = 1$ for I-Top$\mathcal{K}$. The other training setups remain the same.

Setups for Versatility Study. For the pretraining stage, we train MatNet [28] on 5M ATSP20 instances following the original setup from [28]. We further train

a perturbation model by attacking it using reinforcement learning. Concretely, we use the dataset from Lu et al. [70], consisting of 50 'tmat' class ATSP training instances that obey the triangle inequality, to train the perturbation model for 500 epochs. Adam optimizer is used with the learning rate of $1e-3$. The maximum number of actions taken by the perturbation model is $T = 10$. After the pretraining stage, we use the pretrained model to initialize $M = 3$ models, and further adversarially train them. We fix the perturbation model and only update it using the latest model every $E = 10$ epochs. After the 10_{th} epoch, there would be M perturbation models corresponding to M models. Following Lu et al. [70], we use the fixed 1K clean instances for training. In the inner maximization, we generate MK local adversarial instances and 1K global adversarial instances using the perturbation models. However, since the perturbation model is not efficient (i.e., it needs to conduct beam search to find edges to be modified), we generate adversarial instances in advance and reuse them later. Then, in the outer minimization, we load all instances and distribute them to each model using the jointly trained neural router. The models are then adversarially trained for 20 epochs using the Adam optimizer with the learning rate of $4e-5$, the weight decay of $1e-6$, and the batch size of $B = 100$.

Data Generation. We follow the instructions from Kool et al. [11] to generate synthetic instances. 1) *Uniform Distribution:* The node coordinate of each node is uniformly sampled from the unit square $U(0, 1)$. 2) *Rotation Distribution:* Following Bossek et al. [137], we mutate nodes, which originally follow the uniform distribution, by rotating a subset of them (anchored in the origin of the Euclidean plane). The coordinates of selected nodes are transformed by multiplying them with a matrix $\begin{bmatrix} \cos(\varphi) & \sin(\varphi) \\ -\sin(\varphi) & \cos(\varphi) \end{bmatrix}$, where $\varphi \sim [0, 2\pi]$ is the rotation angle. 3) *Explosion Distribution:* Following [137], we mutate nodes, which originally follow the uniform distribution, by simulating a random explosion. Specifically, we first randomly select the center of explosion v_c. All nodes v_i within the explosion radius $R = 0.3$ is moved away from the center of explosion with the form of $v_i = v_c + (R + s) \cdot \frac{v_c - v_i}{||v_c - v_i||}$, where $s \sim \text{Exp}(\lambda = 1/10)$ is a random value drawn from an exponential distribution.

In this paper, we mainly consider the distribution of node coordinates. For CVRP instances, the coordinate of the depot node v_0 is uniformly sampled from the unit square $U(0, 1)$. The demand of each node δ_i is randomly sampled from a discrete

uniform distribution $\{1, \ldots, 9\}$. The capacity of each vehicle is set to $Q = \lceil 30 + \frac{n}{5} \rceil$, where n is the size of a CVRP instance. The demand and capacity are further normalized to $\delta_i' = \delta/Q$ and 1, respectively.

Training Efficiency. We empirically observe that CNF requires more training time than the vanilla AT variants on TSP100 (e.g., POMO_AT (3) 32 hrs vs. CNF (3) 85 hrs). However, simply further training them cannot significantly increase their performance since the training process has nearly converged. For example, given roughly the same training time, POMO_AT achieves 0.246% and 0.278% while CNF achieves 0.118% and 0.236% on the metrics of Uniform and Fixed Adv., respectively. On the other hand, our training time is less than that of advanced AT methods (e.g., POMO_TRS (3) [154] needs around 94 hrs). The above comparison indicates that CNF can achieve a better trade-off to deliver superior results within a reasonable computational budget. Moreover, we find that the global attack generation consumes much more training time than the neural router ($\sim$0.05 million parameters). During inference, the computational complexity of CNF only depends on the number of models, since the neural router is only activated during training, and is discarded afterwards. Therefore, all methods with the same number of trained models have the same inference time.

TABLE A.1: Results on TSPLIB instances. Models are trained on $n = 100$.

Instance	Opt.	POMO		POMO_AT		POMO_HAC		POMO_DivTrain		CNF	
		Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap
kroA100	21282	21420	0.65%	21347	0.31%	**21308**	**0.12%**	21370	0.41%	**21308**	**0.12%**
kroB100	22141	22200	0.27%	22211	0.32%	22200	0.27%	**22199**	**0.26%**	22216	0.34%
kroC100	20749	20799	0.24%	20768	0.09%	**20753**	**0.02%**	20768	0.09%	20758	0.04%
kroD100	21294	21446	0.71%	21391	0.46%	21407	0.53%	21435	0.66%	**21353**	**0.28%**
kroE100	22068	22259	0.87%	22288	1.00%	22167	0.45%	22213	0.66%	**22121**	**0.24%**
eil101	629	630	0.16%	630	0.16%	**629**	**0.00%**	631	0.32%	630	0.16%
lin105	14379	14477	0.68%	14426	0.33%	14408	0.20%	**14402**	**0.16%**	14403	0.17%
pr107	44303	44678	0.85%	47819	7.94%	**44596**	**0.66%**	46285	4.47%	44719	0.94%
pr124	59030	59389	0.61%	59257	0.38%	59385	0.60%	59558	0.89%	**59076**	**0.08%**
bier127	118282	133042	12.48%	118606	0.27%	118608	0.28%	**118337**	**0.05%**	118841	0.47%
ch130	6110	6119	0.15%	6130	0.33%	6115	0.08%	6125	0.25%	**6111**	**0.02%**
pr136	96772	97983	1.25%	100225	3.57%	97617	0.87%	100145	3.49%	**97567**	**0.82%**
pr144	58537	58935	0.68%	59544	1.72%	58913	0.64%	59265	1.24%	**58868**	**0.57%**
ch150	6528	6554	0.40%	6582	0.83%	6556	0.43%	6578	0.77%	**6550**	**0.34%**
kroA150	26524	26755	0.87%	26898	1.41%	26736	0.80%	26813	1.09%	**26722**	**0.75%**
kroB150	26130	26405	1.05%	26506	1.44%	**26379**	**0.95%**	26467	1.29%	26494	1.39%
pr152	73682	**74249**	**0.77%**	77537	5.23%	75291	2.18%	77127	4.68%	74876	1.62%
rat195	2323	2486	7.02%	2500	7.62%	2461	5.94%	2467	6.20%	**2449**	**5.42%**
kroA200	29368	29992	2.12%	30222	2.91%	29771	1.37%	30143	2.64%	**29755**	**1.32%**
kroB200	29437	30298	2.92%	30157	2.45%	29890	1.54%	30267	2.82%	**29862**	**1.44%**
ts225	126643	134609	6.29%	135801	7.23%	**128085**	**1.14%**	134569	6.26%	128436	1.42%
tsp225	3916	4035	3.04%	4031	2.94%	4004	2.25%	4025	2.78%	**4001**	**2.17%**
pr226	80369	**83470**	**3.86%**	89455	11.31%	85466	6.34%	90347	12.42%	84914	5.66%
pr264	49135	55157	12.26%	60390	22.91%	53894	9.69%	60957	24.06%	**53763**	**9.42%**
a280	2579	2740	6.24%	2741	6.28%	**2690**	**4.30%**	2760	7.02%	2701	4.73%
pr299	48191	50785	5.38%	50812	5.44%	50487	4.76%	50535	4.86%	**50408**	**4.60%**
lin318	42029	43430	3.33%	43808	4.23%	**42860**	**1.98%**	43840	4.31%	43060	2.45%
rd400	15281	15775	3.23%	16221	6.15%	**15755**	**3.10%**	16135	5.59%	15778	3.25%
fl417	11861	**13958**	**17.68%**	15240	28.49%	14544	22.62%	15637	31.84%	14317	20.71%
pr439	107217	123357	15.05%	120545	12.43%	117963	10.02%	120212	12.12%	**117632**	**9.71%**
pcb442	50778	54087	6.52%	54686	7.70%	**53165**	**4.70%**	55125	8.56%	53281	4.93%
d493	35002	64215	83.46%	38356	9.58%	**37685**	**7.67%**	38168	9.05%	38051	8.71%
u574	36905	41456	12.33%	42045	13.93%	40804	10.57%	41990	13.78%	**40737**	**10.38%**
rat575	6773	7828	15.58%	7774	14.78%	7658	13.07%	7791	15.03%	**7635**	**12.73%**
p654	34643	46094	33.05%	51915	49.86%	**45447**	**31.19%**	52837	52.52%	46425	34.01%
d657	48912	59082	20.79%	56232	14.97%	55580	13.63%	56635	15.79%	**55066**	**12.58%**
u724	41910	49171	17.33%	50583	20.69%	48818	16.48%	50679	20.92%	**48692**	**16.18%**
rat783	8806	10819	22.86%	10769	22.29%	10512	19.37%	10767	22.27%	**10473**	**18.93%**
pr1002	259045	325734	25.74%	328459	26.80%	322276	24.41%	335954	29.69%	**320624**	**23.77%**

TABLE A.2: Results on CVRPLIB instances. Models are trained on $n = 100$.

Instance	Opt.	POMO		POMO_AT		POMO_HAC		POMO_DivTrain		CNF	
		Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap
X-n101-k25	27591	29282	6.13%	28919	4.81%	29176	5.74%	29019	5.18%	**28911**	**4.78%**
X-n106-k14	26362	26961	2.27%	**26608**	**0.93%**	26789	1.62%	26679	1.20%	26672	1.18%
X-n110-k13	14971	15154	1.22%	15298	2.18%	15305	2.23%	**15125**	**1.03%**	15127	1.04%
X-n120-k6	13332	14574	9.32%	13762	3.23%	13734	3.02%	13801	3.52%	**13652**	**2.40%**
X-n134-k13	10916	11315	3.66%	**11189**	**2.50%**	11250	3.06%	11259	3.14%	11248	3.04%
X-n143-k7	15700	16382	4.34%	16233	3.39%	16019	2.03%	16192	3.13%	**15980**	**1.78%**
X-n148-k46	43448	47613	9.59%	46546	7.13%	46433	6.87%	46504	7.03%	**45694**	**5.17%**
X-n162-k11	14138	14986	6.00%	14923	5.55%	14827	4.87%	14942	5.69%	**14794**	**4.64%**
X-n181-k23	25569	26969	5.48%	26282	2.79%	26299	2.86%	**26211**	**2.51%**	26213	2.52%
X-n195-k51	44225	50296	13.73%	50228	13.57%	48987	10.77%	51367	16.15%	**48823**	**10.40%**
X-n214-k11	10856	11752	8.25%	11868	9.32%	**11570**	**6.58%**	11760	8.33%	11587	6.73%
X-n233-k16	19230	21107	9.76%	21054	9.49%	20997	9.19%	21067	9.55%	**20980**	**9.10%**
X-n251-k28	38684	41355	6.90%	41313	6.80%	41193	6.49%	41212	6.54%	**40992**	**5.97%**
X-n270-k35	35291	39952	13.21%	38837	10.05%	**38376**	**8.74%**	38511	9.12%	38411	8.84%
X-n289-k60	95151	105343	10.71%	104923	10.27%	**104236**	**9.55%**	104786	10.13%	104261	9.57%
X-n294-k50	47161	53937	14.37%	53772	14.02%	52876	12.12%	53789	14.05%	**52699**	**11.74%**
X-n313-k71	94043	105470	12.15%	105647	12.34%	104179	10.78%	104375	10.99%	**103726**	**10.30%**
X-n331-k15	31102	42292	35.98%	39293	26.34%	36957	18.83%	37047	19.11%	**36194**	**16.37%**
X-n376-k94	147713	162224	9.82%	**157156**	**6.39%**	158410	7.24%	158168	7.08%	157880	6.88%
X-n384-k52	65928	76139	15.49%	76527	16.08%	76080	15.40%	78141	18.52%	**74878**	**13.58%**
X-n439-k37	36391	46077	26.62%	48372	32.92%	49018	34.70%	46614	28.09%	**45219**	**24.26%**
X-n469-k138	221824	**252278**	**13.73%**	264366	19.18%	258635	16.59%	263698	18.88%	258751	16.65%
X-n502-k39	69226	85692	23.79%	83326	20.37%	83702	20.91%	80082	15.68%	**79373**	**14.66%**

Appendix B

Appendix for Chapter 4

B.1 Analysis of First-Order Approximation

Without loss of generality, we consider optimization with the stochastic gradient descent (SGD), and treat each task equally. Therefore, the meta-model update in Reptile [128] (i.e., Eq. (4.9)) could be rewritten as:

$$\theta_0 \leftarrow \theta_0 + \beta \frac{1}{B} \sum_{i=1}^{B} (\theta_i^{(K)} - \theta_0). \tag{B.1}$$

For each task $\mathcal{T}_i$, $(\theta_0 - \theta_i^{(K)})/\alpha$ could be viewed as the (Reptile) gradient term g_R in the SGD formulation, and α, β are the step sizes of inner-loop and outer-loop optimization, respectively. When $K = 1$, Reptile is equivalent to the joint training on the expected loss of the training tasks:

$$g_R^1 = \mathbb{E}_{\mathcal{T}_i \sim p(\mathcal{T})}[\frac{\theta_0 - \theta_i^{(1)}}{\alpha}] = \mathbb{E}_{\mathcal{T}_i \sim p(\mathcal{T})}[\frac{\theta_0 - (\theta_0 - \alpha \nabla_{\theta_0} \mathcal{L}_i(\theta_0))}{\alpha}] = \mathbb{E}_{\mathcal{T}_i \sim p(\mathcal{T})}[\nabla_{\theta_0} \mathcal{L}_i(\theta_0)]. \tag{B.2}$$

However, in this case, it is equivalent to the naive pretraining on a large training task set, which requires ad-hoc tricks to achieve desirable fine-tuning performance. When performing multiple gradient updates ($K > 1$) in the inner-loop optimization, Reptile is able to incorporate information from higher-order derivatives of the loss function. For the simplicity of notations, we omit the index for task i, and use

the following definitions:

$$g^{(k)} = \frac{\partial \mathcal{L}(\theta^{(k)})}{\partial \theta^{(k)}}; \quad \tilde{g}^{(k)} = \frac{\partial \mathcal{L}(\theta^{(k)})}{\partial \theta^{(0)}}; \quad \tilde{h}^{(k)} = \frac{\partial \mathcal{L}^2(\theta^{(k)})}{\partial (\theta^{(0)})^2}; \quad k \in [0, K], \tag{B.3}$$

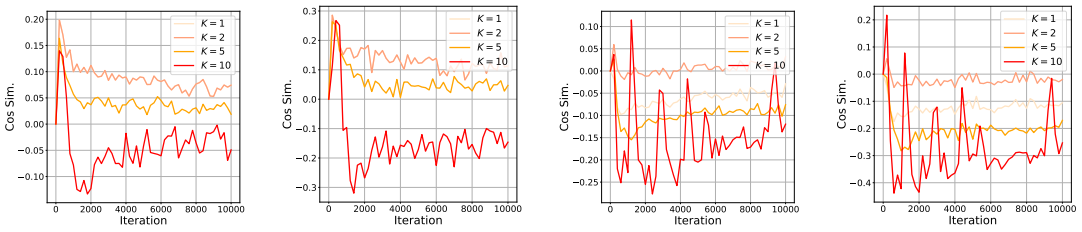
where $g^{(k)}, \tilde{g}^{(k)}$ are the gradients of the loss function with respect to (w.r.t.) the task-specific model $\theta^{(k)}$ and meta-model $\theta^{(0)} = \theta_0$, and $\tilde{h}^{(k)}$ is the hessian w.r.t. the meta-model. With the Taylor expansion, the gradient of the loss function w.r.t the task-specific model can be expressed as:

$$\begin{aligned} g^{(k)} = \frac{d\mathcal{L}}{d\theta}|_{\theta=\theta^{(k)}} &\approx \frac{d\mathcal{L}}{d\theta}|_{\theta=\theta^{(0)}} + \frac{d^2\mathcal{L}}{d\theta^2}|_{\theta=\theta^{(0)}}(\theta^{(k)} - \theta^{(0)}) \\ &\approx \frac{\partial \mathcal{L}(\theta^{(k)})}{\partial \theta^{(0)}} + \frac{\partial \mathcal{L}^2(\theta^{(k)})}{\partial (\theta^{(0)})^2}(\theta^{(0)} - \alpha \sum_{j=0}^{k-1} \frac{\partial \mathcal{L}(\theta^{(j)})}{\partial \theta^{(j)}} - \theta^{(0)}) \\ &\approx \tilde{g}^{(k)} - \alpha \tilde{h}^{(k)} \sum_{j=0}^{k-1} g^{(j)}. \end{aligned} \tag{B.4}$$

Indeed, FOMAML [132] simply drops the higher-order term and uses $g^{(K)}$ as the approximation to the second-order derivative, while Reptile approximates it in the following way:

$$g_R^K = \frac{1}{\alpha}(\theta_0 - \theta^{(K)}) = \frac{1}{\alpha}(\theta_0 - (\theta_0 - \alpha \sum_{j=0}^{K-1} g^{(j)})) = \sum_{j=0}^{K-1} g^{(j)}. \tag{B.5}$$

For example, if we run $K = 2$ steps in the inner-loop optimization, based on Eqs. (B.4)-(B.5), the gradient of Reptile is $g_R^2 = g^{(0)} + g^{(1)} \approx \tilde{g}^{(0)} + \tilde{g}^{(1)} - \alpha \tilde{h}^{(1)} g^{(0)}$, and the gradient of FOMAML is $g_F^2 = g^{(2)} \approx \tilde{g}^{(2)} - \alpha \tilde{h}^{(2)}(g^{(0)} + g^{(1)})$. However, it is non-trivial to execute the above derivations on our more complicated RL setting with Adam optimizer. Therefore, empirically, we further conduct an experiment to check whether g_R^K could serve as a good approximation on our setting. Specifically, we meta-train POMO with Ours-SO for $K \in [1, 2, 5, 10]$ steps in the inner-loop optimization. We collect the gradient direction of the second-order derivative $\text{sign}(\tilde{g}^{(K)})$ and that of the Reptile's approximation $\text{sign}(g_R^K)$, and compute their cosine similarity. Moreover, since we use the Adam optimizer, we also try to load the gradient statistics (e.g., momentum in the optimizer for outer-loop optimization) when conducting the inner-loop optimization. As indicated in Fig. B.1, Reptile fails to well approximate the second-order derivatives on our RL setting. As the step K increases, the cosine similarity decreases accordingly, which

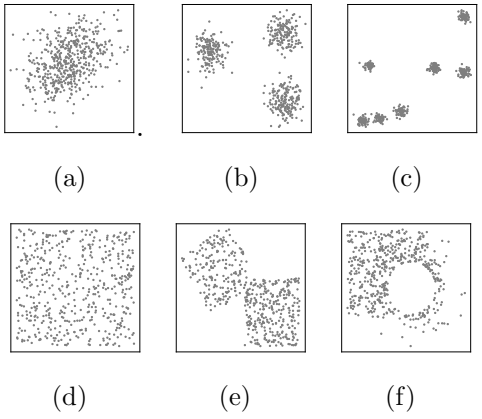


(a) First layer (b) Last layer (c) First layer (d) Last layer

FIGURE B.1: The cosine similarity of gradient directions between the second-order derivative and the Reptile's approximation.

may be attributed to the accumulated effect throughout the K steps. However, a larger step K empirically results in a relatively better zero-shot generalization performance given the same amount of training instances.

B.2 Data Generation



(a) (b) (c)

(d) (e) (f)

FIGURE B.2: TSP500 instances following various distributions: (a) GM_2^5: Gaussian mixture distribution with $c = 2, l = 5$; (b) GM_3^{10}: Gaussian mixture distribution with $c = 3, l = 10$; (c) GM_7^{50}: Gaussian mixture distribution with $c = 7, l = 50$; (d) U: Uniform distribution; (e) R: Rotation distribution; (f) E: Explosion distribution.

We consider VRP instances with various sizes and distributions. Since generating instances with different sizes is relatively easy, here we focus on the details regarding the generation of different distributions of node coordinates. To provide diverse data for meta-training, we generate instances following the uniform distribution, and the gaussian mixture distribution, which is demonstrated to be effective in capturing different hardness levels of the instances [155]. We further generate instances following rotation and explosion distributions to evaluate the generalization of the learned model. Below, we provide the details on the data generation procedure.

Uniform distribution. Following the convention in Kool et al. [11], Kwon et al. [12], as shown in Fig. B.2(d), the node coordinate of each node is uniformly sampled from the unit square $U(0, 1)$.

Gaussian mixture distribution. Following Manchanda et al. [54], Zhang et al. [66], we parameterize the gaussian mixture distribution with two hyperparameters, cluster c and scale l. For the simplicity of notations, we denote it as GM_c^l. Specifically, we first generate the coordinate of the center node v_{c_i} of each cluster c_i by uniformly sampling from $U(0, l)$. Other nodes $\mathcal{V} \setminus \{v_{c_i}\}_{i=1}^{c}$ are then equally distributed into c clusters, where the coordinates of nodes in each cluster forms a (multivariate) gaussian distribution. For example, if a node belongs to the cluster c_i, its coordinate is sampled from $\mathcal{N}(n_{c_i}, \mathbf{I})$, where the mean is the coordinate of the center node, and the covariance matrix is the identity matrix. Finally, we scale and translate the range of coordinates into $[0, 1]$ using the min-max normalization. Some exemplary instances are shown in Fig. B.2(a)-B.2(c).

Rotation distribution. Following Bossek et al. [137], we mutate nodes, which originally follow the uniform distribution, by rotating a subset of them (anchored in the origin of the Euclidean plane) as shown in Fig. B.2(e). The coordinates of selected nodes are transformed by multiplying with $\begin{bmatrix} \cos(\varphi) & \sin(\varphi) \\ -\sin(\varphi) & \cos(\varphi) \end{bmatrix}$, where $\varphi \sim [0, 2\pi]$ is the rotation angle.

Explosion distribution. Following Bossek et al. [137], we mutate nodes, which originally follow the uniform distribution, by simulating a random explosion. Specifically, we first randomly select the center of the explosion v_c (i.e., the hole in Fig. B.2(f)). All nodes v_i within the explosion radius $R = 0.3$ is moved away from the center with the transformation form of $v_i = v_c + (R + s) \cdot \frac{v_c - v_i}{||v_c - v_i||}$, where $s \sim \text{Exp}(\lambda = 1/10)$ is a random value drawn from an exponential distribution.

In this paper, we mainly consider the distribution of node coordinates. For CVRP instances, following Kool et al. [11], Kwon et al. [12], the coordinate of the depot node v_0 is uniformly sampled from the unit square $U(0, 1)$. The demand of each node δ_i is randomly sampled from a discrete uniform distribution $\{1, \cdots, 9\}$. The capacity of each vehicle is set to $Q = \lceil 30 + \frac{n}{5} \rceil$, where $n \geq 50$ is the size of CVRP instances. The demand and capacity are further normalized to $\delta_i' = \delta/Q$ and 1, respectively. During meta-training, we generate the diverse task set with hundreds of tasks $\mathcal{T}(n, d)$, where $n \in \mathcal{N} = \{50, 55, \cdots, 200\}$ and $d(c, l) \in \mathcal{D} =$

$\{(0,0),(1,1)\} \cup \{c[3,5,7] \times l[10,30,50]\}$. We denote the uniform distribution as $d(c=0,l=0)$. For each size, there are 11 tasks with different distributions in the training task set, and therefore $31 \times 11 = 341$ tasks in total.

B.3 Additional Experiments

Setups for Experiments in Fig. 4.2. We follow the training setups presented in Section 4.3. For the left panel of Fig. 4.2, we show the validation result of each method on CVRP (300, R). Specifically, *Ours-SO* refers to the second-order method presented in Algorithm 2, *FOMAML* refers to its first-order approximation, where we replace the line 16 of Algorithm 2 with Eq. (4.8), and *Ours* refers to the proposed approximation method, where we simply early-stop using the second-order derivative when the training tends to be stable (i.e., at the 50K_{th} iteration), and leverage the first-order one afterwards. For the right panel of Fig. 4.2, the training process follows *Ours-SO*. However, in each iteration of meta-training, besides calculating the direction of the second-order derivative $\text{sign}(\nabla_{\theta_0}\mathcal{L}_i(\theta_i^{(K)}))$, we also collect that of FOMAML $\text{sign}(\nabla_{\theta_i}\mathcal{L}_i(\theta_i^{(K)}))$ and Reptile $\text{sign}(\theta_0 - \theta_i^{(K)})$ using the same batch of instances. Note that these two gradients are not used for meta updates. Then, we compute the cosine similarities of these gradient directions with the second-order one (i.e., $\text{sign}(\nabla_{\theta_0}\mathcal{L}_i(\theta_i^{(K)}))$), and show the average result over 500 iterations in the right panel of Fig. 4.2.

Training Setups for Baselines. We conduct all experiments on a machine with NVIDIA A100 PCIe (80GB) cards and AMD EPYC 7513 CPU at 2.6GHz. As shown in Section 4.3, we compare our method with several strong baselines. Following the conventional setups in the community [11, 12, 21], for traditional VRP solvers such as Concorde, LKH3 and HGS, we run them on 32 CPU cores for solving TSP and CVRP instances, while running neural VRP solvers on one GPU card. Below, we provide implementation details of all baselines. 1) Concorde [135]: We use Concorde Version 03.12.19 with the default setting, to solve TSP instances. 2) LKH3 [5]: We use LKH3 Version 3.0.7 to solve TSP and CVRP instances. For each instance, we run LKH3 with 10000 trails and 10 runs. 3) HGS [6]: We run HGS with the default hyperparameters to solve CVRP instances. The maximum number of iterations (without improvement) is set to 20000. 4) For POMO [12], following the training setups presented in Section 4.3, we re-train it for 500K iterations with

totally 32M instances, which are randomly sampled from our training task set. 5) AMDKD-POMO [68] tackles cross-distribution generalization of neural solvers using knowledge distillation. Specifically, it leverages the knowledge from multiple teacher models pretrained on different distributions to yield a generalizable student model. However, it is computationally intractable to obtain a pretrained model for each task since we have hundreds of training tasks on our problem setting. Therefore, following the default setting of Bi et al. [68], we pretrain three teacher models on instances of size $n = 200$, but with distributions chosen from our training task set (i.e., the uniform U and gaussian mixture distributions GM_3^{10}, GM_7^{50}). We train each teacher model using 6.4M instances. After pretraining, we train a light-weight yet generalizable student model by adaptively distilling from the teacher models on another set of 12.8M instances ($n = 200$), so that the total amount of training instances (i.e., 32M) is close to other methods. 6) Meta-POMO [54] leverages Reptile [128], which does not need to split data into training and validation sets. We set $\beta = 0.9, B = 1, K = 50$ and meta-train POMO using Reptile for 10K iterations to keep the same amount of training instances as other methods.

Results on Benchmark Instances. We evaluate all methods on the classical benchmark datasets, such as TSPLIB [125] and CVRPLIB (Set-X) [126], where we choose representative instances with size $n \in [100, 1002]$. We also combine our method with the efficient active search [21]. Specifically, following their original implementation, we run EAS-Lay and EAS-Emb (with 1 run and 200 iterations) on each instance, and report the best result. Due to the huge GPU memory it needs, we only run it on instances with size $n \in [100, 750]$. The detailed results are shown in Table B.1 and Table B.2.

TABLE B.1: Results on TSPLIB instances.

Instance	Opt.	POMO		AMDKD-POMO		Meta-POMO		Ours		Ours+EAS	
		Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap
kroA100	21282	**21282**	**0.00%**	21360	0.37%	21308	0.12%	21305	0.11%	21282	0.00%
kroA150	26524	**26823**	**1.13%**	26997	1.78%	26852	1.24%	26873	1.32%	26566	0.16%
kroA200	29368	**29745**	**1.28%**	30196	2.82%	29749	1.30%	29823	1.55%	29460	0.31%
kroB200	29437	30060	2.12%	30188	2.55%	29896	1.56%	**29814**	**1.28%**	29445	0.03%
ts225	126643	131208	3.60%	**128210**	**1.24%**	131877	4.13%	128770	1.68%	127281	0.50%
tsp225	3916	4040	3.17%	4074	4.03%	4047	3.35%	**4008**	**2.35%**	3933	0.43%
pr226	80369	**81509**	**1.42%**	82430	2.56%	81968	1.99%	81839	1.83%	81235	1.08%
pr264	49135	50513	2.80%	51656	5.13%	**50065**	**1.89%**	50649	3.08%	49212	0.16%
a280	2579	2714	5.23%	2773	7.52%	2703	4.81%	**2695**	**4.50%**	2591	0.47%
pr299	48191	50571	4.94%	51270	6.39%	49773	3.28%	**49348**	**2.40%**	48449	0.54%
lin318	42029	44011	4.72%	44154	5.06%	**43807**	**4.23%**	43828	4.28%	43090	2.52%
rd400	15281	16254	6.37%	16610	8.70%	16153	5.71%	**15948**	**4.36%**	15531	1.64%
fl417	11861	12940	9.10%	13129	10.69%	12849	8.33%	**12683**	**6.93%**	12754	7.53%
pr439	107217	115651	7.87%	117872	9.94%	114872	7.14%	**114487**	**6.78%**	111902	4.37%
pcb442	50778	55273	8.85%	56225	10.73%	55507	9.31%	**54531**	**7.39%**	53069	4.51%
d493	35002	38388	9.67%	38400	9.71%	38641	10.40%	**38169**	**9.05%**	37850	8.14%
u574	36905	41574	12.65%	41426	12.25%	41418	12.23%	**40515**	**9.78%**	39295	6.48%
rat575	6773	**7617**	**12.46%**	7707	13.79%	7620	12.51%	7658	13.07%	7333	8.27%
p654	34643	38556	11.30%	39327	13.52%	38307	10.58%	**37488**	**8.21%**	39141	12.98%
d657	48912	55133	12.72%	55143	12.74%	54715	11.86%	**54346**	**11.11%**	53077	8.52%
u724	41910	48855	16.57%	48738	16.29%	48272	15.18%	**48026**	**14.59%**	48144	14.87%
rat783	8806	10401	18.11%	10338	17.40%	**10228**	**16.15%**	10300	16.97%	–	
pr1002	259045	310855	20.00%	312299	20.56%	308281	19.01%	**305777**	**18.04%**	–	

TABLE B.2: Results on CVRPLIB (Set-X) instances.

Instance	Opt.	POMO		AMDKD-POMO		Meta-POMO		Ours		Ours+EAS	
		Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap
X-n101-k25	27591	**28804**	**4.40%**	28947	4.91%	29647	7.45%	29442	6.71%	27750	0.58%
X-n153-k22	21220	23701	11.69%	23179	9.23%	23428	10.41%	**22810**	**7.49%**	21864	3.03%
X-n200-k36	58578	**60983**	**4.11%**	61074	4.26%	61632	5.21%	61496	4.98%	59765	2.03%
X-n251-k28	38684	**40027**	**3.47%**	40262	4.08%	40477	4.63%	40059	3.55%	39198	1.33%
X-n303-k21	21736	22724	4.55%	22861	5.18%	22661	4.26%	**22624**	**4.09%**	22035	1.38%
X-n351-k40	25896	**27410**	**5.85%**	27431	5.93%	27992	8.09%	27515	6.25%	26644	2.89%
X-n401-k29	66154	68435	3.45%	68579	3.67%	68272	3.20%	**68234**	**3.14%**	67365	1.83%
X-n459-k26	24139	26612	10.24%	26255	8.77%	25789	6.84%	**25706**	**6.49%**	25144	4.16%
X-n502-k39	69226	71435	3.19%	71390	3.13%	71209	2.86%	**70769**	**2.23%**	70277	1.52%
X-n548-k50	86700	90904	4.85%	90890	4.83%	90743	4.66%	**90592**	**4.49%**	89542	3.28%
X-n599-k92	108451	115894	6.86%	115702	6.69%	**115627**	**6.62%**	116964	7.85%	113089	4.28%
X-n655-k131	106780	110327	3.32%	111587	4.50%	110756	3.72%	**110096**	**3.11%**	108433	1.55%
X-n701-k44	81923	86933	6.12%	88166	7.62%	86605	5.72%	**86005**	**4.98%**	85432	4.28%
X-n749-k98	77269	**83294**	**7.80%**	83934	8.63%	84406	9.24%	83893	8.57%	81040	4.88%
X-n801-k40	73311	80584	9.92%	80897	10.35%	79077	7.87%	**78171**	**6.63%**	–	
X-n856-k95	88965	96398	8.35%	95809	7.69%	**95801**	**7.68%**	96739	8.74%	–	
X-n895-k37	53860	61604	14.38%	62316	15.70%	59778	10.99%	**58947**	**9.44%**	–	
X-n957-k87	85465	93221	9.08%	93995	9.98%	92647	8.40%	**92011**	**7.66%**	–	
X-n1001-k43	72355	82046	13.39%	82855	14.51%	79347	9.66%	**78955**	**9.12%**	–	

Appendix C

Appendix for Chapter 5

C.1 Setups of VRP Variants

We mainly consider five constraints as presented in Section 5.1. Our base VRP variant is CVRP, from which we derive 16 VRP variants by adding additional constraints (as listed in Table C.1). The node coordinates are randomly sampled from the unit square $U(0, 1)$. Below, we provide a comprehensive description of the data generation process for each constraint.

Capacity (C). We follow the common settings in literature [11, 12]. The node demand δ_i is randomly sampled from a discrete uniform distribution $\{1, 2, \ldots, 9\}$. The vehicle capacity Q is set to 40 and 50 for $n = 50$ and $n = 100$, respectively. Before feeding into the network, the node demand is further normalized by $\delta'_i = \delta_i/Q$. During the decoding process, we mask all nodes whose demands are larger than the remaining capacity of the current vehicle.

Open Route (O). The open route constraint does not involve extra data generation. During the decoding process, we set the open route indicator $o_t = 1$ in the dynamic feature set $\mathcal{D}_t$. As mentioned in Section 5.1, the integration of the open route constraint into others is not a simple addition. Concretely, 1) *O w. L:* Since the vehicle does not return to the depot node, during the decoding process, we only need to mask the node v_j satisfying $l_t + d_{ij} > \mathbb{L}$, where l_t is the length of the current route; d_{ij} is the distance between the current node v_i and unmasked node v_j; $\mathbb{L}$ is the predefined threshold of the route length. Without the open route

constraint, we should mask the node v_j satisfying $l_t + d_{ij} + d_{j0} > \mathbb{L}$; 2) *O w. TW:* During the decoding process, we mask the node v_j satisfying $t_t + d_{ij} > l_j$, where t_t is the current time (i.e., the completed time of serving the current node v_i); d_{ij} is the distance (i.e., travel time since the vehicle speed is 1) between the current node v_i and unmasked node v_j; l_j is the end time window of v_j. Without the open route constraint, we need to *further* mask the node v_j satisfying $\max(t_t + d_{ij}, e_j) + s_j + d_{j0} > l_0$, where e_j is the start time window of v_j; s_j is the service time; l_0 is the end time window of the depot node v_0. 3) *O w. C or B:* Since the demand of the depot node is 0, it does not affect the constraint satisfaction during decoding, except that the vehicle does not need to return to the depot node at the end of each route.

Backhaul (B). Similar to CVRP, we first generate node demands by randomly sampling from a discrete uniform distribution $\{1, 2, \ldots, 9\}$. Then, following Liu et al. [74], we randomly select 20% of customers as backhauls. In this paper, we consider routes that involve a mixture of linehauls and backhauls, without the presence of strict precedence constraints between them. For neural VRP solvers, to ensure the feasibility of constructed solutions, the initial customer node chosen for each vehicle should be a linehaul, with an exception being that all remaining (unvisited) nodes are backhauls.

Duration Limit (L). Following Liu et al. [74], we set the duration limit (i.e., the maximum length of each vehicle route) as $\mathbb{L} = 3$, which ensures the neural solver can find feasible solutions within the unit square space $U(0, 1)$.

Time Window (TW). Following Li et al. [83] whose data generation is similar to the procedure as in Solomon [156], we set the time window for the depot node v_0 as $[e_0, l_0] = [0, 3]$, and the service time at each customer node v_i as $s_i = 0.2$. The depot node does not have service time. The time window for the customer node v_i is then obtained by 1) sampling the time window center $\gamma_i \sim U(e_0 + d_{0i}, l_0 - d_{i0} - s_i)$, where $d_{0i} = d_{i0}$ denotes the distance or travel time between v_0 and v_i; 2) sampling the time window half-width h_i uniformly at random from $[s_i/2, l_0/3] = [0.1, 1]$; 3) setting the time window $[e_i, l_i]$ as $[\max(e_0, \gamma_i - h_i), \min(l_0, \gamma_i + h_i)]$.

TABLE C.1: 16 VRP variants with five constraints.

	Capacity (C)	Open Route (O)	Backhaul (B)	Duration Limit (L)	Time Window (TW)
CVRP	✓				
OVRP	✓	✓			
VRPB	✓		✓		
VRPL	✓			✓	
VRPTW	✓				✓
OVRPTW	✓	✓			✓
OVRPB	✓	✓	✓		
OVRPL	✓	✓		✓	
VRPBL	✓		✓	✓	
VRPBTW	✓		✓		✓
VRPLTW	✓			✓	✓
OVRPBL	✓	✓	✓	✓	
OVRPBTW	✓	✓	✓		✓
OVRPLTW	✓	✓		✓	✓
VRPBLTW	✓		✓	✓	✓
OVRPBLTW	✓	✓	✓	✓	✓

C.2 Multi-Task VRP Solver with MoEs

We now present more details of the multi-task VRP solver with MoEs (MVMoE). We consider POMO [12] as the backbone model, which is one of the most prevalent autoregressive construction-based neural solvers in VRPs.

Encoder. Given a VRP instance with n nodes, the static features of the $i_{\text{th}}(i \in [0, n])$ node are $\mathcal{S}_i = \{y_i, \delta_i, e_i, l_i\}$, where we use y_i, δ_i, e_i, l_i to denote the coordinate, node demand, start and end time of the time window, respectively. A linear layer is first used to project the raw features to d-dimensional (i.e., $d = 128$) node embeddings $h_i^0 = \text{Linear}([y_i, \delta_i, e_i, l_i])$. Then, a stack of $N = 6$ encoder layers is applied to extract the refined node embeddings h_i^N. Specifically, as shown in Fig. 5.2, each encoder layer consists of a multi-head attention (MHA) layer and a feed forward network (FFN). Following the Transformer architecture [9], their outputs are further processed by a skip-connection and instance normalization (IN),

$$\tilde{h}_i = \text{IN}(h_i^\ell + \text{MHA}(h_i^\ell)), \tag{C.1}$$

$$h_i^{\ell+1} = \text{IN}(\tilde{h}_i + \text{FFN}(\tilde{h}_i)). \tag{C.2}$$

Below, we detail the MHA and FFN layers. 1) *MHA:* the MHA layer in the encoder employs a multi-head self-attention mechanism (i.e., with $A = 8$ heads) to compute attention weights between each two nodes. Formally, we define dimensions d_k and d_v (i.e., $d_k = d_v = d/A = 16$), and compute query $q_i^{\ell,a} \in \mathbb{R}^{d_k}$, key $k_i^{\ell,a} \in \mathbb{R}^{d_k}$, and

value $v_i^{\ell,a} \in \mathbb{R}^{d_v}$ as follows,

$$q_i^{\ell,a} = W_Q^{\ell,a} h_i^{\ell}, \quad k_i^{\ell,a} = W_K^{\ell,a} h_i^{\ell}, \quad v_i^{\ell,a} = W_V^{\ell,a} h_i^{\ell}, \tag{C.3}$$

where $W_Q^{\ell,a}, W_K^{\ell,a}, W_V^{\ell,a}$ are the parameter matrices of the $a_{\text{th}}(a \in [1, A])$ attention head in the $\ell_{\text{th}}(\ell \in [0, N-1])$ encoder layer. Then, the attention weight $u_{ij}^{\ell,a}$ is computed using the Softmax function to represent the correlation between the i_{th} node and the j_{th} node as follows,

$$u_{ij}^{\ell,a} = \text{Softmax}\left(\frac{(q_i^{\ell,a})^T k_j^{\ell,a}}{\sqrt{d_k}}\right). \tag{C.4}$$

By weighted message passing among nodes and aggregating outputs from multiple heads, the final MHA output for the i_{th} node at the ℓ_{th} encoder layer is obtained as follows,

$$h_i^{\ell,a} = \sum_{j=0}^{n} u_{ij}^{\ell,a} v_j^{\ell,a}, \tag{C.5}$$

$$\text{MHA}(h_i^{\ell}) = [h_i^{\ell,1}, h_i^{\ell,2}, \ldots, h_i^{\ell,A}] W_O^{\ell}, \tag{C.6}$$

where W_O^{ℓ} is the learnable parameter and $[,]$ is the concatenate operator. 2) *FFN:* the FFN layer computes node-wise projections using a hidden sublayer with the dimension $d_f = 512$ and a ReLU activation as follows,

$$\text{FFN}(\tilde{h}_i) = W_F^{\ell,1} \cdot \text{ReLU}(W_F^{\ell,0} \tilde{h}_i + b_F^{\ell,0}) + b_F^{\ell,1}, \tag{C.7}$$

where $W_F^{\ell,0}, W_F^{\ell,1}, b_F^{\ell,0}, b_F^{\ell,1}$ are learnable parameters of the FFN in the ℓ_{th} layer. In this paper, we replace the FFN in each encoder layer with an MoE layer, which consists of m FFNs $\{\text{FFN}_1, \ldots, \text{FFN}_m\}$ and a gating network G, such that Eq. (C.2) is rewritten as,

$$h_i^{\ell+1} = \text{IN}(\tilde{h}_i + \sum_{j=1}^{m} G(\tilde{h}_i)_j \text{FFN}_j(\tilde{h}_i)). \tag{C.8}$$

Decoder. The decoder takes all node embeddings $\{h_i^N\}_{i=0}^n$ and a context embedding h_c as inputs. At the t_{th} decoding step, the context embedding $h_c = [h_{\tau_{t-1}}^N, \mathcal{D}_t] \in \mathbb{R}^{d+4}$ includes the embedding of the last selected node $h_{\tau_{t-1}}^N$ (initialized as the depot node h_0^N) and dynamic features $\mathcal{D}_t = \{c_t, t_t, l_t, o_t\}$, where c_t, t_t, l_t, o_t

represent the remaining capacity of the vehicle, the current time, the length of the current partial route, and the presence indicator of the open route, respectively. Next, we compute an updated context embedding h'_c through a MHA layer. Note that the MHA layer in the decoder does not employ the self-attention. Concretely, the context embedding h_c is used for computing query, while the node embeddings h_i^N are used for computing the key and value,

$$q_c^{D,a} = W_Q^{D,a} h_c, \quad k_i^{D,a} = W_K^{D,a} h_i^N, \quad v_i^{D,a} = W_V^{D,a} h_i^N, \tag{C.9}$$

where $W_Q^{D,a}, W_K^{D,a}, W_V^{D,a}$ are the parameter matrices of the a_{th} attention head in the decoder. Then, we follow Eqs. (C.4)-(C.6) to obtain the updated context embedding h'_c. Note that invalid nodes (i.e., appending them to the current partial solution may violate the problem-specific feasibility constraints) are masked such that their attention weights are set to 0 in Eq. (C.4). Finally, the probabilities p_i of valid nodes to be selected is calculated by a single-head attention as follows,

$$p_i = \text{Softmax}\left(C \cdot \tanh\left(\frac{(h'_c)^T h_i^N}{\sqrt{d_k}}\right)\right), \tag{C.10}$$

where C is typically set to 10 to clip the logits so as to benefit the policy exploration [11, 18]. In this paper, we replace the final linear layer of MHA (i.e., W_O^ℓ in Eq. (C.6)) in the decoder with an MoE layer, which consists of m linear layers with parameters $\{W_O^{\ell,1}, \ldots, W_O^{\ell,m}\}$ and a gating network G. In this case, Eq. (C.6) is rewritten as:

$$\text{MHA}(h_i^\ell) = \sum_{j=1}^{m} G([h_i^{\ell,1}, h_i^{\ell,2}, \ldots, h_i^{\ell,A}])_j [h_i^{\ell,1}, h_i^{\ell,2}, \ldots, h_i^{\ell,A}] W_O^{\ell,j}. \tag{C.11}$$

Gating Algorithms. We present more details of the popular gating algorithms [147, 149] in MoEs. Without loss of generality, we take the node-level gating as an example. Suppose that we have m experts in an MoE layer, and its input has a shape of $X \in \mathbb{R}^{I \times d}$, where I is the total number of nodes in a batch of inputs. The gating network G takes X as inputs and outputs the score matrix $H = (X \cdot W_G) \in \mathbb{R}^{I \times m}$, where W_G are trainable parameters of G.

Input-Choice Gating. Each node selects Top$K$ experts based on the score matrix H in the input-choice gating. The Softmax function is applied along the last

dimension of H to obtain the probability matrix P, where $P[i, j]$ denotes the probability of the j_{th} expert being selected by the i_{th} node. However, this method does not inherently ensure load balancing. For example, an expert may receive significantly more nodes than others, resulting in a dominant expert while leaving others underfitting. Following Shazeer et al. [147], we use the noisy Top$K$ gating with the importance & load loss to enforce a similar number of nodes received by each expert. Concretely, a learnable Gaussian noise is added to H before the Softmax,

$$H' = H + \text{StandardNormal}() \cdot \text{Softplus}(X \cdot W_{noise}), \tag{C.12}$$

$$P = \text{Softmax}\left(\text{Top}K(H')\right), \tag{C.13}$$

where $W_{noise} \in \mathbb{R}^{d \times m}$ is a learnable matrix. An example of Python code of Eq. (C.12) is shown below.

```
# An example of Python Code of Eq. (15)
H = x @ w_gate  # w_gate is the parameter of the gating network G
noise_stddev = torch.nn.functional.softplus(x @ w_noise) + 1e-2
noisy_scores = H + (torch.randn_like(H) * noise_stddev)  # H'
```

For load-balancing purposes, the goal is to ensure that each expert receives a roughly equal number of nodes. However, the number of received nodes is discrete, and hence cannot be used as the loss information to update the network through backpropagation. Shazeer et al. [147] proposes to use a smooth estimator to approximate the number of nodes assigned to each expert. The above noise term helps with load balancing by enabling the gradient backpropagation through the smooth estimator. Then, an importance & load loss is used as the auxiliary loss $\mathcal{L}_b$ to enforce load balancing as follows,

$$Importance(X) = \sum_{x \in X} G(x), \tag{C.14}$$

$$Load(X)_j = \sum_{x \in X} \Phi\left(\frac{(x \cdot W_G)_j - \varphi(H'_x, k, j)}{\text{Softplus}((x \cdot W_{noise})_j)}\right), \tag{C.15}$$

$$\mathcal{L}_b = CV(Importance(X))^2 + CV(Load(X))^2, \tag{C.16}$$

where $Importance(X)$ and $Load(X)$ are vectors with the shape of $\mathbb{R}^m$; Φ denotes the cumulative distribution function of the standard normal distribution; $\varphi(a, b, c)$ denotes the b_{th} largest component of a excluding component c; H'_x denotes the

vector corresponding to the node x in H'; CV denotes the coefficient of variation. Specifically, Eq. (C.14) encourages all experts to have equal importance (i.e., the batchwise sum of gate values), and Eq. (C.15) ensures balanced loads among experts. We set the weight of the auxiliary loss (i.e., α in Eq. (5.3)) as 0.01. We refer more details to Shazeer et al. [147].

Expert-Choice Gating. In contrast to the input-choice gating, each expert independently selects Top$K$ nodes based on the score matrix H in the expert-choice gating. The Softmax function is applied along the first dimension of H to obtain P, where $P[i, j]$ denotes the probability of the i_{th} node being selected by the j_{th} expert. It explicitly guarantees load balancing and enables a flexible allocation of computation (e.g., a node can be distributed to various number of experts). In general, K is set to $\frac{I \times \beta}{m}$, where β is the capacity factor, representing on average how many experts are utilized by a node. We set $\beta = 2$ in our experiments. Note that we do not limit the maximum number of experts for each node, since it needs to solve an extra linear programming problem [149] and hence increases the computation.

Random Gating. It is a simple heuristic that randomly route inputs to experts. For example, in the problem-level gating, we randomly route a batch of inputs to K experts, similar to Zuo et al. [146].

Gating Levels. In addition to the node-level gating presented in Section 5.2, we further investigate another two gating levels in VRPs. The detailed empirical results and analyses can be found in Section 5.3.

Instance-Level Gating. The instance-level gating routes inputs at the granularity of instances. Given a batch of inputs $X \in \mathbb{R}^{B \times n \times d}$, all n nodes belong to the same instance are routed to the same expert(s). Before forward passing X to the gating network, we apply the mean pooling operator along the second dimension of X, such that the score matrix has the shape of $H \in \mathbb{R}^{B \times m}$. Similar to the node-level gating, the input-choice and expert-choice gating algorithms are applicable to the instance-level gating as well. In contrast to the node-level gating, x_i denotes the i_{th} instance rather than a single node in Fig. 5.3. Typically, the instance-level gating can save the gating computation since $B \ll I$ as the problem scale n increases. However, its empirical performance may be slightly inferior to that of the node-level gating based on our empirical results.

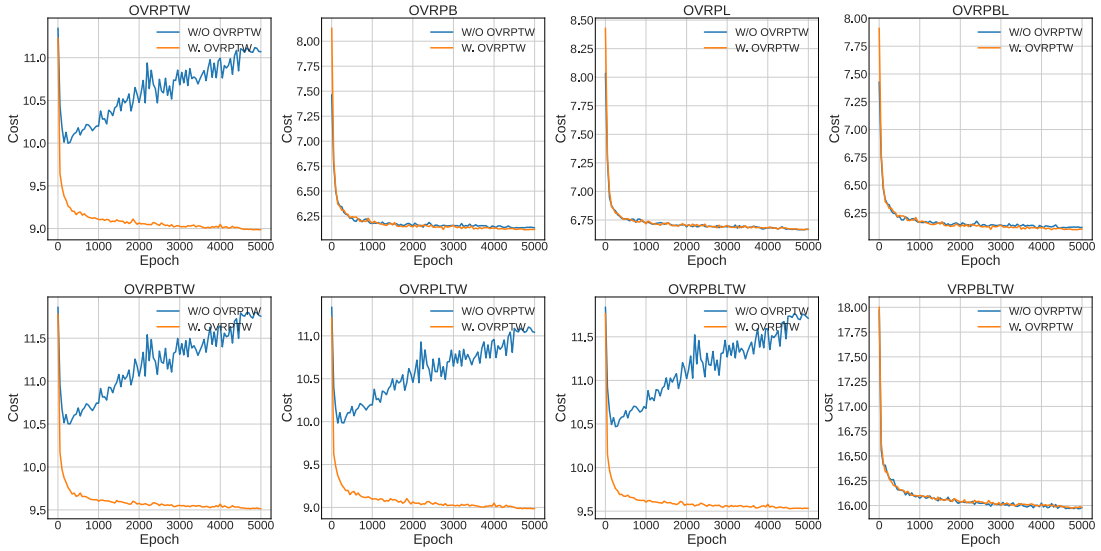


FIGURE C.1: The validation curves of POMO-MTL trained w/o OVRPTW and w. OVRPTW on $n = 50$.

Problem-Level Gating. The problem-level gating views a batch of instances from the same VRP variant independently, and hence we route a batch of inputs X to the same expert(s). Different from the node-level or instance-level gating, the input-choice and expert-choice gating algorithms are not applicable to the problem-level gating. Potential gating algorithms include randomly routing X to K (e.g., 1 or 2) experts [146] or adapting the input-choice gating without extra auxiliary losses. Although the problem-level gating is simple and efficient, it cannot guarantee all experts being sufficiently trained, since only K experts are optimized in each training step.

C.3 Additional Experiments

Training Problem Set. As mentioned in Section 5.3, our training problem set includes CVRP, OVRP, VRPB, VRPL, VRPTW and OVRPTW. In contrast to our setting, the training problem set in Liu et al. [74] does not include OVRPTW. We would like to explain this difference from two perspectives: 1) *different generation of time window:* we follow Li et al. [83] whose data generation is similar to the procedure as in Solomon [156], while Liu et al. [74] follows an artificial generation process. Since Solomon dataset is based on practical constraints and real-life data, we believe its generation of time window is more realistic and challenging;

TABLE C.2: The results of POMO trained on each problem with $n = 100$.

	CVRP	OVRP	VRPB	VRPL	VRPTW	OVRPTW	Average
POMO-CVRP	**1.488%**	20.124%	5.130%	0.105%	31.593%	42.452%	16.815%
POMO-OVRP	11.698%	**2.192%**	17.877%	10.152%	38.001%	27.734%	17.942%
POMO-VRPB	2.422%	20.902%	**0.995%**	1.050%	33.739%	42.849%	16.993%
POMO-VRPL	1.490%	19.606%	5.849%	**0.093%**	30.382%	41.164%	16.431%
POMO-VRPTW	43.850%	80.905%	79.753%	37.869%	**4.307%**	20.832%	44.586%
POMO-OVRPTW	33.839%	58.329%	56.245%	30.257%	23.518%	**2.467%**	34.109%

2) *O meets with TW:* we empirically observe that the zero-shot generalization performance of POMO-MTL trained without OVRPTW becomes worse on some VRP variants, which have open route and time window constraints concurrently. Therefore, we further include OVRPTW into our training problem set to solve the challenge. The comprehensive validation curves are shown in Fig. C.1. We assume this empirical observation is attributed to the different data generation of time window, since it is the only difference between our settings and Liu et al. [74]. Note that we retrain all neural solvers following our training setups for a fair experimental comparison.

Effect of Constraints. We empirically find the time window constraint may have a bigger effect on the generalization performance. We show the result of POMO trained on each problem with $n = 100$ in Table C.2, where we observe the average performance of POMO-VRPTW is the worst, demonstrating that the model is easier to overfit to the time window. Note that the data generation process may also affect the generalization. Modifying the data generation of time window may significantly affect the zero-shot generalization performance of the trained model on specific VRP variants.

Learned Gating Policy. Hierarchical Gating. Here, we give a simple illustration of the learned policy of the first gating network G_1 in MVMoE/4E-L. In specific, in Fig. C.2, we show the gating decision of G_1, and the percentage of decoding steps which route inputs to the dense or sparse layer, on all 16 VRP variants during inference. Since we use the problem-level gating in the first stage, it is expected that G_1 explores various gating policies for different VRPs, which can be justified by the statistics in Fig. C.2, in order to obtain diverse solution construction policies. Note that the interpretability of the learned gating policy is still a challenging task in the literature of MoEs [157, 158]. Since this is an early

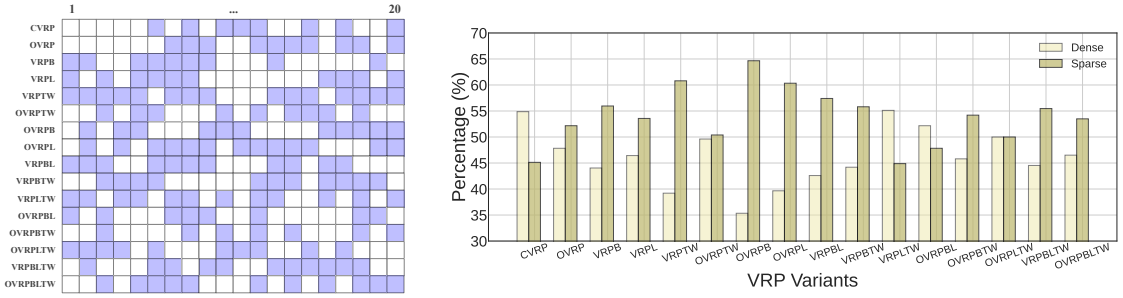


FIGURE C.2: *Left panel:* An illustration of the first gating network's decision in MVMoE/4E-L. *Right panel:* The percentage of decoding steps which route inputs to the dense or sparse layer by the first gating network.

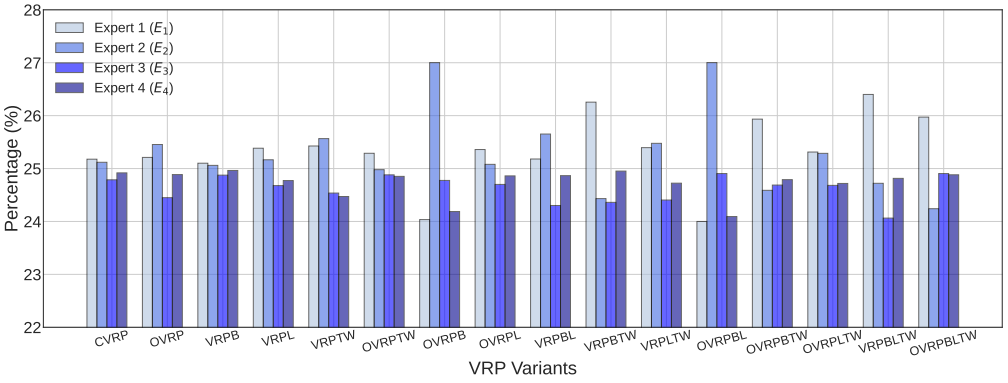


FIGURE C.3: The average percentage of nodes being routed to each expert on 16 VRP Variants.

work of applying MoEs in VRPs, we concentrate on the empirical performance and leave their interpretability to the future work.

Expert Load. We expect the learned gating policy can ensure load balancing, such that each expert is sufficiently trained. Below, we take MVMoE/4E as an example, and show the load of each expert on 16 VRP variants during inference. Concretely, the load is calculated as the average percentage of nodes being routed to each expert. Based on the results in Fig. C.3, we find that 1) the loads are well-balanced among 4 experts on 6 trained VRPs; 2) the learned gating policy can generalize to unseen 10 VRPs, with only slight unbalance (e.g., maximum 27% nodes are routed to one expert).

Sensitivity Analyses. In addition to the extensive studies on MoE settings (see Section 5.3), we further conduct sensitivity analyses on hyperparameters, including the auxiliary loss weight α, normalization layer, and K (representing how many experts are leveraged to process each input). For the training efficiency, we follow

the training setups used in Section 5.3, where the number of training epochs is halved on $n = 50$. The detailed results can be found in Table C.3, where we show the average performance of MVMoE/4E with different hyperparameters on the 6 trained VRPs and 10 unseen VRPs, respectively. Based on the empirical results, we find that further tuning hyperparameters (e.g., normalization layer) may boost the final performance. However, for a fair comparison with baselines, we follow their original setups [12, 74] by taking the first line as the default setting.

TABLE C.3: Sensitivity Analyses on hyperparameters.

α	Normalization Layer	K	Trained VRPs	Unseen VRPs
0.01	Instance Norm	2	2.171%	6.153%
0.001	Instance Norm	2	2.156%	6.091%
0.1	Instance Norm	2	2.338%	6.438%
1.0	Instance Norm	2	2.308%	6.406%
0.01	Batch Norm	2	11.560%	18.821%
0.01	Layer Norm	2	2.048%	5.973%
0.01	None	2	**2.033%**	**5.842%**
0.01	Instance Norm	1	2.610%	6.751%
0.01	Instance Norm	3	2.126%	6.109%

Benchmark Performance. We evaluate all neural solvers on CVRPLIB benchmark dataset, including CVRP and VRPTW instances with various problem sizes and attribute distributions. We mainly consider the classic Set-X [126] and Set-Solomon [156]. Note that all neural solvers are only trained on the simple uniformly distributed instances with the size $n = 100$ (i.e., the customer nodes' locations follow the uniform distribution). Greedy rollout is used by default, except for the last two columns (w. 100 Random Re-Construct (RRC) iterations) in Table C.5. The results are shown in Tables C.4 and C.5, where we observe 1) the single task training method (i.e., POMO) may overfit to the uniformly distributed data, and hence its out-of-distribution (OOD) generalization performance is worst; 2) the OOD generalization capability of our sparse models is generally stronger than the dense model counterpart POMO-MTL, but the superiority tends to vanish on large-scale instances; 3) surprisingly, MVMoE/4E-L performs better than MVMoE/4E, demonstrating more favourable potential of the proposed hierarchical gating in promoting the OOD generalization capability of sparse MoE-based models.

TABLE C.4: Results on CVRPLIB instances, including Set-X on CVRP and Set-Solomon on VRPTW.

Set-X		POMO		POMO-MTL		MVMoE/4E		MVMoE/4E-L		Set-Solomon		POMO		POMO-MTL		MVMoE/4E		MVMoE/4E-L	
Instance	Opt.	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Instance	Opt.	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap
X-n101-k25	27591	30138	9.231%	32482	17.727%	29361	6.415%	**29015**	**5.161%**	R101	1637.7	1805.6	10.252%	1821.2	11.205%	1798.1	9.794%	**1730.1**	**5.641%**
X-n106-k14	26362	39322	49.162%	27369	3.820%	27278	3.475%	**27242**	**3.338%**	R102	1466.6	**1556.7**	**6.143%**	1596.0	8.823%	1572.0	7.187%	1574.3	7.345%
X-n110-k13	14971	15223	1.683%	15151	1.202%	**15089**	**0.788%**	15196	1.503%	R103	1208.7	1341.4	10.979%	**1327.3**	**9.812%**	1328.2	9.887%	1359.4	12.470%
X-n115-k10	12747	16113	26.406%	14785	15.988%	13847	8.629%	**13325**	**4.534%**	R104	971.5	1118.6	15.142%	1120.7	15.358%	1124.8	15.780%	**1098.8**	**13.100%**
X-n120-k6	13332	14085	5.648%	13931	4.493%	14089	5.678%	**13833**	**3.758%**	R105	1355.3	1506.4	11.149%	1514.6	11.754%	1479.4	9.157%	**1456.0**	**7.433%**
X-n125-k30	55539	**58513**	**5.355%**	60687	9.269%	58944	6.131%	58603	5.517%	R106	1234.6	1365.2	10.578%	1380.5	11.818%	1362.4	10.352%	**1353.5**	**9.627%**
X-n129-k18	28940	**29246**	**1.057%**	30332	4.810%	29802	2.979%	29457	1.786%	R107	1064.6	1214.2	14.052%	1209.3	13.592%	**1182.1**	**11.037%**	1196.5	12.391%
X-n134-k13	10916	**11302**	**3.536%**	11581	6.092%	11353	4.003%	11398	4.416%	R108	932.1	1058.9	13.604%	1061.8	13.915%	**1023.2**	**9.774%**	1039.1	11.481%
X-n139-k10	13590	14035	3.274%	13911	2.362%	13825	1.729%	**13800**	**1.545%**	R109	1146.9	1249.0	8.902%	1265.7	10.358%	1255.6	9.478%	**1224.3**	**6.750%**
X-n143-k7	15700	16131	2.745%	16660	6.115%	**16125**	**2.707%**	16147	2.847%	R110	1068.0	1180.4	10.524%	1171.4	9.682%	1185.7	11.021%	**1160.2**	**8.635%**
X-n148-k46	43448	49328	13.533%	50782	16.880%	46758	7.618%	**45599**	**4.951%**	R111	1048.7	1177.2	12.253%	1211.5	15.524%	**1176.1**	**12.148%**	1197.8	14.220%
X-n153-k22	21220	32476	53.040%	26237	23.643%	23793	12.125%	**23316**	**9.877%**	R112	948.6	1063.1	12.070%	1057.0	11.427%	1045.2	10.183%	**1044.2**	**10.082%**
X-n157-k13	16876	17660	4.646%	17510	3.757%	17650	4.586%	**17410**	**3.164%**	RC101	1619.8	2643.0	63.168%	1833.3	13.181%	1774.4	9.544%	**1749.2**	**7.988%**
X-n162-k11	14138	14889	5.312%	14720	4.117%	**14654**	**3.650%**	14662	3.706%	RC102	1457.4	**1534.8**	**5.311%**	1546.1	6.086%	1544.5	5.976%	1556.1	6.771%
X-n167-k10	20557	21822	6.154%	21399	4.096%	21340	3.809%	**21275**	**3.493%**	RC103	1258.0	1407.5	11.884%	**1396.2**	**10.986%**	1402.5	11.486%	1415.3	12.502%
X-n172-k51	45607	49556	8.659%	56385	23.632%	51292	12.465%	**49073**	**7.600%**	RC104	1132.3	**1261.8**	**11.437%**	1271.7	12.311%	1265.4	11.755%	1264.2	11.649%
X-n176-k26	47812	54197	13.354%	57637	20.549%	55520	16.121%	**52727**	**10.280%**	RC105	1513.7	**1612.9**	**6.553%**	1644.9	8.668%	1635.5	8.047%	1619.4	6.980%
X-n181-k23	25569	37311	45.923%	**26219**	**2.542%**	26258	2.695%	26241	2.628%	RC106	1372.7	1539.3	12.137%	1552.8	13.120%	**1505.0**	**9.638%**	1509.5	9.968%
X-n186-k15	24145	25222	4.461%	25000	3.541%	25182	4.295%	**24836**	**2.862%**	RC107	1207.8	1347.7	11.583%	1384.8	14.655%	1351.6	11.906%	**1324.1**	**9.625%**
X-n190-k8	16980	18315	7.862%	**18113**	**6.673%**	18327	7.933%	**18113**	**6.673%**	RC108	1114.2	1305.5	17.169%	1274.4	14.378%	1254.2	12.565%	**1247.2**	**11.939%**
X-n195-k51	44225	49158	11.154%	54090	22.306%	49984	13.022%	**48185**	**8.954%**	RC201	1261.8	2045.6	62.118%	1761.1	39.570%	1577.3	25.004%	**1517.8**	**20.285%**
X-n200-k36	58578	64618	10.311%	61654	5.251%	61530	5.039%	**61483**	**4.959%**	RC202	1092.3	1805.1	65.257%	1486.2	36.062%	1616.5	47.990%	**1480.3**	**35.520%**
X-n209-k16	30656	32212	5.076%	**32011**	**4.420%**	32033	4.492%	32055	4.564%	RC203	923.7	1470.4	59.186%	**1360.4**	**47.277%**	1473.5	59.521%	1479.6	60.182%
X-n219-k73	117595	133545	13.564%	**119887**	**1.949%**	121046	2.935%	120421	2.403%	RC204	783.5	1323.9	68.973%	1331.7	69.968%	1286.6	64.212%	**1232.8**	**57.342%**
X-n228-k23	25742	48689	89.142%	33091	28.549%	31054	20.636%	**28561**	**10.951%**	RC205	1154.0	1568.4	35.910%	1539.2	33.380%	1537.7	33.250%	**1440.8**	**24.850%**
X-n237-k14	27042	29893	10.543%	**28472**	**5.288%**	28550	5.577%	28486	5.340%	RC206	1051.1	1707.5	62.449%	1472.6	40.101%	1468.9	39.749%	**1394.5**	**32.671%**
X-n247-k50	37274	56167	50.687%	45065	20.902%	43673	17.167%	**41800**	**12.143%**	RC207	962.9	1567.2	62.758%	1375.7	42.870%	1442.0	49.756%	**1346.4**	**39.831%**
X-n251-k28	38684	**40263**	**4.082%**	40614	4.989%	41022	6.044%	40822	5.527%	RC208	776.1	1505.4	93.970%	1185.6	52.764%	**1107.4**	**42.688%**	1167.5	50.437%
Avg. Gap		16.629%		9.820%		6.884%		**5.160%**		Avg. Gap		28.054%		21.380%		20.317%		**18.490%**	

TABLE C.5: Results on large-scale CVRPLIB instances.

Set-X		POMO		POMO-MTL		MVMoE/4E		MVMoE/4E-L		LEHD		LEHD/4E-L		LEHD RRC100		LEHD/4E-L RRC100	
Instance	Opt.	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap
X-n502-k39	69226	75617	9.232%	77284	11.640%	73533	6.222%	74429	7.516%	71438	3.195%	71984	3.984%	70678	2.097%	70304	1.557%
X-n513-k21	24201	30518	26.102%	28510	17.805%	32102	32.647%	31231	29.048%	25624	5.880%	25810	6.648%	24808	2.508%	25026	3.409%
X-n524-k153	154593	201877	30.586%	192249	24.358%	186540	20.665%	182392	17.982%	280556	81.480%	198498	28.400%	194569	25.859%	188040	21.636%
X-n536-k96	94846	106073	11.837%	106514	12.302%	109581	15.536%	108543	14.441%	103785	9.425%	104750	10.442%	102098	7.646%	102414	7.979%
X-n548-k50	86700	103093	18.908%	94562	9.068%	95894	10.604%	95917	10.631%	90644	4.549%	88779	2.398%	88161	1.685%	88383	1.941%
X-n561-k42	42717	49370	15.575%	47846	12.007%	56008	31.114%	51810	21.287%	44728	4.708%	44509	4.195%	43890	2.746%	43847	2.645%
X-n573-k30	50673	83545	64.871%	60913	20.208%	59473	17.366%	57042	12.569%	53482	5.543%	54981	8.502%	53222	5.030%	53309	5.202%
X-n586-k159	190316	229887	20.792%	208893	9.761%	215668	13.321%	214577	12.748%	232867	22.358%	217229	14.141%	211415	11.086%	209941	10.312%
X-n599-k92	108451	150572	38.839%	120333	10.956%	128949	18.901%	125279	15.517%	115377	6.386%	117192	8.060%	112742	3.957%	113914	5.037%
X-n613-k62	59535	68451	14.976%	67984	14.192%	82586	38.718%	74945	25.884%	62484	4.953%	62548	5.061%	61843	3.877%	62191	4.461%
X-n627-k43	62164	84434	35.825%	73060	17.528%	70987	14.193%	70905	14.061%	67568	8.693%	66779	7.424%	65509	5.381%	65117	4.750%
X-n641-k35	63682	75573	18.672%	72643	14.071%	75329	18.289%	72655	14.090%	68249	7.172%	67355	5.768%	66053	3.723%	66196	3.948%
X-n655-k131	106780	127211	19.134%	116988	9.560%	117678	10.206%	118475	10.952%	117532	10.069%	113165	5.980%	112228	5.102%	110707	3.678%
X-n670-k130	146332	208079	42.197%	190118	29.922%	197695	35.100%	183447	25.364%	220927	50.977%	183681	25.523%	184934	26.380%	179615	22.745%
X-n685-k75	68205	79482	16.534%	80892	18.601%	97388	42.787%	89441	31.136%	72946	6.951%	73783	8.178%	71635	5.029%	71723	5.158%
X-n701-k44	81923	97843	19.433%	92075	12.392%	98469	20.197%	94924	15.870%	86327	5.376%	85860	4.806%	84330	2.938%	83965	2.493%
X-n716-k35	43373	51381	18.463%	52709	21.525%	56773	30.895%	52305	20.593%	46502	7.214%	47517	9.554%	45655	5.261%	46028	6.121%
X-n733-k159	136187	159098	16.823%	161961	18.925%	178322	30.939%	167477	22.976%	149115	9.493%	150814	10.740%	144934	6.423%	146338	7.454%
X-n749-k98	77269	87786	13.611%	90582	17.229%	100438	29.985%	94497	22.296%	83439	7.985%	83951	8.648%	81801	5.865%	82346	6.571%
X-n766-k71	114417	135464	18.395%	144041	25.891%	152352	33.155%	136255	19.086%	131487	14.919%	130899	14.405%	126293	10.380%	130511	14.066%
X-n783-k48	72386	90289	24.733%	83169	14.897%	100383	38.677%	92960	28.423%	76766	6.051%	77698	7.338%	75611	4.455%	76014	5.012%
X-n801-k40	73305	124278	69.536%	85077	16.059%	91560	24.903%	87662	19.585%	77546	5.785%	78187	6.660%	75453	2.930%	75318	2.746%
X-n819-k171	158121	193451	22.344%	177157	12.039%	183599	16.113%	185832	17.525%	178558	12.925%	181075	14.517%	171025	8.161%	172779	9.270%
X-n837-k142	193737	237884	22.787%	214207	10.566%	229526	18.473%	221286	14.220%	207709	7.212%	208760	7.754%	203657	5.120%	203254	4.912%
X-n856-k95	88965	152528	71.447%	101774	14.398%	99129	11.425%	106816	20.065%	92936	4.464%	93542	5.145%	91221	2.536%	90878	2.150%
X-n876-k59	99299	119764	20.609%	116617	17.440%	119619	20.463%	114333	15.140%	104183	4.918%	106866	7.620%	103465	4.195%	104399	5.136%
X-n895-k37	53860	70245	30.421%	65587	21.773%	79018	46.710%	64310	19.402%	58028	7.739%	58157	7.978%	56481	4.866%	56749	5.364%
X-n916-k207	329179	399372	21.324%	361719	9.885%	383681	16.557%	374016	13.621%	385208	17.021%	363744	10.500%	355168	7.895%	355635	8.037%
X-n936-k151	132715	237625	79.049%	186262	40.347%	220926	66.466%	190407	43.471%	196547	48.097%	172568	30.029%	169696	27.865%	163087	22.885%
X-n957-k87	85465	130850	53.104%	98198	14.898%	113882	33.250%	105629	23.593%	90295	5.651%	89334	4.527%	88187	3.185%	88384	3.415%
X-n979-k58	118976	147687	24.132%	138092	16.067%	146347	23.005%	139682	17.404%	127972	7.561%	130662	9.822%	125137	5.178%	126113	5.999%
X-n1001-k43	72355	100399	38.759%	87660	21.153%	114448	58.176%	94734	30.929%	76689	5.990%	75933	4.945%	75742	4.681%	75403	4.213%
Avg. Gap		29.658%		**16.796%**		26.408%		19.607%		12.836%		**9.678%**		7.001%		**6.884%**	

List of Publications

(# **Equal Contribution**)

- [115] **Jianan Zhou**, Yaoxin Wu, Zhiguang Cao, Wen Song, Jie Zhang, Zhiqi Shen. Collaboration! Towards Robust Neural Methods for Routing Problems. 38th Conference on Neural Information Processing Systems (**NeurIPS**), 2024.
- [139] **Jianan Zhou**, Zhiguang Cao, Yaoxin Wu, Wen Song, Yining Ma, Jie Zhang, Xu Chi. MVMoE: Multi-Task Vehicle Routing Solver with Mixture-of-Experts. 41st International Conference on Machine Learning (**ICML**), 2024.
- [127] **Jianan Zhou**, Yaoxin Wu, Wen Song, Zhiguang Cao, Jie Zhang. Towards Omni-generalizable Neural Methods for Vehicle Routing Problems. 40th International Conference on Machine Learning (**ICML**), 2023.
- [159] **Jianan Zhou**, Yaoxin Wu, Zhiguang Cao, Wen Song, Jie Zhang, Zhenghua Chen. Learning Large Neighborhood Search for Vehicle Routing in Airport Ground Handling. IEEE Transactions on Knowledge and Data (**TKDE**), 2023.
- [62] Ziwei Huang#, **Jianan Zhou#**, Zhiguang Cao, Yixin Xu. Rethinking Light Decoder-based Solvers for Vehicle Routing Problems. 13th International Conference on Learning Representations (**ICLR**), 2025.
- [160] Igor G. Smit#, **Jianan Zhou#**, Robbert Reijnen, Yaoxin Wu, Jian Chen, Cong Zhang, Zaharah Bukhsh, Yingqian Zhang, Wim Nuijten. Graph Neural Networks for Job Shop Scheduling Problems: A Survey. Computers & Operations Research (**COR**), 2025.

- [161] Yaoxin Wu[#], **Jianan Zhou**[#], Yunwen Xia, Xianli Zhang, Zhiguang Cao, Jie Zhang. Neural Airport Ground Handling. IEEE Transactions on Intelligent Transportation Systems (**TITS**), 2023.

- [93] Jieyi Bi, Yining Ma, **Jianan Zhou**, Wen Song, Zhiguang Cao, Yaoxin Wu, Jie Zhang. Learning to Handle Complex Constraints for Vehicle Routing Problems. 38th Conference on Neural Information Processing Systems (**NeurIPS**), 2024.

- [162] Yong Liang Goh, Zhiguang Cao, Yining Ma, **Jianan Zhou**, Mohammad Haroon Dupty, Wee Sun Lee. SHIELD: Multi-task Multi-distribution Vehicle Routing Solver with Sparsity and Hierarchy. 42nd International Conference on Machine Learning (**ICML**), 2025.

- [31] Federico Berto, Chuanbo Hua, Junyoung Park, Laurin Luttmann, Yining Ma, Fanchen Bu, Jiarui Wang, Haoran Ye, Minsu Kim, Sanghyeok Choi, Nayeli Gast Zepeda, Andr Hottung, **Jianan Zhou**, Jieyi Bi, Yu Hu, Fei Liu, Hyeonah Kim, Jiwoo Son, Haeyeon Kim, Davide Angioni, Wouter Kool, Zhiguang Cao, Jie Zhang, Kijung Shin, Cathy Wu, Sungsoo Ahn, Guojie Song, Changhyun Kwon, Lin Xie, and Jinkyoo Park. 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (**KDD**), 2025.

- [163] **Jianan Zhou**[#], Jianing Zhu[#], Jingfeng Zhang, Tongliang Liu, Gang Niu, Bo Han, Masashi Sugiyama. Adversarial Training with Complementary Labels: On the Benefit of Gradually Informative Attacks. 36th Conference on Neural Information Processing Systems (**NeurIPS**), 2022.

Bibliography

[1] IBM(2017). IBM ILOG CPLEX 12.10 User Manual IBM Crop. 2017. 1

[2] LLC Gurobi Optimization. Gurobi optimizer reference manual, 2021. URL `http://www.gurobi.com`. 1

[3] Tobias Achterberg. SCIP: solving constraint integer programs. *Mathematical Programming Computation*, 1(1):1–41, 2009. 1

[4] Keld Helsgaun. An effective implementation of the lin–kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000. 1

[5] Keld Helsgaun. An extension of the lin-kernighan-helsgaun TSP solver for constrained traveling salesman and vehicle routing problems. *Roskilde: Roskilde University*, 2017. 1, 27, 40, 43, 59, 77, 87

[6] Thibaut Vidal. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140: 105643, 2022. 1, 27, 43, 59, 77, 87

[7] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour dhorizon. *European Journal of Operational Research*, 290(2):405–421, 2020. 1

[8] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Proceedings of the 29th Conference on Neural Information Processing Systems (NeurIPS)*, pages 2692–2700, 2015. 2, 7

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st Conference on Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017. 2, 8, 93

[10] Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017. 2

[11] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019. 2, 8, 19, 21, 52, 53, 54, 59, 77, 79, 86, 87, 91, 95

[12] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. POMO: Policy optimization with multiple optima for reinforcement learning. In *the 34th Conference on Advances in Neural Information Processing Systems ((NeurIPS)*, 2020. 2, 8, 19, 21, 27, 28, 38, 43, 44, 45, 52, 53, 54, 59, 60, 77, 86, 87, 91, 93, 101

[13] Chaitanya K Joshi, Thomas Laurent, and Xavier Bresson. An efficient graph convolutional network technique for the travelling salesman problem. *arXiv preprint arXiv:1906.01227*, 2019. 2, 8, 73

[14] Zhang-Hua Fu, Kai-Bin Qiu, and Hongyuan Zha. Generalize a small pretrained model to arbitrarily large TSP instances. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 7474–7482, 2021. 2, 8

[15] Xinyun Chen and Yuandong Tian. Learning to perform local rewriting for combinatorial optimization. In *Advances in Neural Information Processing Systems*, pages 6278–6289, 2019. 2, 9

[16] Yaoxin Wu, Wen Song, Zhiguang Cao, Jie Zhang, and Andrew Lim. Learning improvement heuristics for solving routing problems. *IEEE Transactions on Neural Networks and Learning Systems*, 33(9):5057–5069, 2021. 2, 9

[17] Chaitanya K Joshi, Quentin Cappart, Louis-Martin Rousseau, Thomas Laurent, and Xavier Bresson. Learning TSP requires rethinking generalization. *arXiv preprint arXiv:2006.07054*, 2020. 3

[18] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. In *ICLR Workshop Track*, 2017. 8, 41, 76, 95

[19] Mohammadreza Nazari, Afshin Oroojlooy, Lawrence Snyder, and Martin Takác. Reinforcement learning for solving the vehicle routing problem. In *Proceedings of the 32nd Conference on Neural Information Processing Systems (NeurIPS)*, pages 9839–9849, 2018. 8

[20] Minsu Kim, Junyoung Park, and Jinkyoo Park. Sym-NCO: Leveraging symmetricity for neural combinatorial optimization. In *NeurIPS*, 2022. 8

[21] André Hottung, Yeong-Dae Kwon, and Kevin Tierney. Efficient active search for combinatorial optimization problems. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, 2022. 8, 41, 45, 77, 87, 88

[22] Liang Xin, Wen Song, Zhiguang Cao, and Jie Zhang. Multi-decoder attention model with embedding glimpse for solving vehicle routing problems. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence*, page 1204212049, 2021. 8

[23] Nathan Grinsztajn, Daniel Furelos-Blanco, Shikha Surana, Clément Bonnet, and Thomas D Barrett. Winner Takes It All: Training performant RL populations for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023. 8

[24] André Hottung, Mridul Mahajan, and Kevin Tierney. PolyNet: Learning diverse solution strategies for neural combinatorial optimization. In *International Conference on Learning Representations*, 2025. 8

[25] Darko Drakulic, Sofia Michel, Florian Mai, Arnaud Sors, and Jean-Marc Andreoli. BQ-NCO: Bisimulation quotienting for generalizable neural combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023. 8, 11, 65

[26] Fu Luo, Xi Lin, Fei Liu, Qingfu Zhang, and Zhenkun Wang. Neural combinatorial optimization with heavy decoder: Toward large scale generalization. In *Advances in Neural Information Processing Systems*, 2023. 8, 11, 65

[27] Fu Luo, Xi Lin, Yaoxin Wu, Zhenkun Wang, Tong Xialiang, Mingxuan Yuan, and Qingfu Zhang. Boosting neural combinatorial optimization for large-scale vehicle routing problems. In *International Conference on Learning Representations*, 2025. 8

[28] Yeong-Dae Kwon, Jinho Choo, Iljoo Yoon, Minah Park, Duwon Park, and Youngjune Gwon. Matrix encoding networks for neural combinatorial optimization. In *NeurIPS*, volume 34, pages 5138–5149, 2021. 8, 27, 32, 75, 78

[29] Jingwen Li, Yining Ma, Ruize Gao, Zhiguang Cao, Andrew Lim, Wen Song, and Jie Zhang. Deep reinforcement learning for solving the heterogeneous capacitated vehicle routing problem. *IEEE Transactions on Cybernetics*, 2021. 8

[30] Aigerim Bogyrbayeva, Meraryslan Meraliyev, Taukekhan Mustakhov, and Bissenbay Dauletbayev. Machine learning to solve vehicle routing problems: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 2024. 8

[31] Federico Berto, Chuanbo Hua, Junyoung Park, Laurin Luttmann, Yining Ma, Fanchen Bu, Jiarui Wang, Haoran Ye, Minsu Kim, Sanghyeok Choi, Nayeli Gast Zepeda, Andr Hottung, Jianan Zhou, Jieyi Bi, Yu Hu, Fei Liu, Hyeonah Kim, Jiwoo Son, Haeyeon Kim, Davide Angioni, Wouter Kool, Zhiguang Cao, Jie Zhang, Kijung Shin, Cathy Wu, Sungsoo Ahn, Guojie Song, Changhyun Kwon, Lin Xie, and Jinkyoo Park. RL4CO: an extensive reinforcement learning for combinatorial optimization benchmark. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025. 8, 104

[32] Xuan Wu, Di Wang, Lijie Wen, Yubin Xiao, Chunguo Wu, Yuesong Wu, Chaoyu Yu, Douglas L Maskell, and You Zhou. Neural combinatorial optimization algorithms for solving vehicle routing problems: A comprehensive survey with perspectives. *arXiv preprint arXiv:2406.00415*, 2024. 8

[33] Ruizhong Qiu, Zhiqing Sun, and Yiming Yang. DIMES: A differentiable meta solver for combinatorial optimization problems. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. 9

[34] Zhiqing Sun and Yiming Yang. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023. 9

[35] Yang Li, Jinpei Guo, Runzhong Wang, and Junchi Yan. T2T: From distribution learning in training to gradient search in testing for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023. 9

[36] Yang Li, Jinpei Guo, Runzhong Wang, Hongyuan Zha, and Junchi Yan. Fast T2T: Optimization consistency speeds up diffusion-based training-to-testing solving for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2024. 9

[37] Yimeng Min, Yiwei Bai, and Carla P Gomes. Unsupervised learning for solving the travelling salesman problem. In *Advances in Neural Information Processing Systems*, 2023. 9

[38] Yifan Xia, Xianliang Yang, Zichuan Liu, Zhihao Liu, Lei Song, and Jiang Bian. Position: Rethinking post-hoc search-based neural approaches for solving large-scale traveling salesman problems. In *International Conference on Machine Learning*, 2024. 9

[39] Benjamin Hudson, Qingbiao Li, Matthew Malencia, and Amanda Prorok. Graph neural network guided local search for the traveling salesperson problem. In *International Conference on Learning Representations*, 2022. 9

[40] Haoran Ye, Jiarui Wang, Zhiguang Cao, Helan Liang, and Yong Li. DeepACO: Neural-enhanced ant systems for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023. 9

[41] Minsu Kim, Sanghyeok Choi, Hyeonah Kim, Jiwoo Son, Jinkyoo Park, and Yoshua Bengio. Ant colony sampling with GFlownets for combinatorial optimization. In *International Conference on Artificial Intelligence and Statistics*, 2025. 9

[42] Hao Lu, Xingwen Zhang, and Shuang Yang. A learning-based iterative method for solving vehicle routing problems. In *the 8th International Conference on Learning Representations (ICLR)*, 2020. 9

[43] Michel Deudon, Pierre Cournut, Alexandre Lacoste, Yossiri Adulyasak, and Louis-Martin Rousseau. Learning heuristics for the TSP by policy gradient. In *Proceedings of the 15th International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*, pages 170–181, 2018. 9

[44] André Hottung and Kevin Tierney. Neural large neighborhood search for the capacitated vehicle routing problem. In *Proceedings of the 24th European Conference on Artificial Intelligence (ECAI)*, 2020. 9

[45] Minjun Kim, Junyoung Park, and Jinkyoo Park. Learning to CROSS exchange to solve min-max vehicle routing problems. In *The Eleventh International Conference on Learning Representations*, 2023. 9

[46] Yining Ma, Jingwen Li, Zhiguang Cao, Wen Song, Le Zhang, Zhenghua Chen, and Jing Tang. Learning to iteratively solve routing problems with dual-aspect collaborative transformer. *Advances in Neural Information Processing Systems*, 34, 2021. 9

[47] Yining Ma, Jingwen Li, Zhiguang Cao, Wen Song, Hongliang Guo, Yuejiao Gong, and Yeow Meng Chee. Efficient neural neighborhood search for pickup and delivery problems. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, pages 4776–4784, 7 2022. 9

[48] Paulo R d O Costa, Jason Rhuggenaath, Yingqian Zhang, and Alp Akcay. Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning. In *Asian Conference on Machine Learning*, pages 465–480. PMLR, 2020. 10

[49] Jingyan Sui, Shizhe Ding, Ruizhi Liu, Liming Xu, and Dongbo Bu. Learning 3-opt heuristics for traveling salesman problem via deep reinforcement learning. In *Asian Conference on Machine Learning*, pages 1301–1316. PMLR, 2021. 10

[50] Yining Ma, Zhiguang Cao, and Yeow Meng Chee. Learning to search feasible and infeasible regions of routing problems with flexible neural k-opt. *Advances in Neural Information Processing Systems*, 36:49555–49578, 2023. 10, 13

[51] Chaitanya K Joshi, Quentin Cappart, Louis-Martin Rousseau, and Thomas Laurent. Learning TSP requires rethinking generalization. In *International Conference on Principles and Practice of Constraint Programming*, 2021. 10

[52] Shengcai Liu, Yu Zhang, Ke Tang, and Xin Yao. How good is neural combinatorial optimization? *arXiv preprint arXiv:2209.10913*, 2022. 10

[53] Michal Lisicki, Arash Afkanpour, and Graham W Taylor. Evaluating curriculum learning strategies in neural combinatorial optimization. In *NeurIPS 2020 Workshop on Learning Meets Combinatorial Algorithms*, 2020. 10

[54] Sahil Manchanda, Sofia Michel, Darko Drakulic, and Jean-Marc Andreoli. On the generalization of neural combinatorial optimization heuristics. In *ECMLPKDD*, 2022. 10, 11, 35, 36, 39, 42, 43, 86, 88

[55] Minsu Kim, Jiwoo SON, Hyeonah Kim, and Jinkyoo Park. Scale-conditioned adaptation for large scale combinatorial optimization. In *NeurIPS 2022 Workshop on Distribution Shifts: Connecting Methods and Applications*, 2022. 10

[56] Ahmad Bdeir, Jonas K Falkner, and Lars Schmidt-Thieme. Attention, filling in the gaps for generalization in routing problems. In *ECMLPKDD*, 2022. 10

[57] Yan Jin, Yuandong Ding, Xuanhao Pan, Kun He, Li Zhao, Tao Qin, Lei Song, and Jiang Bian. Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 8132–8140, 2023. 11

[58] Jiwoo Son, Minsu Kim, Hyeonah Kim, and Jinkyoo Park. Meta-SAGE: Scale meta-learning scheduled adaptation with guided exploration for mitigating scale shift on combinatorial optimization. In *International Conference on Machine Learning*, pages 32194–32210. PMLR, 2023. 11

[59] Yang Wang, Ya-Hui Jia, Wei-Neng Chen, and Yi Mei. Distance-aware attention reshaping: Enhance generalization of neural solver for large-scale vehicle routing problems. *arXiv preprint arXiv:2401.06979*, 2024. 11

[60] Chengrui Gao, Haopu Shang, Ke Xue, Dong Li, and Chao Qian. Towards generalizable neural solvers for vehicle routing problems via ensemble with transferrable local policy. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, 2024. 11, 65

[61] Jingwen Li, Yining Ma, Zhiguang Cao, Yaoxin Wu, Wen Song, Jie Zhang, and Yeow Meng Chee. Learning feature embedding refiner for solving vehicle routing problems. *IEEE Transactions on Neural Networks and Learning Systems*, 2023. 11

[62] Ziwei Huang, Jianan Zhou, Zhiguang Cao, and Yixin XU. Rethinking light decoder-based solvers for vehicle routing problems. In *International Conference on Learning Representations*, 2025. 11, 103

[63] Liang Xin, Wen Song, Zhiguang Cao, and Jie Zhang. Generative adversarial training for neural combinatorial optimization models, 2022. URL `https://openreview.net/forum?id=9vsRT9mc7U`. 11

[64] Chenguang Wang, Yaodong Yang, Oliver Slumbers, Congying Han, Tiande Guo, Haifeng Zhang, and Jun Wang. A game-theoretic approach for improving generalization ability of TSP solvers. In *ICLR 2022 Workshop on Gamification and Multiagent Solutions*, 2022. 11

[65] Chenguang Wang, Zhouliang Yu, Stephen McAleer, Tianshu Yu, and Yaodong Yang. ASP: Learn a universal neural solver! *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(6):4102–4114, 2024. 11

[66] Zeyang Zhang, Ziwei Zhang, Xin Wang, and Wenwu Zhu. Learning to solve travelling salesman problem with hardness-adaptive curriculum. In *AAAI*, volume 36, pages 9136–9144, 2022. 11, 17, 21, 24, 27, 32, 74, 78, 86

[67] Yuan Jiang, Yaoxin Wu, Zhiguang Cao, and Jie Zhang. Learning to solve routing problems via distributionally robust optimization. In *AAAI*, 2022. 11

[68] Jieyi Bi, Yining Ma, Jiahai Wang, Zhiguang Cao, Jinbiao Chen, Yuan Sun, and Yeow Meng Chee. Learning generalizable models for vehicle routing problems via knowledge distillation. In *Advances in Neural Information Processing Systems*, 2022. 11, 43, 88

[69] Simon Geisler, Johanna Sommer, Jan Schuchardt, Aleksandar Bojchevski, and Stephan Günnemann. Generalization of neural combinatorial solvers through the lens of adversarial robustness. In *ICLR*, 2022. 11, 17, 20, 21, 73, 74

[70] Han Lu, Zenan Li, Runzhong Wang, Qibing Ren, Xijun Li, Mingxuan Yuan, Jia Zeng, Xiaokang Yang, and Junchi Yan. ROCO: A general framework for evaluating robustness of combinatorial optimization solvers on graphs. In *ICLR*, 2023. 11, 17, 21, 27, 32, 74, 75, 79

[71] Yuan Jiang, Zhiguang Cao, Yaoxin Wu, Wen Song, and Jie Zhang. Ensemble-based deep reinforcement learning for vehicle routing problems under distribution shift. In *Advances in Neural Information Processing Systems*, 2023. 11

[72] Chenguang Wang and Tianshu Yu. Efficient training of multi-task combinatorial neural solver with multi-armed bandits. *arXiv preprint arXiv:2305.06361*, 2023. 12, 51

[73] Zhuoyi Lin, Yaoxin Wu, Bangjian Zhou, Zhiguang Cao, Wen Song, Yingqian Zhang, and Jayavelu Senthilnath. Cross-problem learning for solving vehicle routing problems. In *IJCAI*, 2024. 12, 51

[74] Fei Liu, Xi Lin, Zhenkun Wang, Qingfu Zhang, Tong Xialiang, and Mingxuan Yuan. Multi-task learning for routing problem with cross-problem zero-shot generalization. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1898–1908, 2024. 12, 51, 52, 55, 59, 60, 92, 98, 99, 101

[75] Federico Berto, Chuanbo Hua, Nayeli Gast Zepeda, André Hottung, Niels Wouda, Leon Lan, Junyoung Park, Kevin Tierney, and Jinkyoo Park. RouteFinder: Towards foundation models for vehicle routing problems. *arXiv preprint arXiv:2406.15007*, 2024. 12

[76] Han Li, Fei Liu, Zhi Zheng, Yu Zhang, and Zhenkun Wang. CaDA: Cross-problem routing solver with constraint-aware dual-attention. *arXiv preprint arXiv:2412.00346*, 2024. 12

[77] Darko Drakulic, Sofia Michel, and Jean-Marc Andreoli. GOAL: A generalist combinatorial optimization agent learner. In *International Conference on Learning Representations*, 2025. 12

[78] Xia Jiang, Yaoxin Wu, Yuan Wang, and Yingqian Zhang. UNCO: Towards unifying neural combinatorial optimization through large language model. *arXiv preprint arXiv:2408.12214*, 2024. 12

[79] Yang Li, Jiaxi Liu, Qitian Wu, Xiaohan Qin, and Junchi Yan. Toward learning generalized cross-problem solving strategies for combinatorial optimization, 2025. URL `https://openreview.net/forum?id=VnaJNW8OpN`. 12

[80] Sirui Li, Janardhan Kulkarni, Ishai Menache, Cathy Wu, and Beibin Li. Towards foundation models for mixed integer linear programming. In *International Conference on Learning Representations*, 2025. 12

[81] Junyang Cai, Taoan Huang, and Bistra Dilkina. Multi-task representation learning for mixed integer linear programming. *arXiv preprint arXiv:2412.14409*, 2024. 12

[82] Wenzheng Pan, Hao Xiong, Jiale Ma, Wentao Zhao, Yang Li, and Junchi Yan. UniCO: On unified combinatorial optimization via problem reduction to matrix-encoded general TSP. In *International Conference on Learning Representations*, 2025. 12

[83] Sirui Li, Zhongxia Yan, and Cathy Wu. Learning to delegate for large-scale vehicle routing. In *NeurIPS*, volume 34, pages 26198–26211, 2021. 13, 43, 48, 92, 98

[84] Zefang Zong, Hansen Wang, Jingwei Wang, Meng Zheng, and Yong Li. RBG: Hierarchically solving large-scale routing problems in logistic systems via reinforcement learning. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4648–4658, 2022. 13

[85] Qingchun Hou, Jingwei Yang, Yiqiang Su, Xiaoqing Wang, and Yuming Deng. Generalize learned heuristics to solve large-scale vehicle routing problems in real-time. In *International Conference on Learning Representations*, 2023. 13

[86] Minsu Kim, Jinkyoo Park, et al. Learning collaborative policies to solve NP-hard routing problems. In *Advances in Neural Information Processing Systems*, volume 34, pages 10418–10430, 2021. 13

[87] Xuanhao Pan, Yan Jin, Yuandong Ding, Mingxiao Feng, Li Zhao, Lei Song, and Jiang Bian. H-TSP: Hierarchically solving the large-scale traveling salesman problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 9345–9353, 2023. 13

[88] Hanni Cheng, Haosi Zheng, Ya Cong, Weihao Jiang, and Shiliang Pu. Select and Optimize: Learning to solve large-scale TSP instances. In *International Conference on Artificial Intelligence and Statistics*, pages 1219–1231. PMLR, 2023. 13

[89] Haoran Ye, Jiarui Wang, Helan Liang, Zhiguang Cao, Yong Li, and Fanzhang Li. GLOP: Learning global partition and local construction for solving large-scale routing problems in real-time. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 20284–20292, 2024. 13

[90] Qiaoyue Tang, Yangzhe Kong, Lemeng Pan, and Choonmeng Lee. Learning to solve soft-constrained vehicle routing problems with lagrangian relaxation. *arXiv preprint arXiv:2207.09860*, 2022. 13

[91] Rongkai Zhang, Anatolii Prokhorchuk, and Justin Dauwels. Deep reinforcement learning for traveling salesman problem with time windows and rejections. In *International Joint Conference on Neural Networks*, pages 1–8. IEEE, 2020. 13

[92] Jingxiao Chen, Ziqin Gong, Minghuan Liu, Jun Wang, Yong Yu, and Weinan Zhang. Looking ahead to avoid being late: Solving hard-constrained traveling salesman problem. *arXiv preprint arXiv:2403.05318*, 2024. 13

[93] Jieyi Bi, Yining Ma, Jianan Zhou, Wen Song, Zhiguang Cao, Yaoxin Wu, and Jie Zhang. Learning to handle complex constraints for vehicle routing problems. In *Advances in Neural Information Processing Systems*, 2024. 13, 104

[94] Zhiqin Zhang, Yining Ma, Jingfeng Yang, Zhiguang Cao, and Hoong Chuin Lau. Unveiling neural combinatorial optimization model representations through probing, 2025. URL `https://openreview.net/forum?id=agEy9hliY1`. 14

[95] Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. Large language models for supply chain optimization. *arXiv preprint arXiv:2307.03875*, 2023. 14

[96] Daisuke Kikuta, Hiroki Ikeuchi, Kengo Tajiri, and Yuusuke Nakano. RouteExplainer: An explanation framework for vehicle routing problem. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 30–42. Springer, 2024. 14

[97] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024. 14

[98] Yuxiao Huang, Wenjie Zhang, Liang Feng, Xingyu Wu, and Kay Chen Tan. How multimodal integration boost the performance of LLM for optimization: Case study on capacitated vehicle routing problems. *arXiv preprint arXiv:2403.01757*, 2024. 14

[99] Shengcai Liu, Caishun Chen, Xinghua Qu, Ke Tang, and Yew-Soon Ong. Large language models as evolutionary optimizers. In *IEEE Congress on Evolutionary Computation*, pages 1–8. IEEE, 2024. 14

[100] Zangir Iklassov, Yali Du, Farkhad Akimov, and Martin Takáč. Self-guiding exploration for combinatorial problems. In *Advances in Neural Information Processing Systems*, 2024. 14

[101] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995): 468–475, 2024. 14

[102] Fei Liu, Xialiang Tong, Mingxuan Yuan, and Qingfu Zhang. Algorithm evolution using large language model. *arXiv preprint arXiv:2311.15249*, 2023. 14

[103] Fei Liu, Tong Xialiang, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of Heuristics: Towards efficient automatic algorithm design using large language model. In *International Conference on Machine Learning*, 2024. 14

[104] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *Advances in Neural Information Processing Systems*, 2024. 14

[105] Pham Vu Tuan Dat, Long Doan, and Huynh Thi Thanh Binh. HSEvo: Elevating automatic heuristic design with diversity-driven harmony search and genetic algorithm using LLMs. *arXiv preprint arXiv:2412.14995*, 2024. 14

[106] Zhi Zheng, Zhuoliang Xie, Zhenkun Wang, and Bryan Hooi. Monte carlo tree search for comprehensive exploration in LLM-based automatic heuristic design. *arXiv preprint arXiv:2501.08603*, 2025. 14

[107] Chaoxu Mu, Xufeng Zhang, and Hui Wang. Planning of Heuristics: Strategic planning on large language models with monte carlo tree search for automating heuristic optimization. *arXiv preprint arXiv:2502.11422*, 2025. 14

[108] Shunyu Yao, Fei Liu, Xi Lin, Zhichao Lu, Zhenkun Wang, and Qingfu Zhang. Multi-objective evolution of heuristic using large language model. *arXiv preprint arXiv:2409.16867*, 2024. 14

[109] Kai Li, Fei Liu, Zhenkun Wang, Xialiang Tong, Xiongwei Han, and Mingxuan Yuan. ARS: Automatic routing solver with large language models. *arXiv preprint arXiv:2502.15359*, 2025. 14

[110] Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Jessica Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, et al. Chain-of-experts: When LLMs meet complex operations research problems. In *International Conference on Learning Representations*, 2023. 15

[111] Ali AhmadiTeshnizi, Wenzhi Gao, and Madeleine Udell. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In *International Conference on Machine Learning*, 2024. 15

[112] Segev Wasserkrug, Leonard Boussioux, Dick den Hertog, Farzaneh Mirzazadeh, Ilker Birbil, Jannis Kurtz, and Donato Maragno. From large language models and optimization to decision optimization copilot: A research manifesto. *arXiv preprint arXiv:2402.16269*, 2024. 15

[113] Caigao JIANG, Xiang Shu, Hong Qian, Xingyu Lu, JUN ZHOU, Aimin Zhou, and Yang Yu. LLMOPT: Learning to define and solve general optimization problems from scratch. In *International Conference on Learning Representations*, 2025. 15

[114] Xia Jiang, Yaoxin Wu, Chenhao Zhang, and Yingqian Zhang. DRoC: Elevating large language models for complex vehicle routing via decomposed retrieval of constraints. In *International Conference on Learning Representations*, 2025. 15

[115] Jianan Zhou, Yaoxin Wu, Zhiguang Cao, Wen Song, Jie Zhang, and Zhiqi Shen. Collaboration! towards robust neural methods for routing problems. In *Advances in Neural Information Processing Systems*, 2024. 17, 103

[116] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *ICLR*, 2018. 17, 20, 31

[117] Dimitris Tsipras, Shibani Santurkar, Logan Engstrom, Alexander Turner, and Aleksander Madry. Robustness may be at odds with accuracy. In *ICLR*, 2019. 17

[118] Hongyang Zhang, Yaodong Yu, Jiantao Jiao, Eric Xing, Laurent El Ghaoui, and Michael Jordan. Theoretically principled trade-off between robustness and accuracy. In *ICML*, pages 7472–7482. PMLR, 2019. 17, 20, 23

[119] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992. 19, 37, 54

[120] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *ICLR*, 2015. 20, 31

[121] Aditi Raghunathan, Sang Michael Xie, Fanny Yang, John Duchi, and Percy Liang. Understanding and mitigating the tradeoff between robustness and accuracy. In *ICML*, pages 7909–7919. PMLR, 2020. 20, 23

[122] Yao-Yuan Yang, Cyrus Rashtchian, Hongyang Zhang, Russ R Salakhutdinov, and Kamalika Chaudhuri. A closer look at accuracy vs. robustness. In *NeurIPS*, volume 33, pages 8588–8601, 2020. 20

[123] Sanjay Kariyappa and Moinuddin K Qureshi. Improving adversarial robustness of ensembles with diversity training. *arXiv preprint arXiv:1901.09981*, 2019. 27, 78

[124] David Stutz, Matthias Hein, and Bernt Schiele. Disentangling adversarial robustness and generalization. In *CVPR*, pages 6976–6987, 2019. 31

[125] Gerhard Reinelt. TSPLIB: A traveling salesman problem library. *ORSA journal on computing*, 3(4):376–384, 1991. 31, 47, 88

[126] Eduardo Uchoa, Diego Pecin, Artur Pessoa, Marcus Poggi, Thibaut Vidal, and Anand Subramanian. New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 257(3): 845–858, 2017. 31, 47, 88, 101

[127] Jianan Zhou, Yaoxin Wu, Wen Song, Zhiguang Cao, and Jie Zhang. Towards omni-generalizable neural methods for vehicle routing problems. In *International Conference on Machine Learning*, 2023. 35, 103

[128] Alex Nichol, Joshua Achiam, and John Schulman. On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999*, 2018. 35, 42, 43, 83, 88

[129] David H Wolpert and William G Macready. No free lunch theorems for optimization. *IEEE transactions on evolutionary computation*, 1(1):67–82, 1997. 36

[130] Ricardo Vilalta and Youssef Drissi. A perspective view and survey of meta-learning. *Artificial intelligence review*, 18(2):77–95, 2002. 36

[131] Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. Meta-learning in neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 2021. 36

[132] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*, pages 1126–1135. PMLR, 2017. 38, 39, 42, 44, 84

[133] Aniruddh Raghu, Maithra Raghu, Samy Bengio, and Oriol Vinyals. Rapid learning or feature reuse? towards understanding the effectiveness of maml. In *ICLR*, 2020. 39

[134] Sebastian Flennerhag, Yannick Schroecker, Tom Zahavy, Hado van Hasselt, David Silver, and Satinder Singh. Bootstrapped meta-learning. In *ICLR*, 2022. 39

[135] David Applegate, Ribert Bixby, Vasek Chvatal, and William Cook. Concorde TSP solver. 2006. 43, 87

[136] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Machine Learning*, 2015. 44

[137] Jakob Bossek, Pascal Kerschke, Aneta Neumann, Markus Wagner, Frank Neumann, and Heike Trautmann. Evolving diverse TSP instances by means of novel and creative mutation operators. In *Proceedings of the 15th ACM/SIGEVO Conference on Foundations of Genetic Algorithms*, pages 58–71, 2019. 46, 79, 86

[138] Eduardo Queiroga, Ruslan Sadykov, Eduardo Uchoa, and Thibaut Vidal. 10,000 optimal CVRP solutions for testing machine learning based heuristics. In *AAAI Workshop on Machine Learning for Operations Research (ML4OR)*, 2022. 47

[139] Jianan Zhou, Zhiguang Cao, Yaoxin Wu, Wen Song, Yining Ma, Jie Zhang, and Xu Chi. MVMoE: Multi-task vehicle routing solver with mixture-of-experts. In *International Conference on Machine Learning*, 2024. 51, 103

[140] Frank Ruis, Gertjan Burghouts, and Doina Bucur. Independent prototype propagation for zero-shot compositionality. *NeurIPS*, 34:10641–10653, 2021. 51

[141] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. 51, 63

[142] Luciano Floridi and Massimo Chiriatti. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines*, 30:681–694, 2020. 51

[143] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. 51

[144] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991. 52

[145] Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural computation*, 6(2):181–214, 1994. 52

[146] Simiao Zuo, Xiaodong Liu, Jian Jiao, Young Jin Kim, Hany Hassan, Ruofei Zhang, Jianfeng Gao, and Tuo Zhao. Taming sparsely activated transformer with stochastic experts. In *ICLR*, 2022. 55, 97, 98

[147] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017. 56, 57, 59, 95, 96, 97

[148] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. BASE Layers: Simplifying training of large, sparse models. In *ICML*, pages 6265–6274. PMLR, 2021. 56

[149] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. Mixture-of-experts with expert choice routing. In *NeurIPS*, volume 35, pages 7103–7114, 2022. 57, 59, 95, 97

[150] Joan Puigcerver, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. From sparse to soft mixtures of experts. In *ICLR*, 2024. 59, 64

[151] Vincent Furnon and Laurent Perron. OR-Tools routing library, 2023. URL `https://developers.google.com/optimization/routing`. 59

[152] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. Scaling vision with sparse mixture of experts. In *NeurIPS*, volume 34, pages 8583–8595, 2021. 62

[153] William Fedus, Jeff Dean, and Barret Zoph. A review of sparse expert models in deep learning. *arXiv preprint arXiv:2209.01667*, 2022. 65

[154] Zhuolin Yang, Linyi Li, Xiaojun Xu, Shiliang Zuo, Qian Chen, Pan Zhou, Benjamin Rubinstein, Ce Zhang, and Bo Li. TRS: Transferability reduced ensemble via promoting gradient diversity and model smoothness. In *NeurIPS*, volume 34, pages 17642–17655, 2021. 80

[155] Kate Smith-Miles, Jano van Hemert, and Xin Yu Lim. Understanding TSP difficulty by learning from evolved instances. In *International conference on learning and intelligent optimization*, pages 266–280. Springer, 2010. 85

[156] Marius M Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations research*, 35(2):254–265, 1987. 92, 98, 101

[157] Zixiang Chen, Yihe Deng, Yue Wu, Quanquan Gu, and Yuanzhi Li. Towards understanding the mixture-of-experts layer in deep learning. In *NeurIPS*, volume 35, pages 23049–23062, 2022. 99

[158] Aya Abdelsalam Ismail, Sercan O Arik, Jinsung Yoon, Ankur Taly, Soheil Feizi, and Tomas Pfister. Interpretable mixture of experts. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. 99

[159] Jianan Zhou, Yaoxin Wu, Zhiguang Cao, Wen Song, Jie Zhang, and Zhenghua Chen. Learning large neighborhood search for vehicle routing in airport ground handling. *IEEE Transactions on knowledge and data engineering*, 35 (9):9769–9782, 2023. 103

[160] Igor G Smit, Jianan Zhou, Robbert Reijnen, Yaoxin Wu, Jian Chen, Cong Zhang, Zaharah Bukhsh, Yingqian Zhang, and Wim Nuijten. Graph neural networks for job shop scheduling problems: A survey. *Computers & Operations Research*, page 106914, 2024. 103

[161] Yaoxin Wu, Jianan Zhou, Yunwen Xia, Xianli Zhang, Zhiguang Cao, and Jie Zhang. Neural airport ground handling. *IEEE Transactions on Intelligent Transportation Systems*, 24(12):15652–15666, 2023. 104

[162] Yong Liang Goh, Zhiguang Cao, Yining Ma, Jianan Zhou, Mohammed Haroon Dupty, and Wee Sun Lee. SHIELD: Multi-task multi-distribution vehicle routing solver with sparsity and hierarchy. In *International Conference on Machine Learning*, 2025. 104

[163] Jianan Zhou, Jianing Zhu, Jingfeng Zhang, Tongliang Liu, Gang Niu, Bo Han, and Masashi Sugiyama. Adversarial training with complementary labels: On the benefit of gradually informative attacks. In *Advances in Neural Information Processing Systems*, 2022. 104